



Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«Московский государственный технический университет
имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ «Информатика и системы управления»

КАФЕДРА «Информационные системы и телекоммуникации»

РАСЧЕТНО-ПОЯСНИТЕЛЬНАЯ ЗАПИСКА
К ВЫПУСКНОЙ КВАЛИФИКАЦИОННОЙ РАБОТЕ
НА ТЕМУ:

«Разработка автономного узла распределенной системы акустического мониторинга и локализации источника звука»

Студент	ИУЗ-81Б		И.Е. Леонтьев
	(группа)	(подпись, дата)	(инициалы, фамилия)
Руководитель ВКР		(подпись, дата)	С.В. Федоров
			(инициалы, фамилия)
Нормоконтролер		(подпись, дата)	А.Р. Кадырбаева
			(инициалы, фамилия)

2026 год

АННОТАЦИЯ

Расчетно-пояснительная записка 95 с., 13 рис., 30 табл., 38 источников.

АКУСТИЧЕСКИЙ МОНИТОРИНГ, ЛОКАЛИЗАЦИЯ ИСТОЧНИКА ЗВУКА, TDOA, РАСПРЕДЕЛЁННЫЕ СЕНСОРНЫЕ СЕТИ, МИКРОКОНТРОЛЛЕР, ЦИФРОВОЙ МИКРОФОН, БЕСПРОВОДНАЯ ПЕРЕДАЧА ДАННЫХ

Объектом разработки является автономный узел распределённой системы акустического мониторинга и локализации источника звука.

Цель работы — разработка программно-аппаратного комплекса автономного узла для приёма аудиоданных с цифровых MEMS-микрофонов, выделения первого прихода акустического события, формирования временной метки и передачи полученной информации на сервер локализации.

Поставленная цель достигается за счёт применения микроконтроллера STM32F411CEU6 с аппаратным интерфейсом I2S для приёма аудиоданных, реализации двухступенчатого потокового детектора импульсных событий на основе адаптивного порога, отношения кратковременного и долгосрочного средних, алгоритма кумулятивной суммы и уточнения первого прихода по предтриггерному буферу, организации синхронизации времени по схеме «ведущий — ведомый» с периодической коррекцией, а также использования радиотрансиверов nRF24L01+ для беспроводного обмена между узлами и сетевого контроллера W5500 для передачи агрегированных данных на сервер локализации.

СОДЕРЖАНИЕ

АННОТАЦИЯ	5
ВВЕДЕНИЕ.....	9
1 Исследовательская часть	11
1.1 Постановка задачи	11
1.2 Обзор предметной области	12
1.3 Анализ существующих систем акустического детектирования и локализации	13
1.4 Обоснование необходимости разработки собственной системы.	16
1.5 Методы локализации источника звука	16
1.6 Выбор целевой точности локализации	17
1.7 Обзор методов детектирования событий в аудиопотоке	18
1.8 Выбор метода детектирования событий в аудиопотоке	20
1.9 Анализ способов передачи данных от измерительных модулей к агрегирующему модулю	21
1.10 Анализ способов передачи данных от агрегирующего модуля на сервер локализации	25
1.11 Анализ методов синхронизации времени в распределённых сенсорных системах	27
1.12 Разработка требований к узлу системы локализации	31
2 Конструкторская часть	33
2.1 Проектирование архитектуры распределённой системы локализации источника звука	33
2.2 Разработка структуры ведомого программно-аппаратного модуля	35
2.3 Разработка структуры ведущего программно-аппаратного модуля	39

2.4 Разработка программного модуля приёма и обработки звукового сигнала.....	42
2.5 Разработка программного модуля обнаружения импульсного акустического события.....	43
2.6 Разработка программного модуля локального времени ведомого узла.....	57
2.7 Разработка программного модуля радиопередачи временных меток ведомым программно-аппаратным модулем	59
2.8 Разработка программного модуля радиуправления ведомым узлом.....	60
2.9 Разработка программного модуля синхронизации и проверки работоспособности ведомых модулей	61
2.10 Разработка программного модуля приёма временных меток ведущим модулем.....	62
2.11 Разработка программного модуля группировки событий ведущим модулем	64
2.12 Разработка программного модуля сетевой передачи данных ведущим модулем.....	65
2.13 Объединение программных модулей ведомого модуля	66
2.14 Объединение программных модулей ведущего модуля.....	67
3 Технологическая часть	70
3.1 Разработка методики модульного тестирования программных модулей ведомого модуля	70
3.2 Разработка методики модульного тестирования программных модулей ведущего модуля.....	71
3.3 Проведение модульных тестов программных модулей ведомого модуля.....	73
3.4 Проведение модульных тестов программных модулей ведущего модуля.....	74

3.5 Разработка методики тестирования производительности ведомого модуля.....	76
3.6 Разработка методики тестирования производительности ведущего модуля.....	77
3.7 Проведение тестов производительности ведомого модуля.....	77
3.8 Проведение тестов производительности ведущего модуля	78
3.9 Тест работоспособности системы из ведущего и четырёх ведомых модулей.....	79
3.10 Тестирование системы в реальных условиях.....	80
ЗАКЛЮЧЕНИЕ	89
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	91
ПРИЛОЖЕНИЕ А. Текст разработанной программы.....	96
ПРИЛОЖЕНИЕ Б. Графическая часть выпускной квалификационной работы.....	200

ВВЕДЕНИЕ

В данной работе производится проектирование автономного узла распределённой системы акустического мониторинга и локализации источника звука. Узел предусмотрен в составе распределённой программно-аппаратной системы, предназначенной для обнаружения импульсных акустических событий, фиксации времени их прихода на пространственно разнесённые измерительные модули и последующей локализации источника звука по разности времён прихода сигнала. Разрабатываемое устройство предназначено для приёма аудиоданных с цифрового MEMS-микрофона, выделения первого прихода акустического события, формирования временной метки и передачи полученной информации на сервер локализации.

Исходным документом для проектирования является техническое задание. Проектирование проводилось с учётом основных принципов:

- автономный узел должен обеспечивать непрерывный приём и обработку аудиопотока в реальном времени без потери данных;
- программные и аппаратные модули устройства должны обладать достаточной надёжностью, устойчивостью к помехам и согласованностью временных характеристик;
- алгоритм детектирования должен обеспечивать выделение первого прихода импульсного звукового события и формирование временной метки, пригодной для последующей TDOA-локализации;
- конструируемая система должна иметь рациональную структуру, включающую ведомые измерительные узлы, ведущий модуль синхронизации и сбора данных, канал обмена и сетевой интерфейс передачи результатов;
- программная реализация должна соответствовать ограничениям выбранной микроконтроллерной платформы по быстродействию, объёму памяти и энергопотреблению;

— разрабатываемое изделие должно обеспечивать возможность модульного тестирования и отладки.

При выполнении технического проекта были разработаны: структурная схема распределённой системы акустического мониторинга, программная архитектура ведомого модуля, алгоритм детектирования первого прихода акустического события, алгоритм формирования и передачи временных меток, программное обеспечение ведущего модуля, а также методика модульного и комплексного тестирования программно-аппаратных модулей системы.

1 Исследовательская часть

1.1 Постановка задачи

Под автономным узлом системы акустического мониторинга и локализации в данной работе подразумевается программно-аппаратный комплекс, обеспечивающий регистрацию импульсных акустических событий, предварительную обработку данных, синхронизацию измерительных устройств в составе узла и передачу информации достаточной для локализации источника звука в определённом радиусе от места развёртывания узла на общий сервер локализации.

В качестве исходных ограничений проектирования рассматриваются микроконтроллерная реализация и применение доступных компонентов. Микроконтроллер STM32F411CEU6 содержит аппаратный интерфейс I2S, контроллер прямого доступа к памяти и несколько интерфейсов SPI, что позволяет принимать аудиоданные от цифрового микрофона и подключать внешние коммуникационные модули. [1]. Цифровой MEMS-микрофон INMP441 передаёт аудиоданные по интерфейсу I2S и может использоваться как источник дискретного аудиопотока для встроенной обработки. [2].

В рамках исследовательской части необходимо рассмотреть существующие системы распределённого акустического мониторинга, определить их основные архитектурные особенности, преимущества и ограничения. Также требуется выполнить обзор современных методов локализации источников звука с целью выбора подхода, применимого в разрабатываемой системе. Подробная реализация серверных алгоритмов локализации не входит в рамки данной работы, однако выбранные методы должны быть учтены при формировании требований к автономному узлу.

В результате исследования необходимо получить обоснованный набор требований к автономному узлу системы акустического мониторинга и локализации. Для выполнения данной задачи необходимо проанализировать предметную область акустического мониторинга, выявить ограничения существующих

решений, выбрать методы локализации, детектирования, обмена данными и синхронизации времени, а затем сформулировать требования, которые будут использованы в конструкторской части при проектировании структуры программно-аппаратной системы.

1.2 Обзор предметной области

Акустическое событие представляет собой ограниченный во времени фрагмент аудиосигнала, несущий информацию о физическом процессе, который его породил. В системах мониторинга наиболее важны события, которые возникают редко, имеют малую длительность и требуют быстрого реагирования. Их характерная особенность - резкое изменение энергии сигнала, широкополосный спектр и выраженный фронт атаки [3].

Задачи обработки акустических событий разделяются на три уровня: детектирование, классификация и локализация. Детектирование отвечает на вопрос о наличии события в потоке, классификация определяет тип события, а локализация оценивает направление или координаты источника [4].

Особенность автономного узла состоит в ограниченности вычислительных ресурсов и необходимости непрерывной работы в реальном времени. Микроконтроллер должен принимать аудиоданные без потерь, обслуживать обмен данными по сети и поддерживать локальное время. Поэтому при выборе методов обработки предпочтение отдаётся алгоритмам с малым объёмом памяти, целочисленными операциями и предсказуемым временем выполнения [5].

1.3 Анализ существующих систем акустического детектирования и локализации

Литература по распределённым системам акустического детектирования импульсных событий показывает, что наиболее устойчивой является архитектура с локальным детектированием на пространственно разнесённых устройствах, передачей компактных сообщений о событии и последующим объединением данных на центральном вычислителе [6; 7; 8]. Для разрабатываемой системы особенно важны работы, где узел является недорогим одноканальным устройством и не выполняет тяжёлую локальную классификацию.

Полноценные контрснайперские системы используют развитую аппаратную часть, несколько микрофонов на узел, специализированные платы и сложное объединение признаков [7; 8; 9]. Такие решения полезны как образец архитектуры, но избыточны для задачи локализации пиротехнических или иных коротких импульсных событий на базе STM32F411. Более близкими являются системы, в которых каждый узел оценивает момент начала события и передаёт только временную информацию [10; 11; 12]. В таблице 1 представлено сравнение существующих систем акустической локализации.

Таблица 1 — Сравнение существующих систем акустической локализации

Система	Платформа	ОЗУ	Флеш-па- мять	Аудио- каналы	Где выполня- ется локали- зация	Где выполня- ется детекти- рование	Потоки данных
Pinptr, Саймон и др. [7]	Mica2; Atmel atmega128; плата Vander- bilt; Xilinx Spartan-II	4 Кбайт	128 Кбайт	3	Базовая стан- ция	Акустиче- ская плата и Xilinx Spartan-II	Время прихода дульной волны и ударной волны от узлов к ба- зовой станции
Саллаи и др., много- каналь- ный ва- риант [8]	Micaz; Atmel atmega128l; Xilinx Spartan- II; носимый датчик	4 Кбайт	128 Кбайт	4	Персональное устройство или база	Сенсорная плата	Время прихода и углы прихода от узлов к базовой станции или персональному устройству
Саллаи, Ле- децци, Вёльд- еши [10]	Беспроводные одноканаль- ные датчики; модель не ука- зана	Нет дан- ных	Нет дан- ных	1	Центральный вычислитель	Датчик	Время первого прихода и дли- тельность ударной волны от датчиков к центральному вы- числителю
Кобос и др. [11]	Tmote или TelosB; Texas Instruments MSP430F1611	10 Кбайт	48 Кбайт	1	Центральный узел	Измеритель- ный узел	Время первого прихода и состо- яние локальных часов к цен- тральному вычислителю

Продолжение таблицы Таблица 1

Система	Платформа	ОЗУ	Флеш-па- мять	Аудио- каналы	Где выполня- ется локали- зация	Где выполня- ется детекти- рование	Потоки данных
Ким, Ан, Ча [12]	Telos mote; Texas Instru- ments MSP430F1611; RETOS	10 кбайт	48 кбайт	1	Лидер группы и ба- зовый узел	Каждый Telos-узел	Узлы передают идентификатор, положение и время обнаруже- ния соседям в пределах одного радиоперехода; лидер отправ- ляет результат на базовый узел
Хенги и др. [9]	Асинхронные акустические узлы, модель не указана	—	—	1	Центральный вычислитель	Акустиче- ский узел	Локальные временные разности между временем прихода дуль- ной волны и ударной волны
Ман- ринг и др. [13]	Raspberry Pi 4 Model B; Broadcom BCM2711	1/2/4/8 Гбайт	Карта microSD	1	Сервер	Raspberry Pi	Время начала события от Raspberry Pi к серверу
Святош, Холуб [14]	Измеритель- ные блоки; плата не ука- зана	—	—	—	Центральный блок	Сенсорный блок	Данные возможного выстрела от сенсорных к центральному блоку
Аста- пов, Эхала, Преден [15]	Vansonic PVM-6052, Agilent U2354A,	512 Мбайт	4 гбайт eMMC	6	Узел слияния	Датчик	Направление прихода, локаль- ное время, акустические от дат- чиков признаки на узел слияния

Из сравнения следует, что разрабатываемая система должна занять промежуточную нишу: использовать одноканальные сенсорные устройства, не передавать непрерывный аудиопоток, формировать временную метку на границе сети и переносить вычисление координат источника через выделенное устройство на сервер. Именно такая декомпозиция снижает требования к памяти, питанию и радиоканалу, но сохраняет пригодность данных для метода разности времён прихода.

1.4 Обоснование необходимости разработки собственной системы

Ни одна из рассмотренных работ не является точным воплощением целевой архитектуры: разреженная система на базе STM32F411, один цифровой микрофон на измерительный узел, локальное выделение первого прихода.

Наиболее близкие работы подтверждают отдельные проектные решения: локальное формирование временных меток [11; 12], центральное объединение данных [7; 8; 10], полезность компактных сообщений вместо сырого аудиопотока [6; 11], а также необходимость учитывать синхронизацию как один из главных факторов ошибки локализации [6; 16]. Однако готового решения, одновременно удовлетворяющего требованиям низкой стоимости и реализуемости на STM32F411CEU6 в рассмотренной литературе не обнаружено.

Следовательно, разработка собственной системы является обоснованной. Она не повторяет тяжёлые многоканальные решения, а комбинирует их архитектурные выводы с ограничениями микроконтроллерной реализации.

1.5 Методы локализации источника звука

Локализация по уровню сигнала основана на зависимости амплитуды от расстояния. Метод прост, но сильно зависит от направленности источника, отражений, поглощения и характеристик микрофонов.

Локализация по направлению прихода использует микрофонные решётки и методы GCC-PHAT, SRP-PHAT или beamforming. Эти методы эффективны, но требуют нескольких синхронно снятых каналов и дополнительных вычислений [6; 17].

Локализация по разности времён прихода основана на фиксации времени прихода одного события несколькими пространственно разнесёнными устройствами. Разности времён задают гиперболические линии допустимого положения источника.

Одним из главных следствий выбора TDOA в качестве метода локализации является то, что минимальная конфигурация должна включать четыре пространственно разнесенных микрофона, которые не должны располагаться в одной плоскости; в противном случае трехмерное положение, как правило, не может быть однозначно восстановлено [16]. Принцип локализации источника по разности времён прихода изображён на рисунке 1.

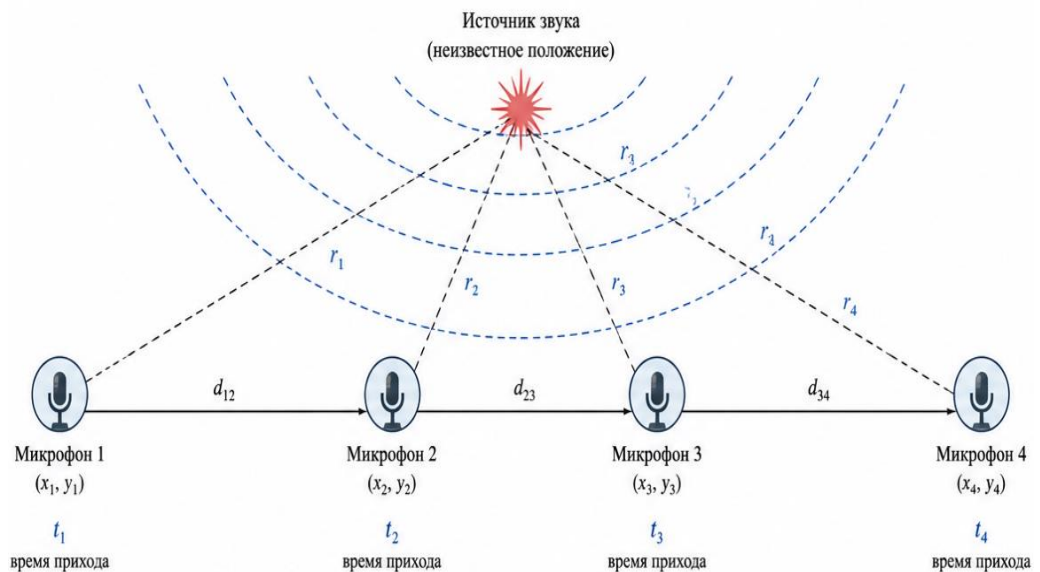


Рисунок 1 — Принцип локализации источника по разности времён прихода

1.6 Выбор целевой точности локализации

В качестве целевой точности системы выбрана относительная погрешности не более 10 % в диапазоне 40–100 м что является инженерным компромиссом

между низкой стоимостью узла и ожидаемым качеством локализации. Для расстояния 100 м это соответствует допустимой абсолютной ошибке порядка 10 м, а для расстояния 40 м — порядка 4 м. Такая точность достаточна для первичного определения зоны события, направления обхода или сужения области поиска источника.

Существующие системы демонстрируют широкий разброс точности, поскольку используют разные источники звука, геометрию, частоты дискретизации, число микрофонов и методы синхронизации. Многоканальные или специализированные системы способны достигать метрической и субметрической точности, однако это достигается ценой более дорогих датчиков, большей вычислительной нагрузки и усложнённой аппаратной части [14; 15]. Одноканальные сети низкой стоимости чаще опираются на передачу временных меток и предъявляют менее строгие требования к абсолютной точности [10; 11; 12].

1.7 Обзор методов детектирования событий в аудиопотоке

В системах, использующих TDOA, детектор выполняет две функции: определяет факт события и формирует временную метку прихода. Из-за этого не каждый детектор, хорошо работающий для обнаружения или классификации, подходит для локализации. Если метка привязана к максимуму амплитуды, завершению окна RMS или моменту обработки блока, то она зависит от отражений, расстояния, клиппинга, задержек обработки и локальных условий распространения.

Фиксированный амплитудный порог является простейшим методом. Он сравнивает текущее значение отсчёта с заданным порогом. Такой метод имеет минимальную вычислительную сложность, но плохо работает при изменении фонового шума и чувствителен к одиночным выбросам. Для распределённой TDOA-системы он не обеспечивает стабильного физического критерия первого прихода.

Энергетические и среднеквадратические методы используют окно усреднения и лучше подавляют одиночные выбросы. Однако временная метка, полученная по окончанию окна, систематически запаздывает относительно фронта события. Поэтому энергетический признак целесообразно использовать как вспомогательный показатель изменения сигнала, но не как окончательное правило выбора момента прихода. [18; 19].

Метод отношения кратковременного и долговременного средних сравнивает быстрый рост локальной энергии или огибающей с долговременным фоном. Он широко применяется в задачах обнаружения первых вступлений и хорошо подходит для потоковой обработки, поскольку требует только обновления сумм в двух скользящих окнах. [20].

Алгоритм кумулятивной суммы предназначен для обнаружения статистического изменения сигнала. В акустической сенсорной сети он повышает устойчивость к случайным кратковременным выбросам, так как событие подтверждается накоплением последовательных превышений, а не единичным отсчётом. [11; 21].

Уточнение первого прихода по предтриггерному буферу решает задачу, характерную именно для TDOA. Событие подтверждается после накопления достаточных признаков, но временная метка возвращается к раннему устойчивому участку фронта, который уже находится в кольцевом буфере. Такой подход разделяет подтверждение события и выбор физически значимого момента прихода [22; 23]. Сравнение методов детектирования события в аудиопотоке приведено в таблице 2.

Таблица 2 — Сравнение методов детектирования события в аудиопотоке

Метод	Роль в системе TDOA	Преимущество	Недостаток
Амплитудный порог	Первичное обнаружение	Минимальная сложность	Низкая устойчивость к шуму и дрейфу
Энергетический признак	Оценка изменения сигнала	Подавляет одиночные выбросы	Окно усреднения смещает момент срабатывания

Продолжения таблицы Таблица 2

Метод	Роль в системе TDOA	Преимущество	Недостаток
Адаптивный порог	Учёт нестационарного фона	Сохраняет чувствительность при изменении шума	Требует корректного обновления только вне события
Отношение кратковременного и долговременного средних	Подтверждение быстрого фронта	Подходит для потоковой обработки	Нужна настройка окон
Кумулятивная сумма	Подтверждение статистического изменения	Снижает ложные срабатывания	Требует настройки порога и утечки
Уточнение первого прихода	Формирование временной метки	Привязывает метку к раннему фронту	Требует предтриггерного буфера

1.8 Выбор метода детектирования событий в аудиопотоке

По итогам обзора выбран двухступенчатый потоковый детектор начала сигнала: чувствительный первичный триггер по короткой огибающей, подтверждение по отношению кратковременного и долговременного средних и алгоритму кумулятивной суммы, а затем уточнение первого прихода по предтриггерному буферу.

Для TDOA данный метод предпочтителен по двум причинам. Во-первых, подтверждение события выполняется устойчивыми критериями, что снижает вероятность ложного срабатывания на одиночный выброс. Во-вторых, итоговая метка определяется по первому устойчивому превышению огибающей над мягким порогом, найденному в предтриггерном буфере. Поэтому временная метка связана с фронтом акустического события, а не с задержкой обработки или максимумом амплитуды.

Для правильной работы алгоритмов необходимо выбрать корректные параметры. Значения некоторых параметров указаны в литературе, значения других будет выбрано в конструкторской части и протестировано в технологической части. Значения параметров, взятых из литературы их обоснование представлены в таблице 3.

Таблица 3 — Параметры значения которых взяты из литературы

Параметр	Значение	Обоснование
Порог отношения средних в формате Q8	768	Соответствует отношению 3,0, типичному для триггеров резких событий [20]
Множитель адаптивного порога	4	Снижает вероятность ложного срабатывания при шумовом фоне [19]
Утечка кумулятивной суммы	96	Не позволяет медленным изменениям фона накапливаться как событию [11; 21]

Логику детектора удобно оформить в виде конечного автомата с состояниями покоя, кандидата и блокировки. В состоянии покоя обновляются оценки фона. В состоянии кандидата накапливаются признаки события. После публикации включается блокировка, которая предотвращает повторное обнаружение хвоста импульса и отражений как нового события. Такой механизм повышает устойчивость алгоритма при работе с реальными импульсными источниками.

1.9 Анализ способов передачи данных от измерительных модулей к агрегирующему модулю

По причине того, что измерительный модуль должен передать на модуль не непрерывный аудиопоток, а компактное сообщение о зарегистрированном со-

бытии, способ передачи должен быть оптимизирован не под большую пропускную способность, а под короткие пакеты, малую задержку, низкую стоимость и простоту подключения к микроконтроллеру.

Возможные способы передачи данных между измерительными модулями и ведущим модулем можно разделить на проводные и беспроводные. Проводные интерфейсы обеспечивают высокую предсказуемость обмена, однако плохо соответствуют задаче пространственно разнесённого размещения датчиков. Беспроводные технологии удобнее для полевого развёртывания, но отличаются по дальности, энергопотреблению, сложности программного стека и задержке доставки. В таблице 4 представлено сравнение способов передачи данных от измерительных модулей к агрегирующему модулю.

Таблица 4 — Сравнение способов передачи данных от измерительных модулей к агрегирующему модулю

Способ передачи	Преимущества	Недостатки	Оценка применимости
UART или RS-485	Простая программная модель, низкая стоимость интерфейса, устойчивость к помехам при правильной прокладке кабеля	Требуется кабель к каждому измерительному модулю; усложняется размещение датчиков; снижается мобильность системы	Подходит для лабораторного стенда, но плохо соответствует автономной распределённой системе
CAN	Арбитраж шины, промышленная применимость, устойчивость к помехам	Требуется общий кабельный сегмент и внешние трансиверы; топология ограничивает свободное размещение датчиков	Избыточен для передачи коротких сообщений в малой системе
Wi-Fi	Пропускная способность десятки Мбит/с и готовая IP-инфраструктура	Сложный стек	Не является рациональным вариантом для передачи редких коротких сообщений от автономных узлов
Bluetooth Low Energy	Распространённость микросхем	Требуется стек BLE; задержка зависит от интервалов соединения; гарантированная срочная доставка события требует дополнительной настройки	Возможен, но повышает трудоёмкость программной части
ZigBee / IEEE 802.15.4	Ориентирован на сенсорные сети, поддерживает ячеистые и древовидные структуры	Сложнее стек; скорость 250 кбит/с; маршрутизация избыточна для малой сети с одним агрегирующим модулем [24; 25; 26].	Подходит для больших сенсорных сетей, но усложняет систему из нескольких датчиков
LoRa	Дальность до 20 км	Низкая скорость; большое время занятия эфира; задержка выше, чем требуется для локального обмена временными метками	Избыточен по дальности и недостаточно удобен для регулярного служебного обмена

Из сравнения следует, что для связи измерительных модулей с ведущим модулем наиболее целесообразна беспроводная топология «звезда». В такой топологии все измерительные модули передают сообщения одному ведущему модулю, а ведущий модуль выполняет сбор и дальнейшую передачу данных на сервер. В отличие от ячеистой сети, «звезда» не требует многопрыжковой маршрутизации, поэтому задержка доставки сообщения меньше зависит от состояния промежуточных узлов. Для TDOA-локализации это важно, потому что ведущий модуль должен получить временные метки от нескольких датчиков в предсказуемом порядке и без лишнего сетевого уровня.

При рассматриваемой постановке задачи не требуется передавать аудиопоток по радиоканалу и не требуется строить сеть из десятков или сотен узлов. Поэтому преимущества Wi-Fi по скорости, ZigBee по масштабируемости и LoRa по дальности не являются определяющими. Определяющими являются низкая стоимость одного измерительного канала, простое подключение к STM32F411CEU6, малое энергопотребление и возможность передачи коротких сообщений с подтверждением доставки. По этим критериям nRF24L01+ является наиболее подходящим вариантом [27].

Отдельно необходимо рассмотреть вопрос разделения служебного обмена и передачи событий. В системе присутствуют два разных типа радиосообщений. Первый тип — регулярные служебные сообщения, связанные с синхронизацией времени, управлением состоянием и контролем работоспособности. Второй тип — сообщения о зарегистрированном акустическом событии, которые должны передаваться после срабатывания детектора. Если использовать один общий радиоканал, то служебный обмен и передача события начинают конкурировать за один и тот же радиоресурс. Это не делает систему неработоспособной, но усложняет расписание обмена и повышает вероятность того, что срочное сообщение о событии будет ожидать завершения служебного обмена.

Поэтому на уровне архитектурного выбора целесообразно разделить служебный обмен и передачу событий. Такое разделение может быть выполнено

двумя независимыми радиоканалами одного типа. Преимущество такого подхода состоит в том, что обе линии обмена остаются простыми с точки зрения драйверов и аппаратного подключения, но получают разные логические роли: один канал ориентирован на регулярную поддержку состояния системы, второй — на передачу сообщений о событиях.

1.10 Анализ способов передачи данных от агрегирующего модуля на сервер локализации

После сбора временных меток ведущий модуль должен передать агрегированную информацию на сервер локализации. Этот канал отличается от внутрисистемного радиоканала. Между измерительными модулями и ведущим модулем важна свобода размещения датчиков, поэтому оправдана беспроводная связь. Между ведущим модулем и сервером важнее стабильность, стандартная интеграция с персональным компьютером или локальной сетью и отсутствие дополнительной радионагрузки.

Для связи ведущего модуля с сервером можно рассмотреть UART/USB, Wi-Fi, сотовую связь, LoRa и Ethernet. При сравнении важно учитывать, что STM32F411CEU6 не имеет встроенного Ethernet MAC/PHY, поэтому для проводной сети требуется внешний контроллер. Однако внешний Ethernet-контроллер может быть подключён по SPI, что позволяет сохранить выбранную микроконтроллерную платформу. В таблице 5 представлено сравнение способов передачи данных от агрегирующего модуля к серверу локализации.

Таблица 5 — Сравнение способов передачи данных от агрегирующего модуля к серверу локализации

Способ передачи	Преимущества	Недостатки	Оценка применимости
USB	Простота отладки и минимальная стоимость	Сервер должен находиться рядом; нет полноценной сетевой интеграции; неудобно для удалённого размещения	Подходит для стендовой проверки, но не как основной канал системы
Wi-Fi	Стандартная IP-сеть без кабеля, скорость до 150 мБит/с	Требуется беспроводной стек или внешний модуль; возможна конкуренция в диапазоне 2,4 ГГц; выше энергопотребление	Возможен, но повышает сложность и добавляет помехи в радиоканал
Сотовая связь	Возможность передачи на удалённый сервер без локальной сети	Дорогой модем, SIM-карта, зависимость от оператора, задержки сети	Избыточна для локальной системы акустической локализации
LoRa / LoRaWAN	Дальность до 20 км	Низкая скорость, необходимость шлюза, непригодность для стандартного локального подключения сервера без инфраструктуры	Не является оптимальным каналом
Ethernet через внешний контроллер	Стабильная проводная связь, стандартная локальная сеть, отсутствие радиопомех, простая интеграция с сервером	Требуется внешний Ethernet-контроллер и разъём	Наиболее рациональный вариант для агрегирующего модуля

Наиболее обоснованным вариантом является Ethernet через внешний контроллер W5500. Контроллер W5500 реализует аппаратный стек TCP/IP, содержит 10/100 Ethernet MAC и PHY, поддерживает TCP, UDP, IPv4, ICMP, ARP, IGMP и PPPoE, имеет 8 независимых сокетов и внутреннюю память 32 Кбайт для буферов передачи и приёма [28]. Для STM32F411CEU6 это важно, поскольку микроконтроллеру не требуется выполнять программную реализацию полного TCP/IP-стека: он может передавать прикладные данные в W5500 через SPI.

Для W5500 доступна библиотека `ioLibrary_Driver`, предназначенная для работы с TCP/IP-контроллерами WIZnet, включая W5500 [29]. Наличие готовой библиотеки уменьшает трудоёмкость разработки: необходимо перенести низкоуровневые функции SPI-обмена, управления выбором кристалла, сброса и задержек, а не писать сетевой стек с нуля. По сравнению с Wi-Fi или сотовым модемом такое решение имеет более предсказуемое поведение и не создаёт дополнительных радиопомех для внутрисистемного обмена.

1.11 Анализ методов синхронизации времени в распределённых сенсорных системах

Для метода разности времён прихода синхронизация времени является не вспомогательной, а одной из ключевых подсистем. Временная метка события, сформированная каждым измерительным модулем, должна быть выражена в согласованной временной шкале. Если шкалы времени измерительных модулей расходятся, то ошибка синхронизации непосредственно превращается в ошибку разности времён прихода и, следовательно, в ошибку координат источника.

Связь между ошибкой времени и ошибкой расстояния определяется скоростью звука. При скорости звука приблизительно 343 м/с ошибка времени 1 мс соответствует ошибке по пути распространения около 0,343 м. Для целевой

относительной погрешности локализации не более 10 % в диапазоне 40–100 м допустимая полная ошибка координаты составляет порядка 4–10 м. Однако эта ошибка включает не только синхронизацию, но и геометрию расположения датчиков, отражения, шум, неточность координат модулей и ошибку выделения первого прихода. Поэтому вклад синхронизации должен быть существенно меньше полной ошибки. Практически разумно стремиться к тому, чтобы ошибка синхронизации давала вклад не более единиц метров, то есть находилась в диапазоне единиц миллисекунд или ниже.

Для распределённых сенсорных сетей разработано несколько классов методов синхронизации: GPS-синхронизация, сетевые протоколы времени, RBS, TPSN, FTSP, методы лавинной передачи и специализированные методы для беспроводных сетей. Их применимость к данной системе различается, поскольку в рассматриваемом случае сеть малая, топология близка к «звезде», а обмен между датчиками выполняется короткими радиопакетами. В таблице 6 представлено сравнение методов синхронизации времени.

Таблица 6 — Сравнение методов синхронизации времени

Метод синхронизации	Преимущества	Недостатки для рассматриваемой системы	Оценка применимости
GPS-синхронизация	Единая внешняя шкала времени; не зависит от ведущего модуля	Требует GPS-модуль на каждом измерительном модуле; увеличивает стоимость и энергопотребление; хуже работает в помещениях и при экранировании	Избыточна для системы малой стоимости
NTP	Простота в компьютерных сетях	Измерительные модули не имеют IP-соединения; требуется сетевой стек и инфраструктура	Не подходит для ведомых измерительных модулей
IEEE 1588 PTP / IEEE 802.1AS	Наносекундная точность в проводных сетях с аппаратной поддержкой [30].	Требует Ethernet на каждом узле и аппаратной поддержки меток времени; противоречит выбору беспроводных измерительных модулей	Может быть полезна для других архитектур, но не для выбранной структуры датчиков

Продолжение таблицы Таблица 6

Метод синхронизации	Преимущества	Недостатки для рассматриваемой системы	Оценка применимости
RBS	Устраняет неопределённость задержки отправителя [31].	Требует дополнительного обмена между приёмниками; увеличивает трафик и усложняет протокол	Точен, но избыточен для малой звезды
TPSN	Даёт оценку смещения и задержки между узлами [32].	Нужны парные обмены; возрастает число служебных сообщений	Возможен, но сложнее необходимого
FTSP	Подходит для беспроводных сенсорных сетей и компенсирует дрейф [33].	Требует хранения истории измерений и более сложной обработки; рассчитан на сети большего масштаба	Хороший эталон, но трудоёмок для простой малой системы
Glossy и синхронное лавинное взаимодействие	Может обеспечивать субмикросекундную точность [34; 35].	Требует специальной организации доступа к среде и точного управления радиопередачей	Избыточен для миллисекундных требований акустической локализации
Специализированные методы для BLE и TSCH	Могут достигать микросекундной точности [36; 37; 38].	Требуют другой радиотехнологии и более сложного стека	Не соответствуют выбранному nRF24L01+
Периодическая синхронизация по внутреннему времени ведущего модуля	Простая реализация, малая коммуникационная нагрузка	Требует контроля задержек приёма	Наиболее рациональный компромисс для данной системы

Сравнение показывает, что высокоточные протоколы синхронизации не всегда являются лучшим выбором для низкостоимостной акустической системы. GPS, PTP, FTSP, Glossy и BLE-методы способны дать более высокую

точность, но требуют либо дополнительных аппаратных модулей, либо другой сетевой технологии, либо более сложного программного стека. Для рассматриваемой системы это повышает стоимость и трудоёмкость сильнее, чем улучшает итоговую точность локализации. При выбранной точности локализации важнее обеспечить устойчивую синхронизацию на уровне единиц миллисекунд, чем добиваться субмикросекундной точности ценой резкого усложнения всей архитектуры.

С учётом выбранной топологии «звезда» и радиоканала nRF24L01+ наиболее рациональным методом является периодическая синхронизация от ведущего модуля к измерительным модулям. В этом подходе ведущий модуль является источником опорной временной шкалы, а измерительные модули периодически получают служебные сообщения синхронизации и корректируют локальное время. Такой метод не требует GPS-приёмника на каждом узле, не требует сетевого IP-стека на измерительных модулях и не требует обмена между ведомыми модулями.

Для оценки применимости такого подхода необходимо сопоставить требуемую точность с ошибкой, возникающей из-за рассинхронизации. При скорости звука 343 м/с ошибка синхронизации 3 мс даёт вклад около 1 м по пути распространения. Это меньше полной допустимой ошибки для диапазона 40–100 м, равной примерно 4–10 м при требовании 10 %. Следовательно, для данной системы не требуется субмикросекундная синхронизация, характерная для некоторых промышленных сенсорных сетей. Достаточно обеспечить стабильность временной шкалы на уровне единиц миллисекунд и не допустить накопления значительного дрейфа между коррекциями.

Период коррекции порядка 100 мс является разумным начальным значением для такой системы. Если использовать кварцевый источник с отклонением 20 ppm, то за 100 мс накопленный дрейф составит 2 мкс что даёт ошибку в 4 мм, что приемлемо для выбранной точности локализации.

1.12 Разработка требований к узлу системы локализации

На основании проведённого обзора методов локализации, детектирования, обмена данными и синхронизации времени, а также выбранных технических решений, сформулированы требования к узлу системы локализации, которые должны быть учтены при разработке архитектуры в конструкторской части. Требования к узлу системы акустического мониторинга и локализации сведены в таблицу 7.

Таблица 7 — Требования к узлу системы акустического мониторинга и локализации

Параметр	Требование
Модель используемых микроконтроллеров	STM32F411CEU6 или функциональный аналог; тактовая частота не менее 100 МГц; Flash не менее 512 Кбайт; SRAM не менее 128 Кбайт
Технология акустического сенсора	Цифровой MEMS-микрофон с выходом I2S;
Технология связи с модулями узла	nRF24L01+
Метод синхронизации	Периодическая синхронизация по внутреннему времени ведущего модуля
Технология связи с сервером локализации	Контроллер W5500 с аппаратным стеком TCP/IP с подключение по SPI
Алгоритмы используемые при детектировании событий	Адаптивный порог, отношение кратковременного и долгосрочного средних, кумулятивная сумма, уточнение первого прихода по предтриггерному буферу
Наличие подсистемы мониторинга состояния модулей узла	Необходимо получать информацию о работоспособности модулей узла не реже одного раза в минуту
Точность локализации	Относительная погрешность не более 10% в диапазоне 40–100 м от центра базы датчиков

Продолжение таблицы Таблица 7

Параметр	Требование
Конфигурация размещения	Не менее четырех измерительных модулей, не расположенных в одной плоскости; минимальное расстояние между модулями не менее 5 м
Рабочая частота дискретизации аудио	16000 Гц
Источник питания аппаратных модулей	Литий-ионный аккумулятор или внешний источник 5 В со стабилизатором 3,3 В

2 Конструкторская часть

2.1 Проектирование архитектуры распределённой системы локализации источника звука

Разработанная архитектура системы построена по иерархическому принципу: четыре ведомых программно-аппаратных модуля и один ведущий. Ведомые программно-аппаратные модули являются автономными узлами регистрации, каждый из которых содержит цифровой микрофон, микроконтроллер, радиомодуль приёма управляющих команд и радиомодуль передачи временных меток. Ведущий программно-аппаратный модуль не выполняет первичную аудиообработку, а обеспечивает общую временную координацию, радиоприём событий и сетевой обмен. Такое разделение функций уменьшает вычислительную нагрузку на центральный узел и повышает масштабируемость системы.

Основной измеряемой величиной является момент первого прихода акустического импульса в точку расположения ведомого модуля. Для задачи разностно-дальномерной локализации важна не форма всего звукового сигнала, а согласованная во времени временная метка начала фронта. Поэтому ведомый модуль выполняет потоковую обработку аудио и формирует только две величины: синхронизированную временную метку и восьмибитный номер события. Передача минимального пакета длиной девять байт позволяет снизить вероятность коллизий и уменьшить время занятости радиоканала.

Система использует два независимых радионаправления. Первое радионаправление предназначено для управления ведомыми модулями: сброс внутреннего времени, периодическая синхронизация и индивидуальная проверка работоспособности. Второе радионаправление предназначено для передачи временных меток от ведомых модулей к ведущему. Разделение этих функций

по двум радиомодулям уменьшает взаимное влияние служебного обмена и передачи событий, а также упрощает диагностику, поскольку каждый радиомодуль имеет собственный набор счётчиков и регистров состояния.

Ведущий модуль после получения временных меток группирует их по номеру события. Если все четыре метки получены до истечения времени ожидания, группа немедленно передаётся на сервер. Если часть меток отсутствует, ведущий модуль завершает группу по тайм-ауту и заполняет отсутствующие значения нулями. Распределение функций между программно-аппаратными модулями представлено в таблице 8.

Таблица 8 — Распределение функций между программно-аппаратными модулями

Функция	Ведомый модуль	Ведущий модуль
Приём цифрового аудиосигнала	выполняет постоянно	не выполняет
Выделение первого прихода	выполняет потоковым алгоритмом	не выполняет
Формирование временной метки	формирует по локальной шкале времени	использует полученные метки
Сброс и синхронизация времени	принимает и применяет	формирует и передаёт
Проверка работоспособности	отвечает аппаратным подтверждением	последовательно опрашивает четыре узла
Передача временной метки события	передаёт радиопакет	принимает радиопакет
Группировка событий	не выполняет	объединяет временные метки по номеру события
Передача на сервер	не выполняет	формирует сетевой кадр и передаёт через сетевой контроллер

Структурная схема распределённой системы локализации источника звука изображена на рисунке 2.

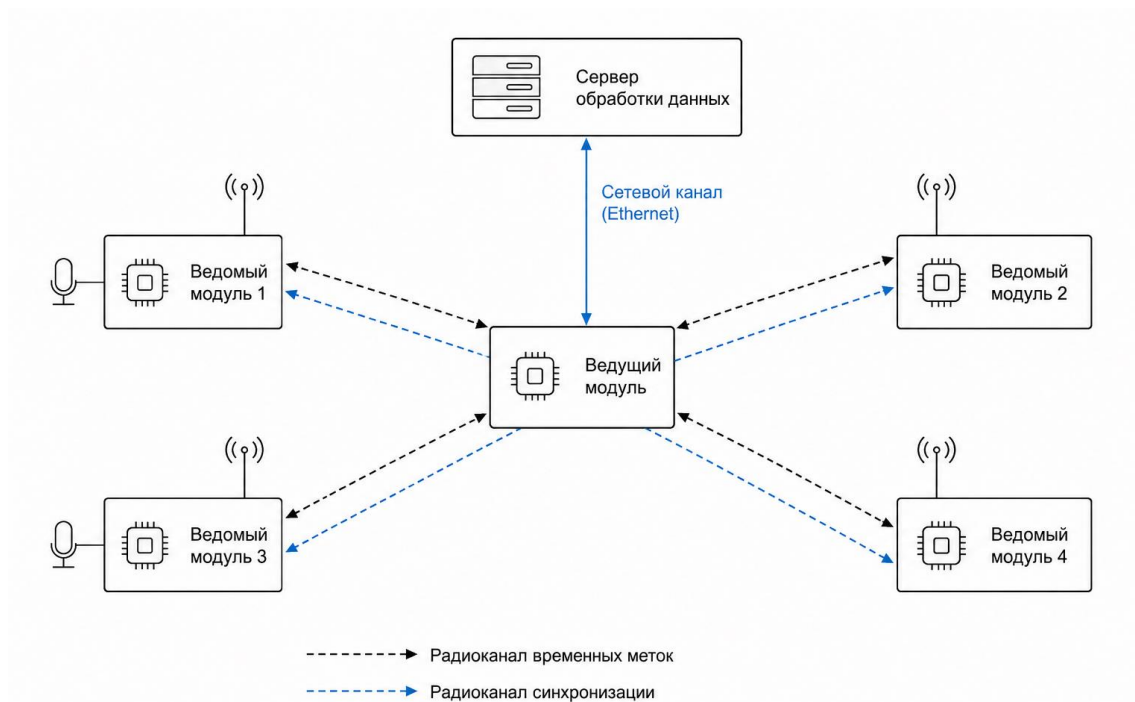


Рисунок 2 — Структурная схема распределённой системы локализации источника звука

2.2 Разработка структуры ведомого программно-аппаратного модуля

Ведомый программно-аппаратный модуль является автономным измерительным узлом. Его аппаратная часть включает микроконтроллер STM32F411CEU6, цифровой микрофон INMP441, первый радиомодуль для передачи временных меток и второй радиомодуль для приёма команд управления. Связь с микрофоном организована по последовательному аудиоинтерфейсу с использованием прямого доступа к памяти. Такое решение исключает необходимость опроса микрофона в основном цикле и обеспечивает регулярное поступление аудиоблоков.

Микрофон INMP441 передаёт двадцатичетырёхразрядные отсчёты в формате, требующем восстановления знакового значения из двух шестнадцатиразрядных слов буфера прямого доступа к памяти. Линия выбора канала

микрофона подключается к общему проводу, что соответствует выбору одного канала. На уровне программной реализации обработка выполняется по половинам кольцевого буфера, что позволяет одновременно принимать следующую часть аудиопотока и обрабатывать уже заполненную часть.

Для радиосвязи ведомый модуль использует два независимых экземпляра nRF24L01+. Радиомодуль управления находится в режиме приёмника и принимает общий канал сброса и синхронизации, а также индивидуальный канал проверки работоспособности. Радиомодуль событий находится в режиме передатчика и отправляет ведущему модулю пакет временной метки. Разделение направлений передачи исключает необходимость часто переключать один и тот же радиомодуль между режимами приёмника и передатчика, что повышает предсказуемость временных задержек.

Программная структура ведомого модуля построена как набор слабо связанных сервисных функций. Прерывания внешних линий радиомодулей и прерывания прямого доступа к памяти не выполняют длительных операций. Они устанавливают признаки готовности, а основная обработка выполняется в основном цикле или в функции обработки аудиоблока. Такой подход снижает вероятность логических гонок между аудиообработкой и радиосервисом. Аппаратные и программные компоненты ведомого программно-аппаратного модуля описаны в таблице 9.

Таблица 9 — Аппаратные и программные компоненты ведомого программно-аппаратного модуля

Компонент	Назначение
Микроконтроллер STM32F411CEU6	центральная обработка аудио и радиосервис
Цифровой микрофон INMP441	формирование цифрового аудиопотока
Буфер прямого доступа к памяти	приём аудиоданных без постоянной загрузки ядра

Продолжение таблицы Таблица 9

Компонент	Назначение
Радиомодуль управления	приём сброса, синхронизации и проверки работоспособности
Радиомодуль событий	передача временной метки события
Программный детектор	выделение первого прихода

Аппаратная структура ведомого программно-аппаратного модуля подробно изображена на рисунке. Аппаратная структура ведомого программно-аппаратного модуля подробно изображена на рисунке 3.

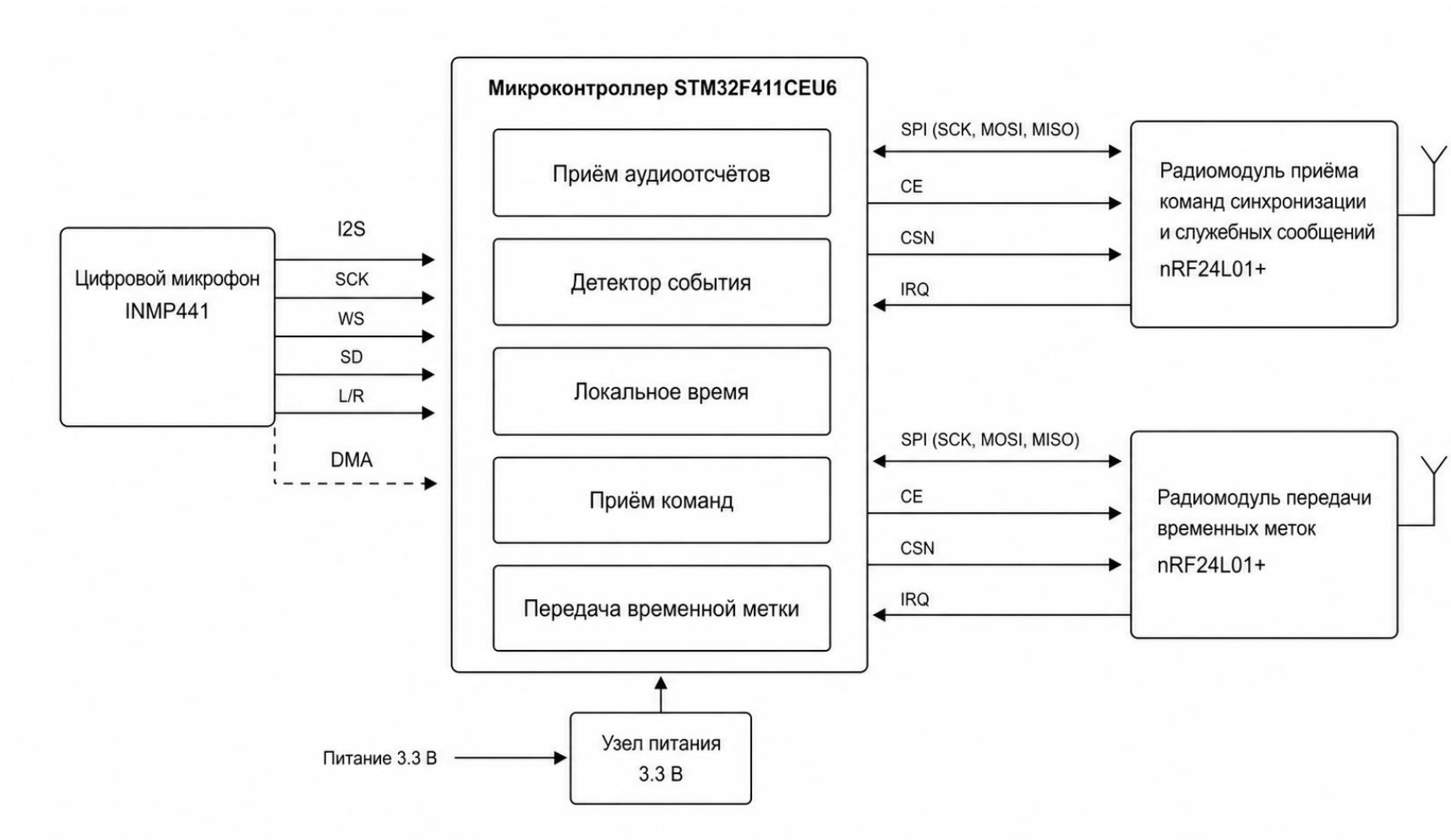


Рисунок 3 — Аппаратная структура ведомого программно-аппаратного модуля

2.3 Разработка структуры ведущего программно-аппаратного модуля

Ведущий программно-аппаратный модуль является центральным координатором системы. Он содержит микроконтроллер STM32F411CEU6, радиомодуль управления, радиомодуль приёма временных меток и сетевой контроллер W5500. В отличие от ведомого модуля, ведущий модуль не обрабатывает аудиосигнал, поэтому его вычислительные ресурсы сосредоточены на обеспечении регулярной радиосинхронизации, обслуживании радиоприёма событий и сетевой передаче агрегированных данных.

Радиомодуль управления ведущего узла работает в режиме передатчика. Через него ведущий модуль отправляет общий пакет сброса внутреннего времени, затем периодически отправляет пакеты синхронизации и один раз в заданный период выполняет последовательную проверку работоспособности четырёх ведомых узлов. Для проверки работоспособности используется индивидуальный адрес каждого ведомого узла. Перед каждой проверкой ведущий модуль явно устанавливает адрес передачи и адрес приёма подтверждения, что требуется для корректной работы режима аппаратного подтверждения.

Радиомодуль приёма временных меток ведущего узла работает в режиме приёмника. Для каждого ведомого узла используется отдельный приёмный канал. Номер канала используется для определения номера ведомого узла, что исключает необходимость передавать номер узла в полезной нагрузке пакета события. Полезная нагрузка события состоит только из временной метки и номера события.

Сетевой контроллер W5500 используется для передачи сформированного набора временных меток на сервер. Ведущий модуль формирует фиксированный кадр, включающий идентификаторы узлов и временные метки. При передаче диагностического результата проверки работоспособности времен-

ное поле рабочего узла заполняется единицами, а поле нерабочего узла заполняется нулями. Это позволяет использовать один и тот же формат кадра для событий и служебных состояний. Аппаратные и программные компоненты ведущего программно-аппаратного модуля описаны в таблице 10.

Таблица 10 — Аппаратные и программные компоненты ведущего программно-аппаратного модуля

Компонент	Назначение
Микроконтроллер STM32F411CEU6	координация обмена и группировка событий
Радиомодуль управления	передача сброса, синхронизации и проверки работоспособности
Радиомодуль приёма событий	приём временных меток от ведомых узлов
Сетевой контроллер W5500	передача кадра на сервер
Таймерная подсистема	формирование микросекундной шкалы времени
Программный буфер групп событий	объединение меток по номеру события

Аппаратная структура ведущего программно-аппаратного модуля подробно изображена на рисунке 4.

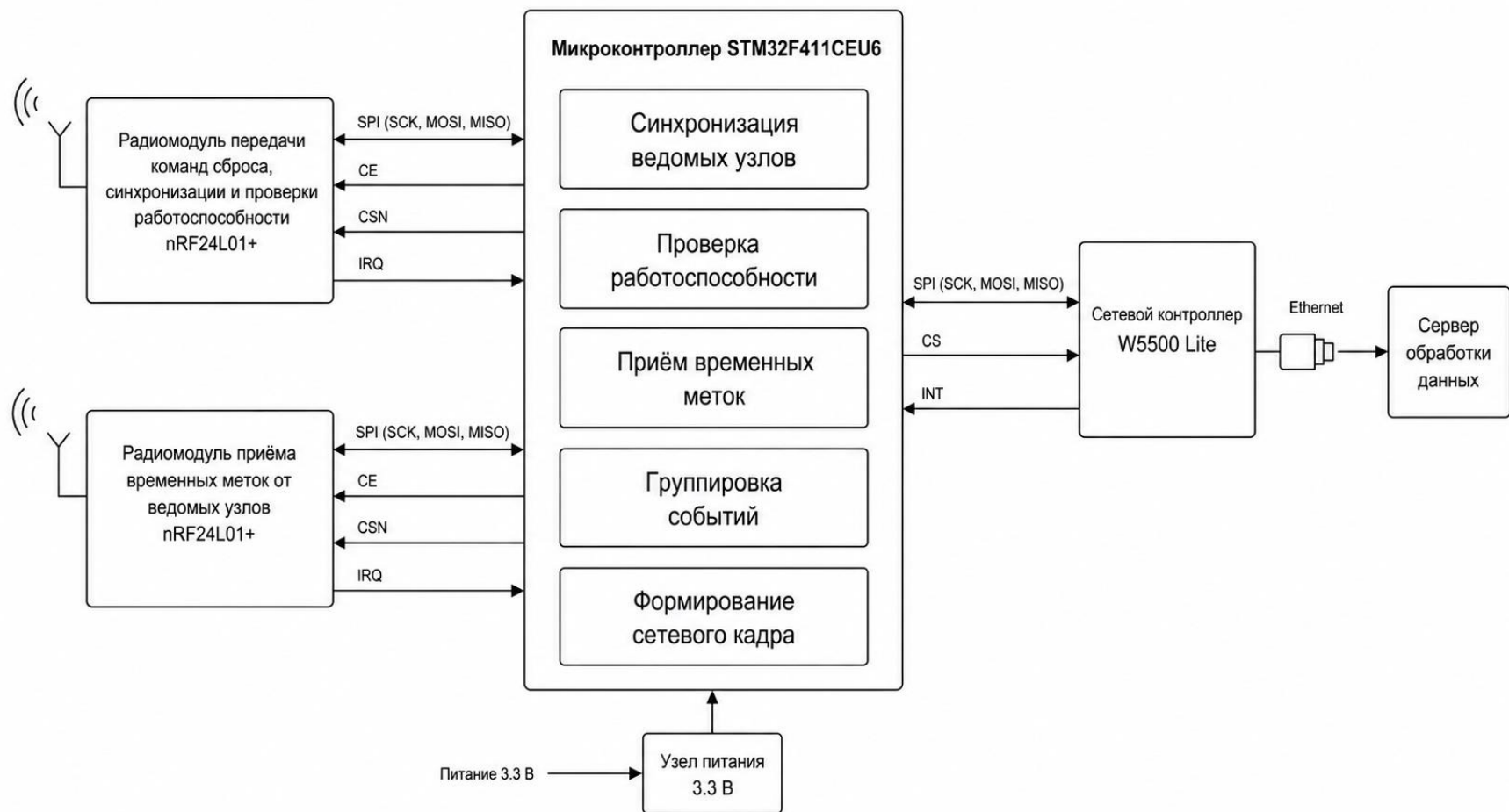


Рисунок 4 — Аппаратная структура ведущего программно-аппаратного модуля

2.4 Разработка программного модуля приёма и обработки звукового сигнала

Программный модуль приёма и обработки звукового сигнала реализован в ведомом узле. Частота дискретизации задаётся равной 16000 Гц, а размер обрабатываемого блока равен 512 отсчётам. Блочная обработка выбрана как компромисс между задержкой определения события и вычислительной эффективностью. При таком размере блока вычислительные операции выполняются пакетно, но временная метка уточняется по предтриггерному буферу, что позволяет не привязывать момент события к границе блока.

Первым этапом является восстановление знакового двадцатичетырёхразрядного отсчёта из данных последовательного аудиоинтерфейса. Затем выполняется оценка постоянной составляющей и вычитание этой оценки из исходного значения. Удаление постоянной составляющей необходимо, поскольку цифровой микрофон и тракт приёма могут иметь медленно изменяющееся смещение. Без удаления смещения адаптивный порог и оценка огибающей смещались бы вверх, что ухудшало бы чувствительность к слабым импульсам.

После удаления постоянной составляющей сигнал масштабируется и выпрямляется. Сглаженная огибающая вычисляется рекурсивно с параметром сдвига 2. Такой способ не использует операции с плавающей точкой и хорошо соответствует ограничениям микроконтроллера. Для оценки фоновой уровня применяется рекурсивное усреднение с параметром 6, а для оценки смещения - параметр 10. Все операции построены на целочисленных сдвигах, что снижает нагрузку на ядро и уменьшает время обработки каждого блока.

Предтриггерный буфер длиной 256 отсчётов хранит значения огибающей и соответствующие номера отсчётов до момента подтверждения события. Наличие такого буфера позволяет обнаружить фронт события раньше, чем бу-

дет выполнено полное подтверждение по отношению кратковременного и долговременного средних. Это важно для задачи разностно-дальномерной локализации, поскольку систематическое запаздывание временной метки приводит к увеличению ошибки определения координат источника. Значения параметров и их назначение сведены в таблицу 11.

Таблица 11 — Параметры программного модуля обработки звукового сигнала

Параметр	Значение	Назначение
Частота дискретизации	16000 Гц	задание временного масштаба аудиотракта
Размер блока обработки	512 отсчётов	единица пакетной обработки аудиоданных
Длина предтриггерного буфера	256 отсчётов	поиск раннего фронта события
Параметр оценки постоянной составляющей	10	сглаживание низкочастотного смещения
Параметр сглаживания огибающей	2	получение устойчивой огибающей
Параметр оценки шума	6	адаптация порога к фоновому уровню
Частота дискретизации	16000 Гц	задание временного масштаба аудиотракта

2.5 Разработка программного модуля обнаружения импульсного акустического события

Программный модуль обнаружения импульсного акустического события предназначен не для классификации источника звука, а для устойчивого выделения первого прихода. Поэтому в основе решения лежит комбинация

адаптивного порога, отношения кратковременного и долговременного средних, кумулятивного подтверждения и конечного автомата состояний. Такая комбинация уменьшает вероятность ложного срабатывания на фоновый шум и одновременно сохраняет чувствительность к резкому фронту импульсного сигнала.

Отношение кратковременного и долговременного средних рассчитывается в целочисленном формате Q8. Порог отношения в финальной версии равен 768, что соответствует значению 3,0 в обычной форме. Кратковременное окно содержит 8 отсчётов, долговременное окно содержит 128 отсчётов. Такое соотношение окон позволяет быстро реагировать на импульсный фронт и одновременно учитывать локальный уровень фоновой энергии.

Конечный автомат детектора имеет состояния покоя, кандидата и блокировки. В состоянии покоя открывается кандидат события при превышении адаптивного порога. В состоянии кандидата обновляются пик огибающей, максимальное отношение кратковременного и долговременного средних и накопитель подтверждения. Для публикации события требуется выполнение условий по длительности, форме фронта и уровню подтверждения. После публикации включается блокировка длительностью 800 отсчётов, предотвращающая повторное срабатывание на хвост того же импульса.

Сформированный номер события является восьмибитным счётчиком. При переполнении он сбрасывается в ноль, что допустимо для системы, в которой события группируются в пределах ограниченного интервала времени. Минимальная полезная нагрузка радиопакета события состоит из восьмибайтовой временной метки и однобайтового номера события. Параметры детектора импульсного акустического события, их значение и назначение сведены в таблицу 12.

Таблица 12 — Параметры детектора импульсного акустического события

Параметр	Значение	Назначение
Кратковременное окно	8 отсчётов	быстрая оценка роста огибающей

Продолжение таблицы Таблица 12

Параметр	Значение	Назначение
Долговременное окно	128 отсчётов	оценка фоновой уровня огибающей
Порог отношения в формате Q8	768	подтверждение им- пульсного характера события
Множитель основного порога	4	порог открытия кан- дидата
Множитель порога начала	2	поиск раннего начала события
Минимальная длительность начала	4 отсчёта	устойчивое превы- шение мягкого по- рога
Минимальная длительность формы	8 отсчётов	защита от кратковре- менных выбросов
Тайм-аут кандидата	320 отсчётов	сброс неподтвер- ждённого кандидата
Длительность блокировки	800 отсчётов	защита от повтор- ного обнаружения одного события
Порог накопителя подтвер- ждения	24000	кумулятивное под- тверждение события

Граф переходов конечного автомата программного детектора импульсного события изображена на рисунке 5.

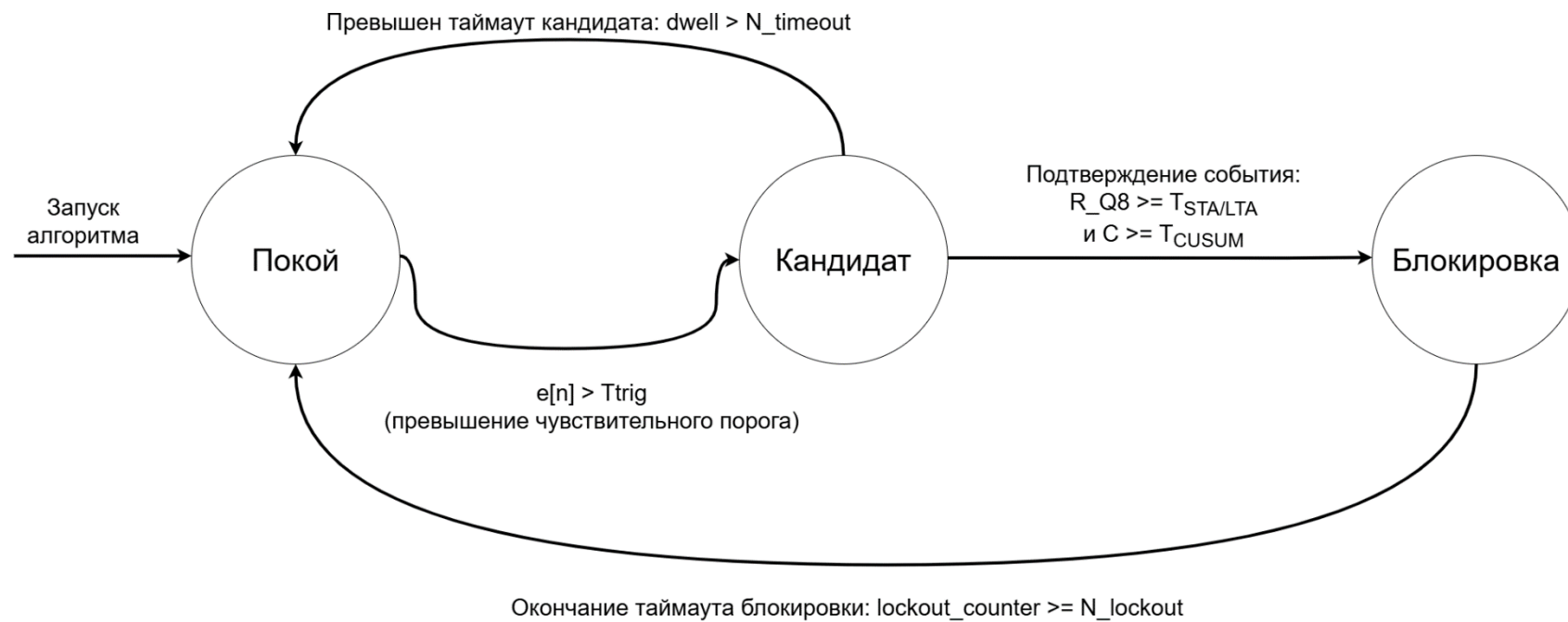


Рисунок 5 — Граф состояний конечного автомата программного детектора импульсного события

Таким образом полное математическое описание тракта обработки звука и детектирования события имеет следующий вид.

Сигнал с цифрового микрофона INMP441 поступает по интерфейсу I2S с частотой дискретизации F_s равной 16000Гц.

Период дискретизации T_s , мкс вычисляется по формуле:

$$T_s = \frac{1}{F_s} \quad (1)$$

Таким образом:

$$T_s = \frac{1}{16000} = 62,5 \text{ мкс} \quad (2)$$

Приём выполняется через DMA-буфер. В одной половине буфера обрабатывается блок из N_{block} аудиоотсчётов, N_{block} равен 512. Так как один 24-битный I2S-отсчёт представлен двумя 16-битными DMA-словами, количество DMA-слов в одной половине буфера N_{half} равняется 1024. Полный DMA-буфер содержит две половины. Таким образом число DMA-слов в буфере N_{DMA} равняется 2048.

Для каждого обработанного блока задаётся начальный абсолютный номер отсчёта n_0 . Абсолютный номер i -го отсчёта внутри блока вычисляется как

$$n = n_0 + i, \quad i = 0, 1, \dots, 511, \quad (3)$$

После обработки блока счётчик отсчётов N_{samples} увеличивается на размер блока:

$$N_{\text{samples}} \leftarrow N_{\text{samples}} + N_{\text{block}}, \quad (4)$$

Первым этапом обработки является восстановление 24-битного РСМ-отсчёта из двух 16-битных слов DMA. Для i -го отсчёта используются слова

$$w_0 = \text{buf}[2i], \quad (5)$$

$$w_1 = \text{buf}[2i + 1], \quad (6)$$

Незнаковое 24-битное представление отсчёта формируется из старшего 16-битного слова и старших восьми бит младшего слова:

$$x_{\text{raw}}[i] = 2^8 w_0 + \left\lfloor \frac{w_1}{2^8} \right\rfloor, \quad (7)$$

Так как микрофон передаёт знаковое 24-битное значение в дополнительном коде, далее выполняется расширение знака до 32 бит. В математической форме эта операция записывается как

$$x_{24}[i] = \begin{cases} x_{\text{raw}}[i] - 2^{24}, & x_{\text{raw}}[i] \geq 2^{23}, \\ x_{\text{raw}}[i], & x_{\text{raw}}[i] < 2^{23}, \end{cases} \quad (8)$$

Полученная диагностическая переменная test24 используется для контроля корректности восстановления исходного I2S-отсчёта:

$$\text{test24} = x_{24}[i], \quad (9)$$

Для 24-битного знакового РСМ допустимый диапазон значений равен

$$-2^{23} \leq x_{24}[i] \leq 2^{23} - 1, \quad (10)$$

Таким образом:

$$-8388608 \leq x_{24}[i] \leq 8388607,$$

Далее выполняется удаление постоянной составляющей. Оценка смещения обновляется рекурсивно по формуле:

$$d[n] = d[n-1] + \frac{x_{24}[n] - d[n-1]}{1024}, \quad (11)$$

После оценки смещения формируется сигнал без постоянной составляющей:

$$x_{hp}[n] = x_{24}[n] - d[n], \quad (12)$$

Для дальнейшей обработки 24-битный сигнал масштабируется к компактному 16-битному представлению:

$$x_{16}^*[n] = x_{hp}[n] \gg 8, \quad (13)$$

Эквивалентно делению на 2^8 :

$$x_{16}^*[n] = \frac{x_{hp}[n]}{2^8}, \quad (14)$$

Затем выполняется насыщение до диапазона знакового 16-битного значения:

$$x_{16}[n] = \begin{cases} 32767, & x_{16}^*[n] > 32767, \\ -32768, & x_{16}^*[n] < -32768, \\ x_{16}^*[n], & -32768 \leq x_{16}^*[n] \leq 32767 \end{cases}, \quad (15)$$

После масштабирования выполняется выпрямление сигнала:

$$r[n] = |x_{16}[n]|, \quad (16)$$

Далее вычисляем сглаженную огибающую являющуюся экспоненциальным скользящим средним:

$$e[n] = \frac{3}{4}e[n-1] + \frac{1}{4}r[n], \quad (17)$$

Путём эквивалентных преобразований получаем выражение с одним делением:

$$e[n] = e[n-1] + \frac{r[n]-e[n-1]}{4}, \quad (18)$$

Для уточнения момента первого прихода используется предтриггерный кольцевой буфер длиной N_{pre} равной 256. В этот буфер сохраняются значения $x_{16}[n]$ и $e[n]$. Так как $N_{\text{pre}} = 256 = 2^8$, индекс записи p обновляется по модулю:

$$p \leftarrow (p + 1) \bmod 256, \quad (19)$$

Для подтверждения импульсного события используется отношение кратковременного и долговременного средних. Сумма огибающей на коротком окне $S_{\text{STA}}[n]$ вычисляется по формуле:

$$S_{\text{STA}}[n] = \sum_{k=0}^{N_{\text{STA}}-1} e[n-k], \quad (20)$$

где N_{STA} — число отсчётов в коротком окне равно 8;

Сумма огибающей на длинном окне вычисляется по формуле:

$$S_{LTA}[n] = \sum_{k=0}^{N_{LTA}-1} e[n-k], \quad (21)$$

где N_{LTA} — число отсчётов в длинном окне равное 128;

Вещественное отношение кратковременного и долговременного средних $R[n]$ имеет вид:

$$R[n] = \frac{S_{STA}[n]/N_{STA}}{S_{LTA}[n]/N_{LTA}}, \quad (22)$$

Для исключения операций с плавающей точкой отношение хранится в формате Q8, вещественное отношение в формате Q8:

$$R_{Q8}[n] = \frac{S_{STA}[n] \cdot N_{LTA} \cdot 256}{S_{LTA}[n] \cdot N_{STA}}, \quad (23)$$

Связь между обычным отношением и Q8-представлением задаётся выражением:

$$R_{Q8}[n] = 256 \cdot R[n]; \quad (24)$$

Порог отношения $R_{thr,Q8}$ в формате Q8 программе задан равным 768. Порог в обычном виде R_{thr} равен 3.

Следовательно, условие подтверждения по отношению долговременного и кратковременного средних записывается как

$$R[n] > 3, \quad (25)$$

или в целочисленной форме

$$R_{Q8}[n] > 768 \quad (26)$$

Фоновый уровень в состоянии покоя оценивается адаптивно. Средний уровень шума $\mu[n]$ рассчитывается по формуле:

$$\mu[n] = \mu[n - 1] + \frac{e[n] - \mu[n-1]}{64}, \quad (27)$$

Среднее отклонение огибающей от текущей оценки фона $a[n]$ определяется по формуле:

$$a[n] = |e[n] - \mu[n]|, \quad (28)$$

Затем оценка отклонения $\sigma_d[n]$ вычисляется по формуле:

$$\sigma_d[n] = \sigma_d[n - 1] + \frac{a[n] - \sigma_d[n-1]}{64}, \quad (29)$$

Для предотвращения слишком малого порога вводится ограничение

$$\sigma_d[n] \geq 8, \quad (30)$$

Адаптивный порог обнаружения $T_m[n]$ строится по среднему уровню шума и оценке отклонения и вычисляется по формуле:

$$T_m[n] = \mu[n] + m\sigma_d[n], \quad (31)$$

Для основного срабатывания $T_{\text{trig}}[n]$ используется множитель m равный 4, поэтому основной порог вычисляется по формуле:

$$T_{\text{trig}}[n] = \mu[n] + 4\sigma_d[n], \quad (32)$$

С учётом нижней границы порога окончательная формула вычисления $T_m[n]$ имеет вид:

$$T_m[n] = \max(32, \mu[n] + m\sigma_d[n]), \quad (33)$$

Для поиска начала события используется более мягкий порог $T_{\text{onset}}[n]$, вычисляемый по формуле:

$$T_{\text{onset}}[n] = \mu[n] + 2\sigma_d[n], \quad (34)$$

Дополнительное подтверждение события выполняется с помощью алгоритма кумулятивной суммы. Базовый уровень для накопления $T_{\text{base}}[n]$ вычисляется по формуле:

$$T_{\text{base}}[n] = \mu[n] + \sigma_d[n], \quad (35)$$

Если огибающая превышает базовый уровень, приращение кумулятивной суммы $\Delta[n]$ вычисляется по формуле:

$$\Delta[n] = e[n] - T_{\text{base}}[n], \quad (36)$$

Приращение ограничивается сверху:

$$\Delta[n] \leq 4096, \quad (37)$$

При превышении базового уровня накопитель обновляется по формуле:

$$C[n] = C[n - 1] + \Delta[n], \quad (38)$$

Если превышения нет, применяется утечка накопителя:

$$C[n] = \begin{cases} C[n-1] - L, & C[n-1] > L, \\ 0, & C[n-1] \leq L, \end{cases} \quad (39)$$

Где L — коэффициент утечки, который равен 96.

Событие считается подтверждённым по алгоритму кумулятивной суммы при выполнении условия:

$$C[n] \geq C_{\text{thr}}, \quad (40)$$

где C_{thr} — порог кумулятивной суммы, который равен 2400.

Работа детектора описывается конечным автоматом с состояниями QUIET, CANDIDATE и LOCKOUT. В состоянии QUIET кандидат события открывается при выполнении условия:

$$e[n] > T_{\text{trig}}[n], \quad (41)$$

При открытии кандидата фиксируются начальный номер отсчёта n_{cand} , пиковая огибающая e_{peak} и максимальное отношение кратковременного и долговременного средних R_{peak} :

$$n_{\text{cand}} = n, \quad (42)$$

$$e_{\text{peak}} = e[n], \quad (43)$$

$$R_{\text{peak}} = R_{Q8}[n], \quad (44)$$

В состоянии CANDIDATE e_{peak} и R_{peak} обновляются по формулам

$$e_{\text{peak}} \leftarrow \max(e_{\text{peak}}, e[n]), \quad (45)$$

$$R_{\text{peak}} \leftarrow \max(R_{\text{peak}}, R_{Q8}[n]), \quad (46)$$

Длительность превышения порога может быть записана как сумма индикаторной функции:

$$D = \sum I(e[n] > T_{\text{trig}}[n]), \quad (47)$$

где $I(\cdot)$ равно единице при истинности условия и нулю в противном случае. Кандидат публикуется как событие только если $R_{Q8}[n]$ больше 768 и $C[n]$ больше 24000.

$$R_{Q8}[n] > 768, \quad (48)$$

и

$$C[n] \geq 24000, \quad (49)$$

Если кандидат не подтверждается в течение времени t_{timeout} то он сбрасывается, t_{timeout} вычисляется по формуле:

$$t_{\text{timeout}} = \frac{N_{\text{timeout}}}{F_s}, \quad (50)$$

где N_{timeout} — количество отсчётов равное 320.

Таким образом:

$$t_{\text{timeout}} = \frac{320}{16000} = 0,02 \text{ с} = 20 \text{ мс}$$

После публикации события включается блокировка повторного срабатывания длительностью t_{lockout} , t_{lockout} вычисляется по формуле:

$$t_{\text{lockout}} = \frac{N_{\text{lockout}}}{F_s}, \quad (51)$$

где N_{lockout} — количество расчётов равное 800.

Таким образом:

$$t_{\text{lockout}} = \frac{800}{16000} = 0,05 \text{ с} = 50 \text{ мс},$$

Уточнение момента первого прихода выполняется по предтриггерному буферу. Ищется первый устойчивый участок, для которого огибающая не менее четырёх отсчётов подряд превышает порог T_{onset} :

$$e[n_0 + k] \geq T_{\text{onset}}, \quad k = 0, 1, 2, 3, \quad (52)$$

Дополнительно учитывается условие неубывания огибающей в начале участка:

$$e[n_0] \geq e[n_0 - 1], \quad (53)$$

Если такой участок найден, момент события n_{event} принимается равным n_0 . Если участок не найден, n_{event} приравнивается к номеру текущего отсчёта n . Итоговая временная метка n_{event} ограничивается так, чтобы она не была позже начала кандидата n_{cand} .

Перевод номера отсчёта n во временную метку $t_{\mu\text{s}}$ в микросекундах выполняется по формуле:

$$t_{\mu\text{s}}(n) = t_{\text{start}} + \frac{n \cdot 10^6}{F_s}, \quad (54)$$

С учётом частоты дискретизации F_s равной 16 кГц формула принимает вид:

$$t_{\mu s}(n) = t_{\text{start}} + 62,5n, \quad (55)$$

Так как расчёт в программе выполняется целочисленно, фактически используется операция взятия целой части:

$$t_{\mu s}(n) = t_{\text{start}} + \left\lfloor \frac{n \cdot 10^6}{16000} \right\rfloor, \quad (56)$$

После подтверждения события увеличивается идентификатор события ID увеличиваются на единицу.

Полезная нагрузка радиопакета включает временную метку и идентификатор события. Так как временная метка занимает 8 байт, а идентификатор занимает 1 байт, размер минимального пакета равен 9 байт.

Данный тракт сохраняет ранний фронт импульсного акустического события, не использует операции с плавающей точкой и обеспечивает получение временной метки, пригодной для последующей передачи на ведущий модуль и применения в алгоритмах TDOA.

Исходный код разработанного программного модуля `main.c` приведен в приложении А.

2.6 Разработка программного модуля локального времени ведомого узла

Локальное время ведомого узла используется для преобразования абсолютного номера аудиоотсчёта в временную метку, сопоставимую с временными метками других ведомых узлов. Поскольку каждый ведомый модуль имеет собственный генератор тактовой частоты, без периодической синхронизации его временная шкала постепенно расходилась бы со шкалами остальных

узлов. Для уменьшения этой погрешности ведущий модуль передаёт периодические пакеты синхронизации.

Пакет сброса времени используется при запуске системы. После его приёма ведомый модуль сбрасывает локальную шкалу времени, счётчики событий и диагностические признаки. Затем ведущий модуль начинает периодически передавать синхронизационные пакеты, содержащие восьмибайтовое значение времени ведущего модуля. Ведомый узел принимает значение времени, вычисляет расхождение с собственной шкалой и применяет коррекцию.

В разработанном протоколе управляющий пакет имеет постоянную длину девять байт. Первый байт содержит код команды, а последующие восемь байт используются как поле времени. Для пакета сброса и пакета проверки работоспособности поле времени заполняется нулями. Выравнивание длины управляющих пакетов исключает неопределённость при статической длине полезной нагрузки радиомодуля. Информация о пакетах радиуправления ведомым программно-аппаратным модулем сведена в таблицу 13.

Таблица 13 — Пакеты радиуправления ведомым программно-аппаратным модулем

Пакет	Код	Длина	Назначение
Сброс внутреннего времени	0xA0	9 байт	обнуление локальной шкалы времени и служебных счётчиков
Синхронизация времени	0xA1	9 байт	передача времени ведущего модуля в ведомый модуль
Проверка работоспособности	0xA2	9 байт	получение аппаратного подтверждения от конкретного ведомого узла

2.7 Разработка программного модуля радиопередачи временных меток ведомым программно-аппаратным модулем

Радиопередача временной метки выполняется ведомым модулем только после подтверждения события. Такой подход исключает передачу промежуточных диагностических значений и снижает загрузку радиоканала. Полезная нагрузка пакета события имеет фиксированную длину девять байт: восемь байт занимают синхронизированная временная метка, один байт занимает номер события.

Передача выполняется отдельным радиомодулем, находящимся в режиме передатчика. Адрес передачи ведомого модуля выбирается в соответствии с номером узла. Для ведущего модуля это соответствует отдельному приёмному каналу. Таким образом, ведущий модуль может определить источник временной метки по номеру канала приёма, не увеличивая размер полезной нагрузки.

После загрузки полезной нагрузки в радиомодуль формируется короткий импульс разрешения передачи. Завершение передачи или достижение максимального числа повторов фиксируется по линии прерывания радиомодуля. Диагностические счётчики успешных передач и отказов позволяют отличить отсутствие события от ошибки радиоканала. Структура радиопакета временной метки ведомого модуля представлена в таблице 14.

Таблица 14 — Структура радиопакета временной метки ведомого модуля

Поле пакета	Размер	Описание
Преамбула	1 байт	Формируется радиомодулем автоматически
Адрес	3 байта	Адрес канала приёма приёмника
Контроль	9 бит	Длина полезной нагрузки, PID, флаг NO_ACK со значением 0

Продолжение таблицы Таблица 14

Поле пакета	Размер	Описание
Полезная нагрузка	9 байт	Временная метка и номер события
Контрольная сумма	2 байта	Аппаратная CRC-16 радио-модуля

2.8 Разработка программного модуля радиуправления ведомым узлом

Радиомодуль управления ведомого узла постоянно находится в режиме приёмника. Он слушает общий адрес сброса и синхронизации, а также индивидуальный адрес проверки работоспособности. Общий адрес одинаков для всех ведомых узлов, поскольку пакеты сброса и синхронизации должны быть доставлены всем узлам. Индивидуальный адрес различается, поскольку ведущий модуль должен проверить работоспособность каждого узла отдельно.

Для корректного приёма пакетов без подтверждения на общем канале включён расширенный режим радиомодуля. Пакеты сброса и синхронизации передаются ведущим модулем с признаком отсутствия подтверждения, поэтому ведомые узлы не создают одновременные ответные пакеты. При проверке работоспособности используется индивидуальный адрес и аппаратное подтверждение, что позволяет ведущему модулю определить наличие конкретного ведомого узла.

Обработчик приёма радиопакета считывает все девять байт полезной нагрузки, сохраняет последний код команды, номер канала и длину пакета в диагностические переменные. Далее по коду команды выполняется сброс времени, синхронизация или только увеличение счётчика проверки работоспособности. Аппаратное подтверждение формируется самим радиомодулем, по-

этому программная отправка ответного пакета для проверки работоспособности не требуется. В таблице 15 представлены адреса ведомых модулей для радиоуправления и передачи временных меток.

Таблица 15 — Адреса ведомых модулей для радиоуправления и передачи временных меток

Ведомый узел	Адрес передачи временной метки	Индивидуальный адрес проверки работоспособности
Первый	00 CC BB AA E1	10 C3 B3 A3 E3
Второй	01 CC BB AA E1	11 C3 B3 A3 E3
Третий	02 CC BB AA E1	12 C3 B3 A3 E3
Четвёртый	03 CC BB AA E1	13 C3 B3 A3 E3

2.9 Разработка программного модуля синхронизации и проверки работоспособности ведомых модулей

Программный модуль синхронизации и проверки работоспособности входит в состав ведущего модуля. После инициализации он отправляет пакет сброса внутреннего времени, затем через одну секунду начинает периодическую синхронизацию с периодом 100 мс. В пакете синхронизации передаётся микросекундное время ведущего модуля, представленное в виде восьми байт в порядке от младшего к старшему.

Проверка работоспособности выполняется один раз в минуту. Ведущий узел последовательно устанавливает адрес первого, второго, третьего и четвёртого ведомого узла, загружает пакет проверки работоспособности и ожидает аппаратное подтверждение. Если подтверждение получено, соответствующий элемент массива состояния устанавливается в единицу. Если получено сообщение о превышении числа повторов или истёк тайм-аут, узел считается недоступным.

Ключевым проектным решением является явное переключение адреса передачи и адреса приёма подтверждения. В режиме передатчика радиомодуля nRF24L01+ аппаратное подтверждение принимается через нулевой канал приёма, поэтому адрес нулевого канала должен совпадать с текущим адресом передачи [24]. После завершения проверки всех четырёх ведомых узлов ведущий модуль возвращает общий адрес синхронизации, чтобы последующие пакеты сброса и синхронизации отправлялись корректно. Основные параметры программного модуля синхронизации и проверки работоспособности ведомых модулей сведены в таблицу 16.

Таблица 16 — Основные параметры программного модуля синхронизации и проверки работоспособности

Параметр	Значение	Назначение
Задержка начала синхронизации	1000 мс	пауза после начального сброса времени
Период синхронизации	100 мс	регулярная коррекция времени ведомых узлов
Период проверки работоспособности	60000 мс	служебный опрос четырёх ведомых узлов
Количество проверок в цикле	4	по одной проверке на каждый ведомый модуль
Общий адрес синхронизации	D2 C2 B2 A2 E2	адрес пакетов сброса и синхронизации

2.10 Разработка программного модуля приёма временных меток ведущим модулем

Приём временных меток в ведущем модуле выполняется радиомодулем, работающим в режиме приёмника. Для четырёх ведомых узлов настроены четыре приёмных канала. Общая старшая часть адреса одинакова, а младший байт соответствует номеру ведомого узла. Такой способ адресации использует

аппаратную возможность радиомодуля принимать несколько адресов и позволяет ведущему модулю различать источники пакетов без дополнительного поля номера узла в полезной нагрузке.

При появлении признака приёма пакет считывается из очереди радиомодуля. Из статуса радиомодуля извлекается номер канала, затем из полезной нагрузки извлекаются восьмибайтовая временная метка и однобайтовый номер события. Программный модуль проверяет допустимость номера канала и передаёт данные в модуль группировки. Если номер канала недопустим или очередь приёма имеет ошибочное состояние, выполняется очистка очереди и увеличивается диагностический счётчик.

Для повышения устойчивости ведущий модуль не полагается только на внешнее прерывание. Если линия прерывания была активна до включения обработчика или фронт был пропущен, сервисная функция дополнительно опрашивает регистр состояния радиомодуля. Это особенно важно при отладке и при работе с несколькими радиопакетами, поступающими с малым интервалом. В таблице 17 сведена информация о канал приёма временных сеток ведущим модулем.

Таблица 17 — Каналы приёма временных меток ведущим модулем

Номер ведомого узла	Канал приёма ведущего модуля	Первый байт адреса	Назначение
1	0	0x00	временная метка первого ведомого узла
2	1	0x01	временная метка второго ведомого узла
3	2	0x02	временная метка третьего ведомого узла
4	3	0x03	временная метка четвёртого ведомого узла

2.11 Разработка программного модуля группировки событий ведущим модулем

Модуль группировки событий предназначен для объединения временных меток, относящихся к одному акустическому воздействию. В качестве ключа группировки используется восьмибитный номер события. При поступлении первой метки создаётся новая группа, в которую записываются номер события, временная метка, номер ведомого узла и крайний момент ожидания остальных меток. При поступлении следующих меток с тем же номером события группа дополняется.

Если группа содержит временные метки от всех четырёх ведомых узлов, она немедленно передаётся в модуль сетевого обмена. Если часть меток отсутствует, но истёк тайм-аут ожидания, группа также передаётся, а отсутствующие значения заполняются нулями. Такой подход снижает риск потери события из-за временной недоступности одного ведомого узла и позволяет серверной части отличить полный измерительный набор от неполного.

Значение тайм-аута группировки составляет 1000 мс. Такое значение выбрано с запасом относительно времени радиопередачи одного пакета и с учётом возможной очередности обслуживания радиомодулей в основном цикле. При этом тайм-аут достаточно мал по сравнению с интервалом между независимыми импульсными событиями в эксперименте.

Таблица 18 — Состояния группы временных меток в ведущем модуле

Состояние группы	Условие перехода	Действие
Свободная ячейка	нет активного события	ожидание первой временной метки
Активная неполная группа	получена одна или несколько меток	ожидание остальных ведомых узлов
Активная полная группа	получены четыре временные метки	немедленная передача на сервер

Продолжение таблицы Таблица 18

Состояние группы	Условие перехода	Действие
Истёкшая группа	истёк тайм-аут ожидания	передача неполного набора с нулевыми полями
Освобождение ячейки	кадр подготовлен	сброс состояния и готовность к новому событию

2.12 Разработка программного модуля сетевой передачи данных ведущим модулем

Сетевой модуль ведущего узла обеспечивает передачу сформированной информации на внешний сервер. Для физического и канального уровня используется контроллер W5500, подключённый к микроконтроллеру по последовательному интерфейсу. В программной части используется портирующий слой WIZnet, который выполняет низкоуровневый обмен с контроллером и предоставляет функции работы с сокетами.

Полезная нагрузка сетевого кадра содержит данные о ведомых узлах и их временных метках. Для события используются реальные микросекундные значения временных меток. Для диагностического кадра проверки работоспособности используется тот же формат, но временные поля заполняются специальными значениями: единицами для доступного узла и нулями для недоступного. Это упрощает серверную обработку, поскольку структура кадра не меняется между измерительными и служебными сообщениями.

Сетевой обмен выполняется в основном цикле ведущего модуля. При проектировании учитывалось, что длительные блокировки сетевого обмена могут нарушить периодичность радиосинхронизации. Поэтому радиомодуль синхронизации обслуживается регулярно, а операции сетевой передачи

должны иметь ограниченное время выполнения и диагностироваться по состоянию сокета и счётчикам отправки. Структура сетевого кадра, передаваемого ведущим модулем на сервер изображена на рисунке 6.

319	256	255	248	247	184	183	176	175	112	111	104	103	40	39	32	31	0
Timestamp3	Node_id3		Timestamp2		Node_id2		Timestamp1		Node_id1		Timestamp0		Node_id0		Length		

Рисунок 6 — Структура сетевого кадра ведущего модуля

2.13 Объединение программных модулей ведомого модуля

Объединение программных модулей ведомого модуля выполнено по принципу разделения быстродействующих обработчиков и сервисной логики. Быстродействующие обработчики прерываний только фиксируют факт поступления данных или события периферии. Обработка аудиоблока, обслуживание радиомодулей и формирование радиопакета выполняются в функциях, вызываемых из основного цикла или из контролируемого контекста.

Аудиомодуль передаёт детектору блок восстановленных отсчётов и абсолютный номер первого отсчёта блока. Детектор после подтверждения события формирует временную метку и устанавливает признак готовности радиопакета. Радиомодуль событий проверяет этот признак и при отсутствии текущей передачи загружает полезную нагрузку в радиомодуль. Радиомодуль управления независимо принимает пакеты сброса, синхронизации и проверки работоспособности.

Особое внимание уделено предотвращению логических гонок. Временная метка события должна формироваться на основе согласованной шкалы времени, а радиопакет не должен перезаписываться до завершения текущей передачи. Для этого используются флаги занятости, счётчики завершения передачи и короткие критические участки. При этом длительный запрет прерываний не применяется, чтобы не нарушить приём аудиоданных. Информация

о взаимодействие программных модулей ведомого модуля сведена в таблицу 19.

Таблица 19 — Взаимодействие программных модулей ведомого модуля

Источник данных	Получатель данных	Передаваемая информация	Механизм синхронизации
Буфер прямого доступа к памяти	модуль аудиообработки	сырые слова аудиоинтерфейса	прерывание половины или полного буфера
Модуль аудиообработки	детектор события	огибающая и номер отсчёта	последовательный вызов обработки блока
Детектор события	модуль радиопередачи	временная метка и номер события	флаг готовности пакета
Радиомодуль управления	модуль локального времени	команды сброса и синхронизации	флаг внешнего прерывания и очередь приёма
Модуль радиопередачи	диагностический интерфейс	счётчики успешной передачи и ошибок	глобальные переменные мониторинга

2.14 Объединение программных модулей ведущего модуля

Ведущий узел объединяет несколько сервисных задач: периодическую передачу синхронизации, проверку работоспособности ведомых узлов, приём временных меток, группировку событий и сетевую передачу. Все эти задачи обслуживаются в основном цикле, что позволяет избежать сложной многозадачной среды и сохранить полный контроль над последовательностью обращения к радиомодулям.

Модуль радиоуправления имеет собственный конечный автомат проверки работоспособности. Во время активной проверки не допускается перезапись адреса радиомодуля синхронизационным пакетом. Если момент син-

хронизации попадает в период проверки, синхронизация откладывается до завершения проверки. После завершения проверки адрес радиомодуля явно возвращается на общий адрес управления.

Модуль приёма временных меток не зависит от модуля радиоуправления, поскольку использует другой радиомодуль и другой радиоканал. Это позволяет принимать события в момент, когда радиомодуль управления выполняет синхронизацию или проверку работоспособности. Модуль группировки событий получает данные из приёмного радиомодуля и не обращается к радиомодулю управления, что снижает связанность программных компонентов.

Сетевой модуль получает уже подготовленную группу временных меток. Если сетевой обмен временно недоступен, диагностические переменные позволяют выявить проблему без нарушения логики приёма радиопакетов. Ведущий модуль рассчитан на работу в условиях, когда не все ведомые узлы доступны постоянно; поэтому отсутствие части временных меток или отрицательный результат проверки работоспособности не приводит к остановке системы. Информация о взаимодействии программных модулей ведущего модуля сведена в таблицу 20.

Таблица 20 — Взаимодействие программных модулей ведущего модуля

Модуль	Входные данные	Выходные данные	Особенности объединения
Радиоуправление	системное время и таймеры	пакеты сброса, синхронизации и проверки	защита от перезаписи адреса во время проверки работоспособности
Приём временных меток	радиопакеты от ведомых узлов	номер канала, временная метка, номер события	дополнительный опрос статуса при пропущенном прерывании
Группировка событий	метки с номером события	полный или неполный набор меток	тайм-аут завершения группы

Продолжение таблицы Таблица 20

Модуль	Входные данные	Выходные данные	Особенности объединения
Сетевая передача	подготовленный набор меток	кадр для сервера	работа через сетевой контроллер
Диагностика	счётчики модулей	данные для мониторинга	используется при модульных тестах

3 Технологическая часть

3.1 Разработка методики модульного тестирования программных модулей ведомого модуля

Модульное тестирование ведомого программно-аппаратного модуля направлено на независимую проверку узлов приёма аудиоданных, выделения импульсного события, ведения локального времени, приёма управляющих радиопакетов и передачи временной метки ведущему модулю. Испытания выполняются с использованием STM32CubeMonitor, при этом контролируются как признаки исправности периферии, так и счётчики программных событий. Информация о методике модульного тестирования ведомого программно-аппаратного модуля сведена в таблицу 21.

Таблица 21 — Методика модульного тестирования ведомого программно-аппаратного модуля

Модуль	Контролируемые признаки	Критерий успешного прохождения
Приём аудиоданных	test24, отсутствие переполнения аудиоблоков	отсчёты находятся в рабочем диапазоне, переполнение отсутствует
Детектор первого прихода	номер события, флаг готовности радиопакета	при импульсном воздействии формируется один пакет события
Локальное время	счётчики сброса и синхронизации	после команды синхронизации обновляется шкала времени
Радиоприём управления	radio_nrf2_present, radio_nrf2_feature, radio_nrf2_en_aa, last_payload	радиомодуль доступен, управляющий пакет имеет ожидаемый код
Радиопередача события	radio_nrf3_tx_ok_count, radio_nrf3_max_rt_count	пакет события передан или ошибка передачи диагностирована

Схема стенда модульного тестирования ведомого программно-аппаратного модуля изображена на рисунке 7.

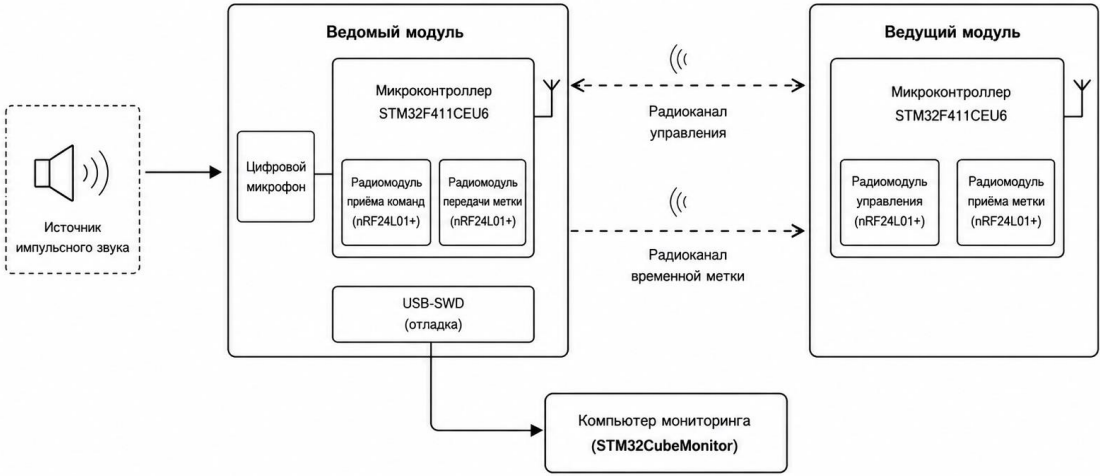


Рисунок 7 — Схема стенда модульного тестирования ведомого программно-аппаратного модуля

3.2 Разработка методики модульного тестирования программных модулей ведущего модуля

Модульное тестирование ведущего программно-аппаратного модуля разделено на проверку радиомодуля синхронизации, радиомодуля приёма временных меток, алгоритма проверки работоспособности ведомых узлов, модуля группировки событий и сетевого обмена с сервером. Для каждого модуля задаётся набор переменных мониторинга и допустимый диапазон значений. Информация о методике модульного тестирования ведущего программно-аппаратного модуля сведена в таблицу 22.

Таблица 22 — Методика модульного тестирования ведущего программно-аппаратного модуля

Модуль	Контролируемые переменные	Критерий успешного прохождения
Передача сброса и синхронизации	g_radio_nrf2_sync_tx_count, g_radio_nrf2_last_payload_spi_cmd, g_radio_nrf2_tx_addr0	частота синхронизации близка к 10 Гц, адрес равен общему адресу управления

Продолжение таблицы Таблица 22

Модуль	Контролируемые переменные	Критерий успешного прохождения
Проверка работоспособности ведомых узлов	g_radio_nrf2_health_tx_count, g_radio_health_mask, g_radio_nrf2_addr_mode	за цикл выполняются четыре передачи по индивидуальным адресам
Приём временных меток	g_radio_timestamp_rx_count, g_radio_last_pipe, g_radio_last_event_id	метка принимается по каналу, соответствующему номеру ведомого модуля
Группировка событий	g_radio_grouped_event_count	метки с одинаковым номером события объединяются в один набор
Сетевая передача	g_w5500_version_debug, состояние сокета, счётчик отправки	контроллер Ethernet определён, сформированный кадр отправлен на сервер

Схема стенда модульного тестирования ведущего программно-аппаратного модуля изображена на рисунке 8.

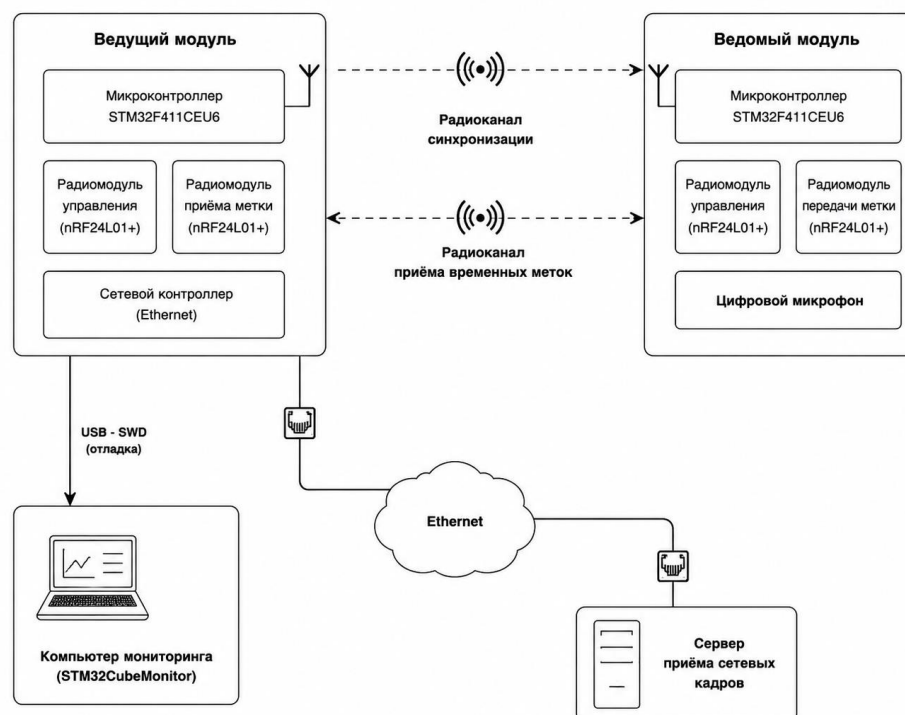


Рисунок 8 — Схема стенда модульного тестирования ведущего модуля

3.3 Проведение модульных тестов программных модулей ведомого модуля

По диагностическому журналу ведомого модуля установлено, что радиомодуль управления доступен по последовательному интерфейсу, установлен канал 40, ширина адреса соответствует пятибайтовой адресации, длина управляющего пакета равна девяти байтам. В ходе предварительного испытания была подтверждена работа индивидуального канала проверки работоспособности и выявлена необходимость включения расширенного режима передачи на общем канале управления. В финальной версии ведомого модуля указанное исправление внесено установкой разрешения подтверждения на первом и втором каналах приёма, при этом пакеты синхронизации передаются ведущим модулем с признаком отсутствия подтверждения. Результаты модульного тестирования радиоприёма ведомого программно-аппаратного модуля сведены в таблицу 23.

Таблица 23 — Результаты модульного тестирования радиоприёма ведомого программно-аппаратного модуля

Показатель	Полученное значение	Норма	Результат
Доступность радиомодуля управления	radio_nrf2_present = 1	1	соответствует
Рабочий радиоканал	radio_nrf2_rf_ch_read = 40	40	соответствует
Ширина адреса	3	3	соответствует
Конфигурация приёмника	radio_nrf2_config = 63	режим приёма, питание включено	соответствует
Регистр расширенных функций	radio_nrf2_feature = 1	1	соответствует
Длина управляющего пакета	9	9 байт	соответствует
Пакеты проверки работоспособности	1, 2, 3	рост счётчика	соответствует
Последний код принятого пакета	162, то есть 0xA2	0xA2 для проверки работоспособности	соответствует
Канал принятого пакета	radio_nrf2_last_pipe = 1	1 для индивидуальной проверки	соответствует
Приём синхронизации до исправления	рост счётчика синхронизации	рост счётчика синхронизации	соответствует

3.4 Проведение модульных тестов программных модулей ведущего модуля

Модульные испытания ведущего модуля выполнены по диагностическому журналу продолжительностью 133,156 с. В журнале подтверждены доступность радиомодуля управления, включение расширенного режима передачи, корректная передача пакетов синхронизации без подтверждения, а также

переключение адресов при проверке работоспособности ведомых узлов. Результаты модульного тестирования радиоприёма ведущего программно-аппаратного модуля сведены в таблицу 24.

Таблица 24 — Результаты модульного тестирования радиоуправления ведущего программно-аппаратного модуля.

Показатель	Полученное значение	Норма	Результат
Доступность радиомодуля управления	g_radio_nrf2_present = 1	1	соответствует
Рабочий радиоканал	g_radio_nrf2_rf_ch_read = 40	40	соответствует
Ширина адреса	g_radio_nrf2_setup_aw_read = 3	3	соответствует
Регистр расширенных функций	g_radio_nrf2_feature = 1	1	соответствует
Код пакета синхронизации	g_radio_nrf2_last_tx_cmd = 161, то есть 0xA1	0xA1	соответствует
Команда загрузки пакета синхронизации	g_radio_nrf2_last_payload_spi_cmd = 176, то есть 0xB0	0xB0	соответствует
Общий адрес синхронизации	g_radio_nrf2_tx_addr0 = 210, то есть 0xD2	0xD2	соответствует
Адрес при проверке работоспособности	0x10, 0x11, 0x12, 0x13	индивидуальные адреса ведомых узлов	соответствует
Возврат общего адреса	после проверки адрес снова 0xD2	возврат обязательен	соответствует

3.5 Разработка методики тестирования производительности ведомого модуля

Тестирование производительности ведомого модуля проводится при непрерывном поступлении аудиоданных и периодическом радиоуправлении со стороны ведущего модуля. Основными критериями являются отсутствие потери аудиоблоков, устойчивость вычисления огибающей и адаптивного порога, однократная фиксация импульсного события, а также отсутствие блокирования радиоприёма длительными операциями обработки сигнала. Информация о показателях производительности ведомого программно-аппаратного модуля сведена в таблицу 25.

Таблица 25 — Показатели производительности ведомого программно-аппаратного модуля

Показатель	Способ контроля	Допустимое значение
Стабильность приёма аудиоотсчётов	контроль test24 и счётчика переполнений	отсчёты в рабочем диапазоне, переполнения отсутствуют
Время обработки аудиоблока	расчёт по частоте дискретизации и размеру блока	меньше времени заполнения половины буфера
Потери радиосинхронизации	radio_nrf2_lost_sync_estimate	отсутствие устойчивого роста
Передача временной метки	radio_nrf3_tx_ok_count и radio_event_ready	пакет формируется после события

3.6 Разработка методики тестирования производительности ведущего модуля

Производительность ведущего модуля оценивается по стабильности периодической синхронизации, длительности цикла проверки работоспособности, способности принять временные метки от четырёх ведомых узлов и сформировать сетевой кадр без нарушения периодичности радиоуправления. Информация о показателе производительности ведущего программно-аппаратного модуля сведена в таблицу 26.

Таблица 26 — Показатели производительности ведущего программно-аппаратного модуля

Показатель	Способ контроля	Допустимое значение
Частота синхронизации	прирост g_radio_nrf2_sync_tx_count за время журнала	10 Гц с отклонением не более 5 процентов
Период проверки работоспособности	интервал между циклами проверки работоспособности	около 60 с
Число передач в цикле проверки	g_radio_nrf2_health_tx_count	4 передачи за цикл
Возврат общего адреса	g_radio_nrf2_addr_mode и первый байт адреса	режим общего адреса после каждого цикла
Стабильность сетевой передачи	состояние W5500 и счётчик отправки	нет зависания основного цикла

3.7 Проведение тестов производительности ведомого модуля

При диагностике ведомого модуля длительностью 137,667 с радиомодуль управления оставался доступным, регистр расширенных функций сохранял значение 1, длина последнего принятого управляющего пакета составляла 9 байт. Счётчик проверки работоспособности увеличился с 1 до 3, что подтвер-

ждает устойчивость индивидуального канала управления. Отдельное предварительное наблюдение аудиоотсчётов test24 показало наличие отсчётов в рабочем диапазоне; нулевые участки в журнале использовались для последующей оптимизации обработки аудиобуфера и исключения блокирующих задержек. Результаты тестов производительности ведомого модуля сведены в таблицу 27.

Таблица 27 — Результаты тестов производительности ведомого модуля

Показатель	Способ контроля	Результат
Стабильность приёма аудиоотсчётов	контроль test24 и счётчика переполнений	отсчёты в рабочем диапазоне, переполнения отсутствуют
Время обработки аудиоблока	расчёт по частоте дискретизации и размеру блока	меньше времени заполнения половины буфера
Потери радиосинхронизации	radio_nrf2_lost_sync_estimate	0
Передача временной метки	radio_nrf3_tx_ok_count и radio_event_ready	radio_nrf3_tx_ok_count растёт, radio_event_ready выставляется в единицу

3.8 Проведение тестов производительности ведущего модуля

По журналу ведущего модуля установлено, что за 133,156 с счётчик отправленных пакетов синхронизации увеличился с 8 до 1305. Средняя частота отправки составила 9,74 Гц, что соответствует требуемому периоду 100 мс с отклонением 2,6 процента. За время наблюдения выполнены циклы проверки работоспособности с переключением адресов 0x10, 0x11, 0x12, 0x13 и возвратом на общий адрес 0xD2. Результаты тестов производительности ведущего модуля сведены в таблицу 28.

Таблица 28 — Результаты тестов производительности ведущего модуля

Показатель	Полученное значение	Норма	Результат
Длительность журнала	133,156 с	не менее 120 с	соответствует
Прирост счётчика синхронизации	1297	около 1330 за 133 с	соответствует
Средняя частота синхронизации	9,74 Гц	10 Гц	соответствует
Отклонение частоты синхронизации	2,6 процента	не более 5 процентов	соответствует
Код команды синхронизации	0xA1	0xA1	соответствует
Команда загрузки без подтверждения	0xB0	0xB0	соответствует
Команда проверки работоспособности	0xA2	0xA2	соответствует
Команда загрузки с подтверждением	0xA0	0xA0	соответствует
Первый байт общего адреса	0xD2	0xD2	соответствует
Индивидуальные адреса проверки	0x10, 0x11, 0x12, 0x13	четыре адреса ведомых узлов	соответствует

3.9 Тест работоспособности системы из ведущего и четырёх ведомых модулей

Интеграционное испытание системы выполняется после успешной проверки ведущего и одного ведомого программно-аппаратного модуля. В испытании используются четыре ведомых модуля, расположенные в вершинах тетраэдра, и один ведущий узел. Система считается работоспособной, если ведущий модуль получает временные метки от нескольких ведомых узлов с одинаковым номером события, выполняет группировку и передаёт результат на сервер. Протокол интеграционного испытания системы представлен в таблице 29.

Таблица 29 — Результаты интеграционного испытания системы

Этап	Ожидаемое состояние	Критерий успешности	Результат
Инициализация системы	ведущий и четыре ведомых модуля включены	все радиомодули доступны	соответствует
Сброс времени	ведущий отправляет общий пакет сброса	счётчики сброса ведомых узлов увеличены	соответствует
Периодическая синхронизация	ведущий передаёт общий пакет 10 раз в секунду	счётчики синхронизации ведомых узлов растут	соответствует
Проверка работоспособности	ведущий опрашивает адреса 0x10-0x13	маска работоспособности соответствует включённым модулям	соответствует
Импульсное событие	ведомые узлы формируют временные метки	ведущий получает четыре метки	соответствует
Передача результата	ведущий отправляет сетевой кадр	сервер принимает пакет с временными метками	соответствует

3.10 Тестирование системы в реальных условиях

Для комплексной оценки работоспособности разработанного программного комплекса и верификации результатов, полученных в ходе модульного и нагрузочного тестирования, были проведены натурные испытания с использованием реального оборудования. Целью данного этапа являлась проверка корректности взаимодействия всех компонентов системы при передаче данных по физическому сетевому каналу, оценка устойчивости алгоритма локализации к

реальным задержкам передачи, синхронизации и внешним акустическим возмущениям, а также подтверждение точности определения координат источника звука в условиях, приближённых к эксплуатационным.

Экспериментальный стенд помимо одного ведущего и четырёх ведомых модулей включал сервер локализации, развёрнутый на персональном компьютере с операционной системой Windows. Ведомые модули были размещены в пространстве согласно конфигурации тетраэдра: ведомый модуль 1 – в точке (0.0, 0.0, 8.16), ведомый модуль 2 – (5.77, 0.0, 0.0), ведомый модуль 3 – (-2.89, 5.0, 0.0), ведомый модуль 4 – (-2.89, -5.0, 0.0).

Узел агрегации был подключён к серверу локализации посредством проводного Ethernet-соединения, что гарантировало стабильную передачу данных без потерь пакетов и с минимальной задержкой. В качестве источника акустического сигнала использовались пиротехнические изделия типа «Корсар К0401», создающие кратковременный импульсный звуковой сигнал с высоким уровнем звукового давления и широкой полосой частот. Выбор данного источника обусловлен его способностью генерировать чёткий, хорошо детектируемый фронт волны, что упрощает выделение временной метки прихода сигнала на каждом сенсорном узле.

Источник звукового сигнала представлен на рисунке 9:



Рисунок 9 – Источник звукового сигнала

Процедура испытаний включала последовательное размещение источника звука в контрольных точках с известными координатами, инициирование акустического события, сбор временных меток ведомыми модулями, агрегацию данных на ведущем модуле и передачу пакета на сервер для обработки.

На рисунке 10 представлен модуль, расположенный на высоте 8.16 м:



Рисунок 10 – Узел, расположенный на высоте 8.16 м

На рисунке 11 представлен один из ведомых узлов:

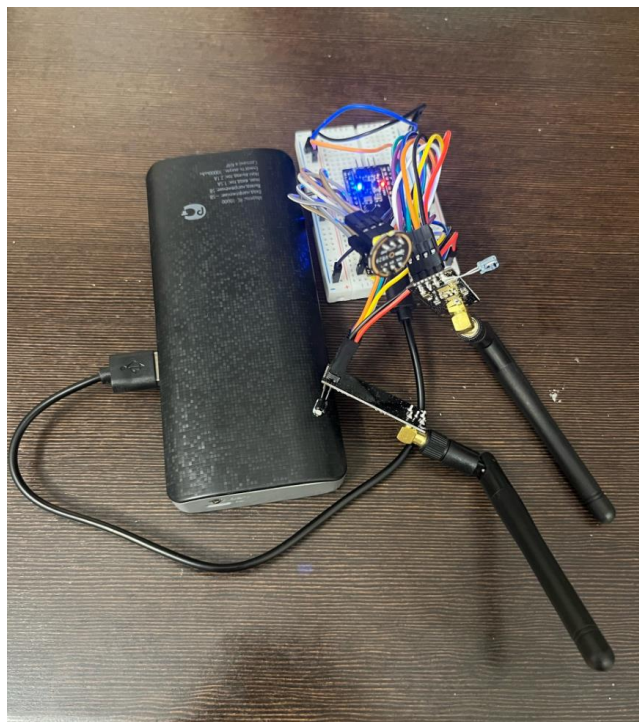


Рисунок 11 – Ведомый модуль

На рисунке 12 представлен ведущий модуль, подключенный к серверу через Ethernet.

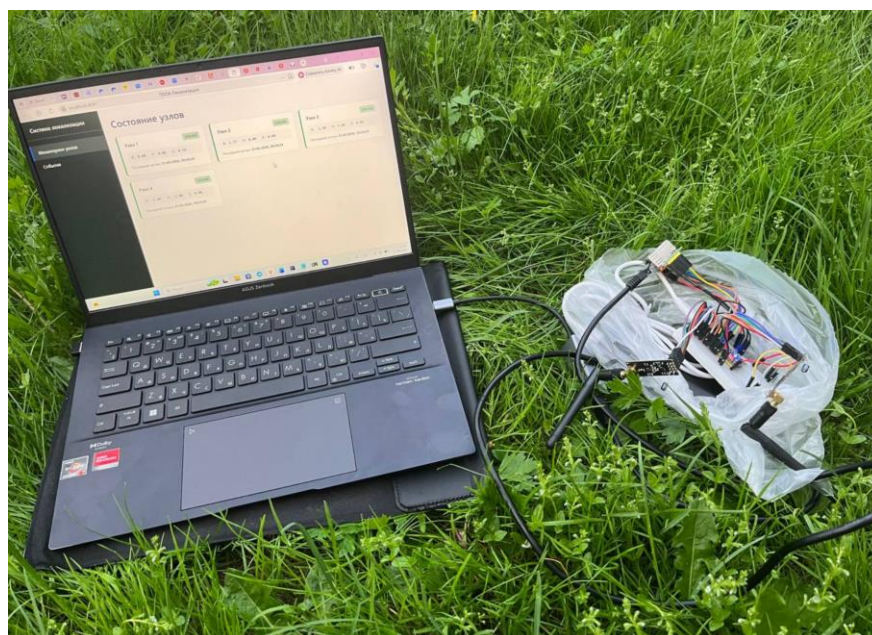


Рисунок 12 – Ведущий модуль, подключенный к серверу

Для комплексной оценки точности алгоритма локализации в различных направлениях относительно базы датчиков была разработана методика размещения контрольных точек, обеспечивающая равномерное покрытие рабочего пространства. Контрольные точки размещались по окружностям с центром в начале координат на трёх характерных радиусах: 40 м (ближняя зона), 70 м (средняя зона) и 100 м (дальняя зона). На каждой окружности точки располагались с угловым шагом 60° , что обеспечивает шестиугольную симметрию и равномерное покрытие. Были протестированы 18 точек вокруг системы локализации по радиусам 40 м, 70 м и 100 м. Все контрольные точки располагались в плоскости OXY (координата $z = 0$), что соответствует типичному сценарию наземного размещения источников звука и упрощает вычисление координат с помощью лазерного дальномера. Результаты испытаний представлены в таблице 30.

Таблица 30 – Результаты тестирования

Координаты источника (м)	Вычисленные алгоритмом локализации координаты (м)	Погрешность вычисления алгоритмом (м)	Относительная погрешность вычисления алгоритмом (%)	Вычисленные системой локализации координаты (м)	Погрешность вычисления системой (м)	Относительная погрешность вычисления системой (%)
(35, -20, 0)	(34.92, -19.96, 0.01)	0.09	0.22	(32.91, -18.39, -0.08)	2.64	6.55
(0, -40, 0)	(0.01, -40.08, -0.01)	0.08	0.20	(-0.06, -36.60, 1.09)	3.57	8.93
(-35, -20, 0)	(-35.05, -20.03, -0.01)	0.06	0.15	(-31.82, -20.23, -0.55)	3.24	8.03
(-35, 20, 0)	(-35.05, 20.03, 0.01)	0.06	0.15	(-31.61, 20.13, -0.79)	3.48	8.63
(0, 40, 0)	(-0.01, 40.09, -0.01)	0.09	0.23	(0.35, 37.16, 1.50)	3.23	8.08
(35, 20, 0)	(34.92, 19.96, 0.01)	0.09	0.22	(32.45, 18.41, 0.03)	3.01	7.46
(60, -35, 0)	(59.18, -34.54, 0.04)	0.94	1.35	(56.04, -30.23, 1.02)	6.28	9.04
(0, -70, 0)	(0.03, -68.17, 0.08)	1.83	2.62	(-1.92, -64.31, -0.10)	6.00	8.57
(-60, -35, 0)	(-58.61, -34.15, 0.07)	1.63	2.35	(-54.65, -33.79, 0.43)	5.50	7.91
(-60, 35, 0)	(-58.61, 34.15, 0.07)	1.63	2.35	(-56.67, 30.58, -0.41)	5.55	7.99

Продолжение таблицы Таблица 30

Координаты источника (м)	Вычисленные алгоритмом локализации координаты (м)	Погрешность вычисления алгоритмом (м)	Относительная погрешность вычисления алгоритмом (%)	Вычисленные системой локализации координаты (м)	Погрешность вычисления системой (м)	Относительная погрешность вычисления системой (%)
(0, 70, 0)	(0.03, 68.17, 0.08)	1.83	2.62	(2.14, 64.18, -0.13)	6.20	8.86
(60, 35, 0)	(59.18, 34.54, 0.04)	0.94	1.35	(55.52, 33.23, 2.16)	5.28	7.60
(87, -50, 0)	(88.62, -50.90, -0.06)	1.85	1.85	(77.70, -48.56, -1.07)	9.48	9.44
(0, -100, 0)	(0.01, -99.13, 0.03)	0.87	0.87	(-1.73, -92.26, -1.15)	8.01	8.01
(-87, -50, 0)	(-86.86, -49.91, 0.00)	0.17	0.17	(-80.00, -45.39, 0.44)	8.39	8.36
(-87, 50, 0)	(-86.86, 49.91, 0.00)	0.17	0.17	(-86.52, 41.29, 1.24)	8.81	8.78
(0, 100, 0)	(0.01, 99.13, 0.03)	0.87	0.87	(3.12, 91.55, -0.25)	9.01	9.01
(87, 50, 0)	(88.62, 50.90, -0.06)	1.85	1.85	(85.14, 42.38, -0.81)	7.89	7.86

В ближней зоне, на расстоянии 40 метров от центра базы датчиков, узел локализации демонстрирует наивысшую точность: абсолютная погрешность не превышает 3.57 метров, а относительная погрешность составляет в среднем 7.95%. Незначительный разброс значений погрешности между точками с одинаковым радиусом, но различными азимутальными углами, подтверждает корректность выбранной конфигурации тетраэдра и отсутствие выраженной зависимости точности от направления на источник.

В средней зоне, на расстоянии 70 метров, фиксируется ожидаемый рост абсолютной погрешности до 6.28 метров, при этом относительная погрешность остаётся в пределах 9.04%.

В дальней зоне, на расстоянии 100 метров, абсолютной погрешности также растёт до 9.48 метров, при этом относительная погрешность не превышает 9.44%.

Статистическая обработка результатов по всем 18 контрольным точкам показывает, что средняя относительная погрешность алгоритма составляет 8.28%, а максимальное значение не превышает 9.44%. Все полученные результаты удовлетворяют требованию технического задания об ограничении погрешности на уровне 10%. При этом наблюдается устойчивая тенденция: относительная погрешность растёт с увеличением расстояния до источника. График изменения относительной погрешности на различных расстояниях изображён на рисунке 13.

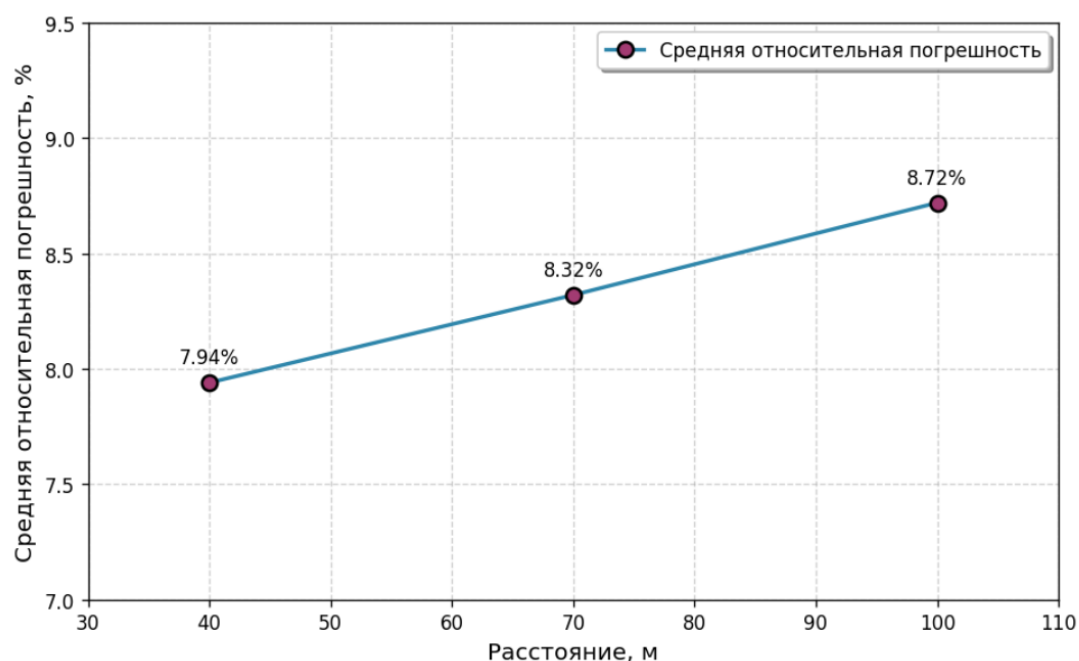


Рисунок 13 –График изменения средней относительной погрешности на различных расстояниях

Равномерность покрытия рабочего пространства контрольными точками и сопоставимость результатов для симметричных конфигураций позволяют сделать вывод о том, что разработанная методика тестирования обеспечивает корректную оценку точности, а выбранный угловой шаг 60° является достаточным для выявления анизотропии, если таковая присутствует.

Отсутствие систематического смещения вычисленных координат в каком-либо одном направлении свидетельствует о корректности калибровки системы и отсутствии существенных методических погрешностей.

Таким образом, экспериментальные исследования подтверждают, что реализованный узел локализации обеспечивает точность, соответствующую составленным требованиям, во всём диапазоне испытанных расстояний и азимутов. Полученные результаты обосновывают применимость разработанного программного комплекса для решения практических задач акустического мониторинга.

ЗАКЛЮЧЕНИЕ

Осуществлена разработка автономного узла распределённой системы акустического мониторинга и локализации источника звука, заключающаяся в следующем:

— разработана архитектура распределённой системы акустической локализации, включающая четыре автономных ведомых измерительных узла и один ведущий координирующий узел, функционально разделённые по задачам приёма аудиоданных, детектирования акустических событий, синхронизации времени и передачи результатов на сервер локализации;

— выбран и обоснован комплект аппаратных средств на базе микроконтроллера STM32F411CEU6, цифрового микрофона INMP441, радиотрансиверов nRF24L01+ для организации беспроводного обмена между модулями и сетевого контроллера W5500 для передачи данных на сервер;

— разработан алгоритм двухступенчатого детектирования импульсных акустических событий, включающий адаптивный порог, отношение кратковременного и долгосрочного средних в целочисленном формате Q8, алгоритм кумулятивной суммы и уточнение момента первого прихода по предтриггерному кольцевому буферу, обеспечивающий формирование временной метки с привязкой к фронту сигнала, а не к моменту завершения обработки;

— разработан протокол синхронизации времени по схеме «ведущий — ведомый» с периодической коррекцией (период 100 мс), обеспечивающий согласованность временных шкал измерительных узлов с погрешностью, не превышающей требований метода TDOA;

— разработаны программные модули приёма и обработки звукового сигнала, обнаружения импульсных событий, управления локальным временем, радиопередачи временных меток и радиоуправления для ведомого модуля, а также модули синхронизации, проверки работоспособности, приёма и группировки событий, сетевой передачи данных для ведущего модуля;

— разработаны методики модульного и комплексного тестирования программно-аппаратных модулей системы, проведены испытания, подтвердившие работоспособность алгоритмов детектирования, синхронизации и передачи данных;

— проведены натурные испытания системы из четырёх ведомых и одного ведущего узлов с использованием импульсных акустических источников (пиротехнические изделия «Корсар К0401»), размещённых в контрольных точках на расстояниях 40 м, 70 м и 100 м от базы датчиков, подтвердившие относительную погрешность локализации не более 10 %, что соответствует разработанным требованиям.

Полученные результаты обосновывают применимость разработанного программно-аппаратного комплекса для решения практических задач акустического мониторинга и локализации источников импульсных звуковых событий в распределённых системах наблюдения.

.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. STMicroelectronics. STM32F411xC STM32F411xE datasheet [Электронный ресурс]// STMicroelectronics. — URL: <https://www.st.com/resource/en/datasheet/stm32f411ce.pdf> (дата обращения: 06.05.2026).
2. TDK InvenSense. INMP441 Omnidirectional Microphone with Bottom Port and I2S Digital Output [Электронный ресурс]// TDK InvenSense. — URL: <https://invensense.tdk.com/download-pdf/inmp441-datasheet/> (дата обращения: 06.05.2026).
3. Virtanen T., Plumbley M.D., Ellis D. Computational Analysis of Sound Scenes and Events [Текст] / T. Virtanen, M.D. Plumbley, D. Ellis. — Cham : Springer International Publishing, 2018. — 422 с.
4. Adavanne S., Politis A., Nikunen J., Virtanen T. Sound event localization and detection of overlapping sources using convolutional recurrent neural networks [Текст] / S. Adavanne, A. Politis, J. Nikunen, T. Virtanen// IEEE Journal of Selected Topics in Signal Processing. — 2019. — Vol. 13, № 1. — P. 34–48.
5. Gairí P., Pallejà T., Tresanchez M. Environmental sound recognition on embedded devices using deep learning: a review [Текст] / P. Gairí, T. Pallejà, M. Tresanchez// Artificial Intelligence Review. — 2025. — Vol. 58, art. 163.
6. Chen J.C. Coherent acoustic array processing and localization on wireless sensor networks [Текст] / J.C. Chen [и др.]// Proceedings of the IEEE. — 2003. — Vol. 91. — P. 1154–1162. — DOI: 10.1109/JPROC.2003.814924.
7. Simon G. Sensor network-based countersniper system [Текст] / G. Simon [и др.]// ACM International Conference on Embedded Networked Sensor Systems. — 2004. — P. 1–12.
8. Sallai J. Weapon classification and shooter localization using distributed multichannel acoustic sensors [Текст] / J. Sallai [и др.]// Journal of Systems Architecture. — 2011. — Vol. 57. — P. 869–885.

9. Hengy S. Acoustic shooter localisation using a network of asynchronous acoustic nodes [Текст] / S. Hengy [и др.]// IET Radar, Sonar & Navigation. — 2016. — DOI: 10.1049/IET-RSN.2016.0029.
10. Sallai J. Acoustic shooter localization with a minimal number of single-channel wireless sensor nodes [Текст] / J. Sallai, Á. Lédeczi, P. Völgyesi. — 2011. — DOI: 10.1145/2070942.2070953.
11. Cobos M. Cumulative-Sum-Based Localization of Sound Events in Low-Cost Wireless Acoustic Sensor Networks [Текст] / M. Cobos, J. Solano, S. Felici-Castell, J. Segura, J.M. Navarro// IEEE/ACM Transactions on Audio, Speech, and Language Processing. — 2014. — Vol. 22. — P. 1792–1802. — DOI: 10.1109/TASLP.2014.2351132.
12. Kim Y., Ahn J., Cha H. Locating Acoustic Events Based on Large-Scale Sensor Networks [Текст] / Y. Kim, J. Ahn, H. Cha// Sensors (Basel, Switzerland). — 2009. — Vol. 9. — P. 9925–9944.
13. Manring I. Low-Cost Gunshot Detection System with Localization for Community Based Violence Interruption [Текст] / I. Manring [и др.]// 2023 IEEE 10th International Conference on Data Science and Advanced Analytics (DSAA). — 2023. — P. 1–7.
14. Svatos J., Holub J. Impulse Acoustic Event Detection, Classification, and Localization System [Текст] / J. Svatos, J. Holub// IEEE Transactions on Instrumentation and Measurement. — 2023. — Vol. 72. — P. 1–15.
15. Astapov S., Ehala J., Preden J. Performing acoustic localization in a network of embedded smart sensors [Текст] / S. Astapov, J. Ehala, J. Preden// International Journal of Microelectronics and Computer Science. — 2015. — Vol. 6.
16. Alameda-Pineda X., Horaud R. A Geometric Approach to Sound Source Localization from Time-Delay Estimates [Текст] / X. Alameda-Pineda, R. Horaud// IEEE Transactions on Audio, Speech, and Language Processing. — 2014. — Vol. 22, № 8. — P. 1292–1304. — DOI: 10.1109/TASLP.2014.2309072.
17. Grondin F., Maheux M.-A., Lauzon J.-S., Vincent J., Michaud F. Fast Cross-Correlation for TDoA Estimation on Small Aperture Microphone Arrays [Текст] / F.

Grondin, M.-A. Maheux, J.-S. Lauzon, J. Vincent, F. Michaud// ICASSP 2023. — 2023. — P. 1–5. — DOI: 10.1109/ICASSP49357.2023.10094989.

18. Freire I., Apolinário J.A. Gunshot detection in noisy environments [Текст] / I. Freire, J.A. Apolinário. — 2010. — DOI: 10.14209/sbrt.2010.122.

19. Shcherbina A., Lukovenkova O., Solodchuk A. Adaptive Threshold Scheme for Pulse Detection under Condition of Background Nonstationary Noise [Текст] / A. Shcherbina, O. Lukovenkova, A. Solodchuk// Journal of Physics: Conference Series. — 2021. — DOI: 10.1088/1742-6596/2096/1/012019.

20. Trnkoczy A. Understanding and parameter setting of STA/LTA trigger algorithm [Текст] / A. Trnkoczy. — 2009. — DOI: 10.2312/GFZ.NMSOP_R1_IS_8.1.

21. Yang H., Dai Y., Cao F., Wu S., Tong X. Transient Event Detection Based on CUSUM-Differential STA/LTA Algorithm [Текст] / H. Yang, Y. Dai, F. Cao, S. Wu, X. Tong// 2025 IEEE 8th International Conference on Automation, Electronics and Electrical Engineering. — 2025. — P. 861–867. — DOI: 10.1109/AUTEEE67053.2025.11321986.

22. Chen Z., Stewart R. Multi-window algorithm for detecting seismic first arrivals [Текст] / Z. Chen, R. Stewart. — 2005.

23. Reddy P.G. Efficient Time of Arrival Calculation for Acoustic Source Localization Using Wireless Sensor Networks [Текст] / P.G. Reddy. — 2011.

24. Raptis T.P., Passarella A., Conti M. A Survey on Industrial Internet With ISA100 Wireless [Текст] / T.P. Raptis, A. Passarella, M. Conti// IEEE Access. — 2020. — Vol. 8. — P. 157177–157196.

25. Nobre M., Silva I., Guedes L.A. Routing and Scheduling Algorithms for WirelessHART Networks: A Survey [Текст] / M. Nobre, I. Silva, L.A. Guedes// Sensors (Basel, Switzerland). — 2015. — Vol. 15. — P. 9703–9740.

26. Scanzio S. [и др.] Wireless Sensor Networks and TSCH: A Compromise Between Reliability, Power Consumption, and Latency [Текст] / S. Scanzio [и др.]// IEEE Access. — 2020. — Vol. 8. — P. 167042–167058.

27. Nordic Semiconductor. nRF24L01P Product Specification. Vers. 1.0 [Электронный ресурс]// SparkFun. — URL: https://cdn.sparkfun.com/assets/3/d/8/5/1/nRF24L01P_Product_Specification_1_0.pdf (дата обращения: 23.05.2026).
28. WIZnet. W5500 [Электронный ресурс]// WIZnet. — URL: <https://wiznet.io/products/ethernet-chips/w5500> (дата обращения: 03.06.2026).
29. WIZnet. ioLibrary_Driver [Электронный ресурс]// GitHub. — URL: https://github.com/Wiznet/ioLibrary_Driver (дата обращения: 03.06.2026).
30. Stanton K. Distributing Deterministic, Accurate Time for Tightly Coordinated Network and Software Applications: IEEE 802.1AS, the TSN profile of PTP [Текст] / K. Stanton// IEEE Communications Standards Magazine. — 2018. — Vol. 2. — P. 34–40.
31. Elson J., Girod L., Estrin D. Fine-grained network time synchronization using reference broadcasts [Текст] / J. Elson, L. Girod, D. Estrin. — 2002.
32. Ganeriwal S., Kumar R., Srivastava M. Timing-sync protocol for sensor networks [Текст] / S. Ganeriwal, R. Kumar, M. Srivastava// ACM International Conference on Embedded Networked Sensor Systems. — 2003. — P. 138–149.
33. Maróti M. [и др.] The flooding time synchronization protocol [Текст] / M. Maróti [и др.]// ACM International Conference on Embedded Networked Sensor Systems. — 2004. — P. 39–49.
34. Ferrari F. [и др.] Efficient network flooding and time synchronization with Glossy [Текст] / F. Ferrari [и др.]// Proceedings of the 10th ACM/IEEE International Conference on Information Processing in Sensor Networks. — 2011. — P. 73–84.
35. Chang T. [и др.] Constructive Interference in 802.15.4: A Tutorial [Текст] / T. Chang [и др.]// IEEE Communications Surveys & Tutorials. — 2019. — Vol. 21. — P. 217–237.
36. Harada M. [и др.] 20- μ s Accuracy Time-Synchronization Method using Bluetooth Low Energy for Internet-of-Things Sensors [Текст] / M. Harada [и др.]// CCNC. — 2022. — P. 181–186.

37. Elsts A. [и др.] Microsecond-Accuracy Time Synchronization Using the IEEE 802.15.4 TSCH Protocol [Текст] / A. Elsts [и др.]// LCN Workshops. — 2016. — P. 156–164.

38. Asgarian F., Najafi K. BlueSync: Time Synchronization in Bluetooth Low Energy With Energy-Efficient Calculations [Текст] / F. Asgarian, K. Najafi// IEEE Internet of Things Journal. — 2022. — Vol. 9. — P. 8633–8645.

ПРИЛОЖЕНИЕ А.

Текст разработанной программы

Модуль main.c ведомого модуля.

```
/* USER CODE BEGIN Header */
/**

*****
*****
* @file           : main.c
* @brief          : Main program body

*****
*****
* @attention
*
* Copyright (c) 2026 STMicroelectronics.
* All rights reserved.
*
* This software is licensed under terms that can be found in
the LICENSE file
* in the root directory of this software component.
* If no LICENSE file comes with this software, it is provided
AS-IS.
*

*****
*****
*/
/* USER CODE END Header */

/* Includes -----
-----*/
```

```

#include "main.h"

/* Private includes -----
-----*/

/* USER CODE BEGIN Includes */
#include <math.h>
#include <string.h>
#include <stdint.h>
/* USER CODE END Includes */

/* Private typedef -----
-----*/

/* USER CODE BEGIN PTD */
typedef enum {
    DET_STATE_QUIET = 0,
    DET_STATE_CANDIDATE,
    DET_STATE_LOCKOUT
} detector_state_t;
/* USER CODE END PTD */

/* Private define -----
-----*/

/* USER CODE BEGIN PD */
#define AUDIO_FS          16000U
#define BLOCK_SAMPLES     512U
#define DMA_HALVES        2U
/* INMP441 with L/R tied to GND transmits the valid microphone
sample in the
    * left I2S slot. STM32 I2S still receives the full stereo
frame: left slot +
    * right slot. Each 24-bit/32-bit slot occupies two 16-bit DMA
half-words.
    * Therefore one valid mono microphone sample occupies four 16-
bit DMA words:
    *   left:  buf[4*i + 0], buf[4*i + 1]

```

```

    *    right: buf[4*i + 2], buf[4*i + 3]    -- ignored
    */

#define I2S_WORDS_PER_SAMPLE 4U

#define I2S_ACTIVE_SLOT_WORD_OFFSET 0U    /* 0 = left slot for
INMP441 L/R=GND; use 2 for right slot */

#define I2S_HALF_WORDS          (BLOCK_SAMPLES * I2S_WORDS_PER_SAM-
PLE)

#define I2S_DMA_WORDS            (BLOCK_SAMPLES * DMA_HALVES *
I2S_WORDS_PER_SAMPLE)

    /* For 24-bit I2S STM32 HAL expands Size internally to 16-bit
DMA transfers. */

#define I2S_HAL_RX_SIZE_PARAM I2S_HALF_WORDS


#define PRETRIGGER_SAMPLES      256U
#define PRETRIGGER_MASK        (PRETRIGGER_SAMPLES - 1U)
#define STA_SAMPLES              8U
#define LTA_SAMPLES             128U
#define STA_LTA_RATIO_Q8        768U
#define TRIGGER_MULTIPLIER      4U
#define ONSET_MULTIPLIER        2U
#define MIN_ONSET_DWELL         4U
#define MIN_SHAPE_DWELL         8U
#define LOCKOUT_SAMPLES         800U
#define CANDIDATE_TIMEOUT       320U
#define DC_SHIFT                 10U
#define ENV_SHIFT                2U
#define NOISE_SHIFT              6U
#define CUSUM_LEAK              96U
#define CUSUM_CONFIRM           24000U


    /* USER CODE END PD */


    /* Private macro -----
-----*/

    /* USER CODE BEGIN PM */

```

```

/* USER CODE END PM */

/* Private variables -----
-----*/

I2S_HandleTypeDef hi2s1;
DMA_HandleTypeDef hdma_spi1_rx;

SPI_HandleTypeDef hspi2;
SPI_HandleTypeDef hspi3;

TIM_HandleTypeDef htim2;

/* USER CODE BEGIN PV */
typedef struct {
    SPI_HandleTypeDef *hspi;
    GPIO_TypeDef *csn_port;
    uint16_t csn_pin;
    GPIO_TypeDef *ce_port;
    uint16_t ce_pin;
} nrf_dev_t;

volatile transient_event_t last_event = {0};
volatile uint64_t last_event_timestamp_us = 0;
volatile uint64_t last_event_sample_index = 0;
volatile uint32_t dma_audio_overruns = 0;
volatile uint8_t recognized_event_id = 0;
volatile uint8_t radio_event_ready = 0;
volatile radio_event_packet_t radio_event_packet = {0};

/* Диагностика радиоканала для STM32CubeMonitor. */
volatile uint8_t SynEnable = 0;
volatile int32_t radio_last_sync_error_us = 0;
volatile int32_t radio_time_correction_us = 0;
volatile uint8_t radio_sync_seen = 0;

```

```

volatile uint8_t radio_nrf3_tx_busy = 0;
volatile uint8_t radio_nrf3_last_status = 0;
volatile uint8_t radio_nrf2_last_status = 0;
volatile uint8_t radio_nrf2_last_pipe = 0;
volatile uint8_t radio_nrf2_last_payload = 0;
volatile uint64_t radio_nrf2_last_sync_master_us = 0;
volatile uint32_t radio_nrf2_last_sync_master_us_lo = 0;
volatile uint32_t radio_nrf2_last_sync_master_us_hi = 0;
volatile uint8_t radio_nrf2_last_payload_len = 0;
volatile uint8_t radio_nrf2_en_aa = 0xFFU;
volatile uint32_t radio_nrf3_max_rt_count = 0;
volatile uint32_t radio_nrf3_tx_ok_count = 0;
volatile uint32_t radio_nrf2_sync_count = 0;
volatile uint32_t radio_nrf2_reset_count = 0;
volatile uint32_t radio_nrf2_health_count = 0;
volatile uint32_t radio_nrf2_lost_sync_estimate = 0;

/* Расширенная диагностика nRF24L01+ для CubeMonitor.
 * present = 1, если по SPI читается RF_CH, равный ожидаемой
частоте.
 * irq_pin_level: 0 означает активное IRQ, 1 - линия отпущена.
 * *_status_after_clear должен иметь биты RX_DR/TX_DS/MAX_RT
сброшенными. */
volatile uint8_t radio_nrf2_present = 0;
volatile uint8_t radio_nrf3_present = 0;
volatile uint8_t radio_nrf2_config = 0;
volatile uint8_t radio_nrf3_config = 0;
volatile uint8_t radio_nrf2_fifo_status = 0;
volatile uint8_t radio_nrf3_fifo_status = 0;
volatile uint8_t radio_nrf2_observe_tx = 0;
volatile uint8_t radio_nrf3_observe_tx = 0;
volatile uint8_t radio_nrf2_rf_ch_read = 0;
volatile uint8_t radio_nrf3_rf_ch_read = 0;
volatile uint8_t radio_nrf2_setup_aw_read = 0;
volatile uint8_t radio_nrf3_setup_aw_read = 0;

```

```

volatile uint8_t radio_nrf2_irq_pin_level = 1;
volatile uint8_t radio_nrf3_irq_pin_level = 1;
volatile uint8_t radio_nrf2_status_after_clear = 0;
volatile uint8_t radio_nrf3_status_after_clear = 0;
volatile uint32_t radio_nrf2_init_clear_count = 0;
volatile uint32_t radio_nrf3_init_clear_count = 0;
volatile uint32_t radio_nrf2_spi_error_count = 0;
volatile uint32_t radio_nrf3_spi_error_count = 0;
volatile uint8_t radio_nrf2_status_nop = 0;
volatile uint8_t radio_nrf3_status_nop = 0;
volatile uint8_t radio_nrf2_present_code = 0;
volatile uint8_t radio_nrf3_present_code = 0;
volatile uint8_t radio_nrf2_feature = 0xFFU;
volatile uint8_t radio_nrf3_feature = 0xFFU;
volatile uint8_t radio_nrf2_dynpd = 0xFFU;
volatile uint8_t radio_nrf3_dynpd = 0xFFU;

/* EXTI must be short: only post these flags. SPI service is
done in main loop. */
volatile uint8_t radio_nrf2_irq_pending = 0;
volatile uint8_t radio_nrf3_irq_pending = 0;

/* Адреса должны совпадать с настройками мастера.
 * NRF3 передаёт timestamp-пакеты на pipe мастера, выделенную
данному slave.
 * NRF2 принимает pipe0 общий sync/reset и pipe1 индивидуальный
health-check. */
/* Адреса согласованы с master_radio_impl_v2/master_radio.c.
 * Для Slave_0:
 *   NRF3 timestamp TX -> master NRF3 RX pipe0: 00 CC BB AA E1
 *   NRF2 common RX    <- master reset/sync:      D2 C2 B2 A2
E2
 *   NRF2 health RX     <- master health slave0:   10 C3 B3 A3
E3

```

```

    * Для Slave_1..3 нужно изменить только первый байт nrf3_mas-
    ter_pipe0_addr
    * на 0x01..0x03 и первый байт nrf2_slave_pipe1_addr на
    0x11..0x13.
    */

    static uint8_t nrf3_master_pipe0_addr[RADIO_ADDR_WIDTH] =
    {0x03U, 0xCCU, 0xBBU, 0xAAU, 0xE1U};

    static uint8_t nrf2_common_pipe0_addr[RADIO_ADDR_WIDTH] =
    {0xD2U, 0xC2U, 0xB2U, 0xA2U, 0xE2U};

    static uint8_t nrf2_slave_pipe1_addr[RADIO_ADDR_WIDTH] =
    {0x13U, 0xC3U, 0xB3U, 0xA3U, 0xE3U};


    static const nrf_dev_t nrf2_rx = {
        .hspi = &hspi2,
        .csn_port = NRF2_CSN_GPIO_Port,
        .csn_pin = NRF2_CSN_Pin,
        .ce_port = NRF2_CE_GPIO_Port,
        .ce_pin = NRF2_CE_Pin
    };

    static const nrf_dev_t nrf3_tx = {
        .hspi = &hspi3,
        .csn_port = NRF3_CSN_GPIO_Port,
        .csn_pin = NRF3_CSN_Pin,
        .ce_port = NRF3_CE_GPIO_Port,
        .ce_pin = NRF3_CE_Pin
    };


    /*
    * 24-битный I2S через 16-битный DMA.
    * Каждый сэмпл = 2 полуслова (half-words) по 16 бит.
    * Двойной буфер: 2 половины * BLOCK_SAMPLES сэмплов * 2 полу-
    слова = BLOCK_SAMPLES * 4
    */

    static uint16_t i2s_buffer[I2S_DMA_WORDS] __attrib-
    ute__((aligned(4)));

```

```

/* ISR only posts completed DMA halves; main loop consumes them
atomically. */
static volatile uint8_t process_pending_mask = 0;
static volatile uint64_t process_block_start_sample[DMA_HALVES] = {0, 0};

volatile uint32_t tim2_ovf = 0;
static uint64_t recording_start_us = 0;
static volatile uint64_t sample_counter = 0;
volatile uint32_t sample_counter_lo = 0U;
volatile uint32_t tim2_counter_snapshot = 0U;

static int16_t pretrigger_pcm[PRETRIGGER_SAMPLES];
static uint16_t pretrigger_env[PRETRIGGER_SAMPLES];
static uint32_t pretrigger_wr = 0;

static detector_state_t det_state = DET_STATE_QUIET;
static int32_t dc_estimate = 0;
static uint32_t env_state = 0;
static uint32_t noise_mean = 512U;
static uint32_t noise_deviation = 64U;
static uint32_t sta_ring[STA_SAMPLES];
static uint32_t lta_ring[LTA_SAMPLES];
static uint32_t sta_sum = 0;
static uint32_t lta_sum = 1U;
static uint32_t sta_pos = 0;
static uint32_t lta_pos = 0;
static uint32_t cusum_pos = 0;
static uint32_t lockout_left = 0;
static uint32_t candidate_age = 0;
static uint32_t candidate_peak = 0;
static uint32_t candidate_peak_sta_lta_q8 = 0;
static uint16_t candidate_dwell = 0;
static uint64_t candidate_start_sample = 0;

```

```

int32_t test24 = 0;
/* USER CODE END PV */

/* Private function prototypes -----
-----*/
void SystemClock_Config(void);
static void MX_GPIO_Init(void);
static void MX_DMA_Init(void);
static void MX_I2S1_Init(void);
static void MX_SPI2_Init(void);
static void MX_SPI3_Init(void);
static void MX_TIM2_Init(void);
/* USER CODE BEGIN PFP */
uint64_t get_time_us(void);
void process_audio_block(uint16_t *buf, uint16_t len, uint64_t
block_start_sample);
static void Radio_Init(void);
static void Radio_ResetInternalTimeAndEvents(void);
static          uint64_t          Radio_ApplySyncCorrection(uint64_t
timestamp_us);
static void Radio_HandleNrf2Rx(void);
static void Radio_HandleNrf3TxIrq(void);
static void nrf_read_payload_buf(const nrf_dev_t *dev, uint8_t
*payload, uint8_t len);
static uint64_t get_u64_le(const uint8_t *p);
static void nrf_clear_all_irqs_and_fifos(const nrf_dev_t *dev,
uint8_t is_nrf3);
static void Radio_UpdateNrfDebugSnapshot(void);
/* USER CODE END PFP */

/* Private user code -----
-----*/
/* USER CODE BEGIN 0 */
static uint8_t nrf_spi_xfer(const nrf_dev_t *dev, uint8_t data)

```

```

    {
        uint8_t rx = 0xFFU;
        HAL_StatusTypeDef st = HAL_SPI_TransmitReceive(dev->hspi,
&data, &rx, 1U, 2U);
        if (st != HAL_OK) {
            if (dev == &nrf2_rx) {
                radio_nrf2_spi_error_count++;
            } else if (dev == &nrf3_tx) {
                radio_nrf3_spi_error_count++;
            }
        }
        return rx;
    }

    static void nrf_csn_low(const nrf_dev_t *dev)
    {
        HAL_GPIO_WritePin(dev->csn_port, dev->csn_pin,
GPIO_PIN_RESET);
    }

    static void nrf_csn_high(const nrf_dev_t *dev)
    {
        HAL_GPIO_WritePin(dev->csn_port, dev->csn_pin,
GPIO_PIN_SET);
    }

    static void nrf_ce_low(const nrf_dev_t *dev)
    {
        HAL_GPIO_WritePin(dev->ce_port, dev->ce_pin, GPIO_PIN_RE-
SET);
    }

    static void nrf_ce_high(const nrf_dev_t *dev)
    {

```

```

        HAL_GPIO_WritePin(dev->ce_port, dev->ce_pin,
GPIO_PIN_SET);
    }

    static uint8_t nrf_read_reg(const nrf_dev_t *dev, uint8_t reg)
    {
        uint8_t val;
        nrf_csn_low(dev);
        nrf_spi_xfer(dev, NRF_CMD_R_REGISTER | (reg & 0x1FU));
        val = nrf_spi_xfer(dev, 0x00U);
        nrf_csn_high(dev);
        return val;
    }

    static void nrf_write_reg(const nrf_dev_t *dev, uint8_t reg,
uint8_t val)
    {
        nrf_csn_low(dev);
        nrf_spi_xfer(dev, NRF_CMD_W_REGISTER | (reg & 0x1FU));
        nrf_spi_xfer(dev, val);
        nrf_csn_high(dev);
    }

    static void nrf_read_multi(const nrf_dev_t *dev, uint8_t reg,
uint8_t *data, uint8_t len)
    {
        nrf_csn_low(dev);
        nrf_spi_xfer(dev, NRF_CMD_R_REGISTER | (reg & 0x1FU));
        for (uint8_t i = 0; i < len; i++) {
            data[i] = nrf_spi_xfer(dev, 0x00U);
        }
        nrf_csn_high(dev);
    }

```

```

static void nrf_write_multi(const nrf_dev_t *dev, uint8_t reg,
const uint8_t *data, uint8_t len)
{
    nrf_csn_low(dev);
    nrf_spi_xfer(dev, NRF_CMD_W_REGISTER | (reg & 0x1FU));
    for (uint8_t i = 0; i < len; i++) {
        nrf_spi_xfer(dev, data[i]);
    }
    nrf_csn_high(dev);
}

static void nrf_command(const nrf_dev_t *dev, uint8_t cmd)
{
    nrf_csn_low(dev);
    nrf_spi_xfer(dev, cmd);
    nrf_csn_high(dev);
}

static void nrf_activate_features(const nrf_dev_t *dev)
{
    /* nRF24L01+ FEATURE/DYNPD registers are locked after reset
on many
        * modules.  ACTIVATE 0x73 unlocks them. Without it,
EN_DYN_ACK can stay
        * zero and NO_ACK sync/reset packets from the master may
be ignored. */
    nrf_csn_low(dev);
    nrf_spi_xfer(dev, 0x50U);
    nrf_spi_xfer(dev, 0x73U);
    nrf_csn_high(dev);
}

static void nrf_write_feature_checked(const nrf_dev_t *dev,
uint8_t value)
{

```

```

        /* ACTIVATE 0x73 toggles FEATURE access. If the MCU resets
but the radio
        * module remains powered, unconditional ACTIVATE may
switch FEATURE access
        * off. Write-read first; ACTIVATE only if FEATURE did not
accept value. */
        nrf_write_reg(dev, NRF_REG_FEATURE, value);
        if (nrf_read_reg(dev, NRF_REG_FEATURE) != value) {
            nrf_activate_features(dev);
            nrf_write_reg(dev, NRF_REG_FEATURE, value);
        }
    }

static uint8_t nrf_get_status_nop(const nrf_dev_t *dev)
{
    uint8_t st;
    nrf_csn_low(dev);
    st = nrf_spi_xfer(dev, NRF_CMD_NOP);
    nrf_csn_high(dev);
    return st;
}

static void nrf_clear_irq(const nrf_dev_t *dev, uint8_t flags)
{
    nrf_write_reg(dev, NRF_REG_STATUS, flags & (NRF_STA-
TUS_RX_DR | NRF_STATUS_TX_DS | NRF_STATUS_MAX_RT));
}

static void nrf_clear_all_irqs_and_fifos(const nrf_dev_t *dev,
uint8_t is_nrf3)
{
    /* nRF24L01+ IRQ is active-low while RX_DR, TX_DS or MAX_RT
is set and

```

```

        * unmasked by CONFIG. On power-up or after a failed TX
these flags may
        * remain latched. Clear them before enabling CE and before
enabling
        * unmasked IRQ sources, otherwise the IRQ line can stay
low forever. */
    nrf_ce_low(dev);
    nrf_write_reg(dev, NRF_REG_STATUS, NRF_STATUS_RX_DR |
NRF_STATUS_TX_DS | NRF_STATUS_MAX_RT);
    nrf_command(dev, NRF_CMD_FLUSH_RX);
    nrf_command(dev, NRF_CMD_FLUSH_TX);
    nrf_write_reg(dev, NRF_REG_STATUS, NRF_STATUS_RX_DR |
NRF_STATUS_TX_DS | NRF_STATUS_MAX_RT);

    if (is_nrf3 != 0U) {
        radio_nrf3_init_clear_count++;
        radio_nrf3_status_after_clear = nrf_read_reg(dev,
NRF_REG_STATUS);
        radio_nrf3_fifo_status = nrf_read_reg(dev,
NRF_REG_FIFO_STATUS);
    } else {
        radio_nrf2_init_clear_count++;
        radio_nrf2_status_after_clear = nrf_read_reg(dev,
NRF_REG_STATUS);
        radio_nrf2_fifo_status = nrf_read_reg(dev,
NRF_REG_FIFO_STATUS);
    }
}

static void Radio_UpdateNrfDebugSnapshot(void)
{
    radio_nrf2_status_nop = nrf_get_status_nop(&nrf2_rx);
    radio_nrf3_status_nop = nrf_get_status_nop(&nrf3_tx);
    radio_nrf2_last_status = nrf_read_reg(&nrf2_rx,
NRF_REG_STATUS);

```

```

        radio_nrf3_last_status      =      nrf_read_reg(&nrf3_tx,
NRF_REG_STATUS);

        radio_nrf2_config  =  nrf_read_reg(&nrf2_rx,  NRF_REG_CON-
FIG);

        radio_nrf3_config  =  nrf_read_reg(&nrf3_tx,  NRF_REG_CON-
FIG);

        radio_nrf2_fifo_status      =      nrf_read_reg(&nrf2_rx,
NRF_REG_FIFO_STATUS);

        radio_nrf3_fifo_status      =      nrf_read_reg(&nrf3_tx,
NRF_REG_FIFO_STATUS);

        radio_nrf2_observe_tx = nrf_read_reg(&nrf2_rx, NRF_REG_OB-
SERVE_TX);

        radio_nrf3_observe_tx = nrf_read_reg(&nrf3_tx, NRF_REG_OB-
SERVE_TX);

        radio_nrf2_rf_ch_read      =      nrf_read_reg(&nrf2_rx,
NRF_REG_RF_CH);

        radio_nrf3_rf_ch_read      =      nrf_read_reg(&nrf3_tx,
NRF_REG_RF_CH);

        radio_nrf2_setup_aw_read    =      nrf_read_reg(&nrf2_rx,
NRF_REG_SETUP_AW);

        radio_nrf3_setup_aw_read    =      nrf_read_reg(&nrf3_tx,
NRF_REG_SETUP_AW);

        radio_nrf2_feature  =  nrf_read_reg(&nrf2_rx,  NRF_REG_FEA-
TURE);

        radio_nrf3_feature  =  nrf_read_reg(&nrf3_tx,  NRF_REG_FEA-
TURE);

        radio_nrf2_en_aa = nrf_read_reg(&nrf2_rx, NRF_REG_EN_AA);
        radio_nrf2_dynpd = nrf_read_reg(&nrf2_rx, NRF_REG_DYNPD);
        radio_nrf3_dynpd = nrf_read_reg(&nrf3_tx, NRF_REG_DYNPD);
        radio_nrf2_irq_pin_level      =      (uint8_t)HAL_GPIO_Read-
Pin(NRF2_IRQ_GPIO_Port, NRF2_IRQ_Pin);

        radio_nrf3_irq_pin_level      =      (uint8_t)HAL_GPIO_Read-
Pin(NRF3_IRQ_GPIO_Port, NRF3_IRQ_Pin);

        radio_nrf2_present_code = 0U;

```

```

        if (radio_nrf2_rf_ch_read == NRF2_FREQ) { radio_nrf2_pre-
sent_code |= 0x01U; }
        if (radio_nrf2_setup_aw_read == 0x03U) { radio_nrf2_pre-
sent_code |= 0x02U; }
        if ((radio_nrf2_status_nop != 0xFFU) && (radio_nrf2_sta-
tus_nop != 0x00U)) { radio_nrf2_present_code |= 0x04U; }
        radio_nrf2_present = ((radio_nrf2_present_code & 0x03U) ==
0x03U) ? 1U : 0U;

        radio_nrf3_present_code = 0U;
        if (radio_nrf3_rf_ch_read == NRF3_FREQ) { radio_nrf3_pre-
sent_code |= 0x01U; }
        if (radio_nrf3_setup_aw_read == 0x03U) { radio_nrf3_pre-
sent_code |= 0x02U; }
        if ((radio_nrf3_status_nop != 0xFFU) && (radio_nrf3_sta-
tus_nop != 0x00U)) { radio_nrf3_present_code |= 0x04U; }
        radio_nrf3_present = ((radio_nrf3_present_code & 0x03U) ==
0x03U) ? 1U : 0U;
    }

static void delay_us_blocking(uint32_t us)
{
    uint64_t start = get_time_us();
    while ((uint64_t)(get_time_us() - start) < us) {
        __NOP();
    }
}

uint64_t get_time_us(void)
{
    uint32_t primask = __get_PRIMASK();
    __disable_irq();
    uint64_t t = ((uint64_t)tim2_ovf << 32) | TIM2->CNT;
    if (primask == 0U) {
        __enable_irq();
    }
}

```

```

    }
    return t;
}

static void set_time_us(uint64_t us)
{
    uint32_t primask = __get_PRIMASK();
    __disable_irq();
    tim2_ovf = (uint32_t)(us >> 32);
    TIM2->CNT = (uint32_t)us;
    if (primask == 0U) {
        __enable_irq();
    }
}

static uint8_t pop_dma_half(uint16_t **block_ptr, uint64_t
*block_start_sample)
{
    uint8_t half_index = 0xFFU;

    uint32_t primask = __get_PRIMASK();
    __disable_irq();
    if (process_pending_mask & 0x01U) {
        process_pending_mask &= (uint8_t)~0x01U;
        half_index = 0U;
        *block_start_sample = process_block_start_sample[0];
    } else if (process_pending_mask & 0x02U) {
        process_pending_mask &= (uint8_t)~0x02U;
        half_index = 1U;
        *block_start_sample = process_block_start_sample[1];
    }
    if (primask == 0U) {
        __enable_irq();
    }
}

```

```

        if (half_index != 0xFFU) {
            *block_ptr    =    &i2s_buffer[(uint32_t)half_index    *
I2S_HALF_WORDS];
        }
        return half_index;
    }

static uint8_t audio_has_pending_block(void)
{
    return process_pending_mask != 0U;
}

static void mark_dma_half_ready(uint8_t half_index)
{
    uint8_t bit = (uint8_t)(1U << half_index);
    uint64_t start_sample = sample_counter;

    process_block_start_sample[half_index] = start_sample;
    sample_counter = start_sample + BLOCK_SAMPLES;
    sample_counter_lo = (uint32_t)sample_counter;
    tim2_counter_snapshot = TIM2->CNT;

    if (process_pending_mask & bit) {
        dma_audio_overruns++;
    }
    process_pending_mask |= bit;
}

void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim)
{
    if (htim->Instance == TIM2) {
        tim2_ovf++;
    }
}

```

```

void HAL_I2S_RxHalfCpltCallback(I2S_HandleTypeDef *hi2s)
{
    if (hi2s->Instance == SPI1) {
        mark_dma_half_ready(0U);
    }
}

void HAL_I2S_RxCpltCallback(I2S_HandleTypeDef *hi2s)
{
    if (hi2s->Instance == SPI1) {
        mark_dma_half_ready(1U);
    }
}

static inline uint32_t u32_abs_i32(int32_t v)
{
    return (v < 0) ? (uint32_t)(-v) : (uint32_t)v;
}

static inline int32_t decode_inmp441_sample24(const uint16_t
*buf, uint16_t i)
{
    uint32_t base = (uint32_t)i * I2S_WORDS_PER_SAMPLE +
I2S_ACTIVE_SLOT_WORD_OFFSET;
    uint32_t raw24 = ((uint32_t)buf[base] << 8) |
((buf[base + 1U] >> 8) & 0xFFU);

    if (raw24 & 0x8000000U) {
        return (int32_t)(raw24 | 0xFF000000U);
    }
    return (int32_t)raw24;
}

static uint32_t detector_threshold(uint32_t multiplier)
{

```

```

        uint32_t thr = noise_mean + noise_deviation * multiplier;
        return (thr < 32U) ? 32U : thr;
    }

static uint32_t sta_lta_ratio_q8(void)
{
    uint32_t lta = (lta_sum == 0U) ? 1U : lta_sum;
    return (uint32_t)((uint64_t)sta_sum * LTA_SAMPLES *
256ULL) /
                                ((uint64_t)lta * STA_SAMPLES));
}

static uint8_t sta_lta_confirmed(void)
{
    uint32_t lta = (lta_sum == 0U) ? 1U : lta_sum;
    return (((uint64_t)sta_sum * LTA_SAMPLES * 256ULL) >
            ((uint64_t)lta * STA_SAMPLES * STA_LTA_RATIO_Q8));
}

static void update_sta_lta(uint32_t env)
{
    sta_sum -= sta_ring[sta_pos];
    sta_ring[sta_pos] = env;
    sta_sum += env;
    if (++sta_pos >= STA_SAMPLES) {
        sta_pos = 0U;
    }

    lta_sum -= lta_ring[lta_pos];
    lta_ring[lta_pos] = env;
    lta_sum += env;
    if (++lta_pos >= LTA_SAMPLES) {
        lta_pos = 0U;
    }
}

```

```

static void update_noise_if_quiet(uint32_t env)
{
    if (det_state != DET_STATE_QUIET || env > detector_thresh-
old(TRIGGER_MULTIPLIER)) {
        return;
    }

    int32_t mean_delta = (int32_t)env - (int32_t)noise_mean;
    int32_t new_mean = (int32_t)noise_mean + (mean_delta >>
NOISE_SHIFT);
    noise_mean = (new_mean < 0) ? 0U : (uint32_t)new_mean;

    uint32_t abs_delta = u32_abs_i32((int32_t)env -
(int32_t)noise_mean);
    int32_t dev_delta = (int32_t)abs_delta - (int32_t)noise_de-
viation;
    int32_t new_dev = (int32_t)noise_deviation + (dev_delta >>
NOISE_SHIFT);
    noise_deviation = (new_dev < 8) ? 8U : (uint32_t)new_dev;
}

static void update_cusum(uint32_t env)
{
    uint32_t baseline = detector_threshold(1U);

    if (env > baseline) {
        uint32_t inc = env - baseline;
        if (inc > 4096U) {
            inc = 4096U;
        }
        cusum_pos += inc;
        if (cusum_pos > (CUSUM_CONFIRM * 4U)) {
            cusum_pos = CUSUM_CONFIRM * 4U;
        }
    }
}

```

```

    } else if (cusum_pos > CUSUM_LEAK) {
        cusum_pos -= CUSUM_LEAK;
    } else {
        cusum_pos = 0U;
    }
}

static uint64_t refine_first_arrival(uint64_t current_sample)
{
    uint32_t onset_thr = detector_threshold(ONSET_MULTIPLIER);

    for (uint32_t back = PRETRIGGER_SAMPLES - MIN_ONSET_DWELL
- 1U;
        back > MIN_ONSET_DWELL;
        back--) {
        uint32_t idx = (pretrigger_wr - back) & PRETRIG-
GER_MASK;
        uint32_t ok = 1U;

        for (uint32_t k = 0; k < MIN_ONSET_DWELL; k++) {
            if (pretrigger_env[(idx + k) & PRETRIGGER_MASK] <
onset_thr) {
                ok = 0U;
                break;
            }
        }

        if (ok) {
            uint32_t prev = (idx - 1U) & PRETRIGGER_MASK;
            if (pretrigger_env[idx] >= pretrigger_env[prev]) {
                return current_sample - back;
            }
        }
    }

    return current_sample;
}

```

```

}

static void publish_event(uint64_t sample_index,
                          uint32_t peak_env,
                          uint32_t peak_sta_lta_q8,
                          uint16_t dwell)
{
    transient_event_t ev;

    ev.sample_index = sample_index;
    ev.timestamp_us = recording_start_us + ((sample_index *
1000000ULL) / AUDIO_FS);
    ev.peak_envelope = peak_env;
    ev.peak_sta_lta_q8 = peak_sta_lta_q8;
    ev.noise_mean = noise_mean;
    ev.noise_deviation = noise_deviation;
    ev.dwell_samples = dwell;
    ev.quality_flags = 0U;
    ev.valid = 1U;

    if (dwell >= MIN_SHAPE_DWELL) {
        ev.quality_flags |= 0x01U;
    }
    if (peak_sta_lta_q8 >= STA_LTA_RATIO_Q8) {
        ev.quality_flags |= 0x02U;
    }
    if (peak_env > detector_threshold(TRIGGER_MULTIPLIER)) {
        ev.quality_flags |= 0x04U;
    }

    uint32_t primask = __get_PRIMASK();
    __disable_irq();
    last_event = ev;
    last_event_sample_index = ev.sample_index;
    last_event_timestamp_us = ev.timestamp_us;
}

```

```

        if (primask == 0U) {
            __enable_irq();
        }

        /* User-requested radio mode: transmit ev.timestamp_us in
the 64-bit packet field. */
        Radio_PublishEventTimestamp(ev.timestamp_us);
        HAL_GPIO_TogglePin(LED_GPIO_Port, LED_Pin);
    }

    static void detector_process_pcm16(int16_t pcm, uint64_t sam-
ple_index)
    {
        uint32_t rect = u32_abs_i32((int32_t)pcm);
        int32_t env_delta = (int32_t)rect - (int32_t)env_state;
        int32_t new_env = (int32_t)env_state + (env_delta >>
ENV_SHIFT);
        uint32_t env = (new_env < 0) ? 0U : (uint32_t)new_env;
        uint32_t threshold;

        env_state = env;
        pretrigger_pcm[pretrigger_wr] = pcm;
        pretrigger_env[pretrigger_wr] = (env > 65535U) ? 65535U :
(uint16_t)env;
        pretrigger_wr = (pretrigger_wr + 1U) & PRETRIGGER_MASK;

        update_sta_lta(env);
        update_noise_if_quiet(env);
        update_cusum(env);
        threshold = detector_threshold(TRIGGER_MULTIPLIER);

        switch (det_state) {
        case DET_STATE_QUIET:
            if (env > threshold) {
                det_state = DET_STATE_CANDIDATE;
            }
        }
    }

```

```

        candidate_age = 0U;
        candidate_peak = env;
        candidate_peak_sta_lta_q8 = sta_lta_ratio_q8();
        candidate_dwell = 1U;
        candidate_start_sample = sample_index;
    }
    break;

case DET_STATE_CANDIDATE:
    candidate_age++;
    if (env > candidate_peak) {
        candidate_peak = env;
    }
    {
        uint32_t ratio = sta_lta_ratio_q8();
        if (ratio > candidate_peak_sta_lta_q8) {
            candidate_peak_sta_lta_q8 = ratio;
        }
    }
    if (env > threshold && candidate_dwell < 65535U) {
        candidate_dwell++;
    }

    if (sta_lta_confirmed() && cusum_pos >= CUSUM_CONFIRM)
    {
        uint64_t ts_sample = refine_first_arrival(sample_index);

        if (ts_sample > candidate_start_sample) {
            ts_sample = candidate_start_sample;
        }

        publish_event(ts_sample, candidate_peak, candidate_peak_sta_lta_q8, candidate_dwell);
        det_state = DET_STATE_LOCKOUT;
        lockout_left = LOCKOUT_SAMPLES;
        cusum_pos = 0U;
    }

```

```

        } else if (candidate_age > CANDIDATE_TIMEOUT && env <
threshold) {
            det_state = DET_STATE_QUIET;
            cusum_pos = 0U;
        }
        break;

case DET_STATE_LOCKOUT:
    if (lockout_left > 0U) {
        lockout_left--;
    } else {
        det_state = DET_STATE_QUIET;
        candidate_age = 0U;
        candidate_peak = 0U;
        candidate_peak_sta_lta_q8 = 0U;
        candidate_dwell = 0U;
    }
    break;

default:
    det_state = DET_STATE_QUIET;
    break;
}

}

void process_audio_block(uint16_t *buf, uint16_t len, uint64_t
block_start_sample)
{
    for (uint16_t i = 0; i < len; i++) {
        int32_t sample24 = decode_inmp441_sample24(buf, i);
        test24 = sample24;

        dc_estimate += (sample24 - dc_estimate) >> DC_SHIFT;
        int32_t filtered24 = sample24 - dc_estimate;
        int32_t pcm32 = filtered24 >> 8;
    }
}

```

```

        if (pcm32 > 32767) {
            pcm32 = 32767;
        } else if (pcm32 < -32768) {
            pcm32 = -32768;
        }

        detector_process_pcm16((int16_t)pcm32,
block_start_sample + i);
    }
}

static void nrf3_init_ptx_timestamp(void)
{
    nrf_ce_low(&nrf3_tx);
    HAL_Delay(5U);

    nrf_write_reg(&nrf3_tx, NRF_REG_CONFIG, 0x0CU);          /*
PWR_DOWN, 2-byte CRC, PTX */
    nrf_clear_all_irqs_and_fifos(&nrf3_tx, 1U);
    nrf_write_reg(&nrf3_tx, NRF_REG_EN_AA, 0x01U);          /*
Auto ACK on pipe 0 */
    nrf_write_reg(&nrf3_tx, NRF_REG_EN_RXADDR, 0x01U);      /*
Pipe 0 enabled for ACK reception */
    nrf_write_reg(&nrf3_tx, NRF_REG_SETUP_AW, 0x03U);      /* 5-
byte addresses */
    nrf_write_reg(&nrf3_tx, NRF_REG_SETUP_RETR, 0x3FU);     /*
750 us, 15 retries */
    nrf_write_reg(&nrf3_tx, NRF_REG_RF_CH, NRF3_FREQ);
    nrf_write_reg(&nrf3_tx, NRF_REG_RF_SETUP, 0x0EU);      /* 2
Mbps, 0 dBm */
    nrf_write_multi(&nrf3_tx, NRF_REG_TX_ADDR, nrf3_mas-
ter_pipe0_addr, RADIO_ADDR_WIDTH);
    nrf_write_multi(&nrf3_tx, NRF_REG_RX_ADDR_P0, nrf3_mas-
ter_pipe0_addr, RADIO_ADDR_WIDTH);
    nrf_write_reg(&nrf3_tx, NRF_REG_DYNPD, 0x00U);

```

```

    nrf_write_reg(&nrf3_tx, NRF_REG_FEATURE, 0x00U);
    nrf_clear_all_irqs_and_fifos(&nrf3_tx, 1U);

    /* IRQ: RX_DR masked, TX_DS and MAX_RT unmasked. */
    nrf_write_reg(&nrf3_tx, NRF_REG_CONFIG, 0x4EU);
    HAL_Delay(2U);
    nrf_clear_all_irqs_and_fifos(&nrf3_tx, 1U);
}

static void nrf2_init_prx_control(void)
{
    nrf_ce_low(&nrf2_rx);
    HAL_Delay(5U);

    nrf_write_reg(&nrf2_rx, NRF_REG_CONFIG, 0x0CU);          /*
PWR_DOWN, 2-byte CRC */
    nrf_clear_all_irqs_and_fifos(&nrf2_rx, 0U);
    nrf_write_reg(&nrf2_rx, NRF_REG_EN_AA, 0x03U);          /*
Enhanced ShockBurst on pipe0+pipe1; NO_ACK bit suppresses ACK for
sync/reset */
    nrf_write_reg(&nrf2_rx, NRF_REG_EN_RXADDR, 0x03U);      /*
Pipe 0 common + pipe 1 individual */
    nrf_write_reg(&nrf2_rx, NRF_REG_SETUP_AW, 0x03U);      /* 5-
byte addresses */
    nrf_write_reg(&nrf2_rx, NRF_REG_SETUP_RETR, 0x00U);
    nrf_write_reg(&nrf2_rx, NRF_REG_RF_CH, NRF2_FREQ);
    nrf_write_reg(&nrf2_rx, NRF_REG_RF_SETUP, 0x0EU);      /* 2
Mbps, 0 dBm */
    nrf_write_multi(&nrf2_rx, NRF_REG_RX_ADDR_P0, nrf2_com-
mon_pipe0_addr, RADIO_ADDR_WIDTH);
    nrf_write_multi(&nrf2_rx, NRF_REG_RX_ADDR_P1,
nrf2_slave_pipe1_addr, RADIO_ADDR_WIDTH);
    nrf_write_reg(&nrf2_rx, NRF_REG_RX_PW_P0, RADIO_CTRL_PAY-
LOAD_LEN);

```

```

        nrf_write_reg(&nrf2_rx, NRF_REG_RX_PW_P1, RADIO_CTRL_PAYLOAD_LEN);

        nrf_write_reg(&nrf2_rx, NRF_REG_DYNPD, 0x00U);
        nrf_write_feature_checked(&nrf2_rx, 0x01U);          /*
Accept NO_ACK packets from master */
        nrf_clear_all_irqs_and_fifos(&nrf2_rx, 0U);

        /* IRQ: RX_DR unmasked, TX_DS/MAX_RT masked, PRX enabled.
*/

        nrf_write_reg(&nrf2_rx, NRF_REG_CONFIG, 0x3FU);
        HAL_Delay(2U);
        nrf_clear_all_irqs_and_fifos(&nrf2_rx, 0U);
        nrf_ce_high(&nrf2_rx);
        HAL_Delay(2U);
    }

static void Radio_Init(void)
{
    nrf_csn_high(&nrf2_rx);
    nrf_csn_high(&nrf3_tx);
    nrf_ce_low(&nrf2_rx);
    nrf_ce_low(&nrf3_tx);
    radio_nrf2_irq_pending = 0U;
    radio_nrf3_irq_pending = 0U;
    radio_nrf3_tx_busy = 0U;
    nrf3_init_ptx_timestamp();
    nrf2_init_prx_control();
    Radio_UpdateNrfDebugSnapshot();
}

static void nrf_read_payload_buf(const nrf_dev_t *dev, uint8_t
*payload, uint8_t len)
{
    nrf_csn_low(dev);
    nrf_spi_xfer(dev, NRF_CMD_R_RX_PAYLOAD);

```

```

        for (uint8_t i = 0U; i < len; i++) {
            payload[i] = nrf_spi_xfer(dev, 0x00U);
        }
        nrf_csn_high(dev);
    }

static uint64_t get_u64_le(const uint8_t *p)
{
    return ((uint64_t)p[0]) |
           ((uint64_t)p[1] << 8) |
           ((uint64_t)p[2] << 16) |
           ((uint64_t)p[3] << 24) |
           ((uint64_t)p[4] << 32) |
           ((uint64_t)p[5] << 40) |
           ((uint64_t)p[6] << 48) |
           ((uint64_t)p[7] << 56);
}

static void nrf3_send_event_packet(const radio_event_packet_t
*pkt)
{
    nrf_ce_low(&nrf3_tx);
    nrf_write_reg(&nrf3_tx, NRF_REG_STATUS, NRF_STATUS_TX_DS |
NRF_STATUS_MAX_RT);
    nrf_command(&nrf3_tx, NRF_CMD_FLUSH_TX);

    nrf_csn_low(&nrf3_tx);
    nrf_spi_xfer(&nrf3_tx, NRF_CMD_W_TX_PAYLOAD);
    const uint8_t *p = (const uint8_t *)pkt;
    for (uint8_t i = 0; i < RADIO_EVENT_PAYLOAD_LEN; i++) {
        nrf_spi_xfer(&nrf3_tx, p[i]);
    }
    nrf_csn_high(&nrf3_tx);

    radio_nrf3_tx_busy = 1U;
}

```

```

        nrf_ce_high(&nrf3_tx);
        delay_us_blocking(20U);
        nrf_ce_low(&nrf3_tx);
    }

    static      uint64_t      Radio_ApplySyncCorrection(uint64_t
timestamp_us)
    {
        if (SynEnable == 0U) {
            return timestamp_us;
        }

        int64_t corrected = (int64_t)timestamp_us + (int64_t)ra-
dio_time_correction_us;
        if (corrected < 0) {
            corrected = 0;
        }
        return (uint64_t)corrected;
    }

void Radio_ProcessMainLoop(void)
{
    static uint32_t last_debug_snapshot_ms = 0U;
    uint8_t handle_nrf2 = 0U;
    uint8_t handle_nrf3 = 0U;

    /* EXTI only posts flags; all nRF SPI transactions are
executed here,
    * outside the interrupt context, so I2S DMA callbacks are
not blocked. */
    uint32_t primask = __get_PRIMASK();
    __disable_irq();
    if (radio_nrf2_irq_pending != 0U) {
        radio_nrf2_irq_pending = 0U;
        handle_nrf2 = 1U;
    }

```

```

    }

    if (radio_nrf3_irq_pending != 0U) {
        radio_nrf3_irq_pending = 0U;
        handle_nrf3 = 1U;
    }

    if (primask == 0U) {
        __enable_irq();
    }

    if (handle_nrf2 != 0U) {
        Radio_HandleNrf2Rx();
    }

    if (handle_nrf3 != 0U) {
        Radio_HandleNrf3TxIrq();
    }

    if ((HAL_GetTick() - last_debug_snapshot_ms) >= 100U) {
        last_debug_snapshot_ms = HAL_GetTick();
        Radio_UpdateNrfDebugSnapshot();
    }

    if ((radio_event_ready != 0U) && (radio_nrf3_tx_busy ==
0U)) {

        radio_event_packet_t pkt;

        primask = __get_PRIMASK();
        __disable_irq();
        pkt = radio_event_packet;
        radio_event_ready = 0U;
        if (primask == 0U) {
            __enable_irq();
        }

        pkt.timestamp_sample = Radio_ApplySyncCorrec-
tion(pkt.timestamp_sample);
    }

```

```

        nrf3_send_event_packet(&pkt);
    }
}

void Radio_PublishEventTimestamp(uint64_t timestamp_us)
{
    radio_event_packet_t pkt;

    recognized_event_id++; /* uint8_t automatically wraps 255
-> 0. */

    pkt.timestamp_sample = timestamp_us; /* User-requested
mode: transmit ev.timestamp_us in this field. */
    pkt.event_id = recognized_event_id;

    uint32_t primask = __get_PRIMASK();
    __disable_irq();
    radio_event_packet = pkt;
    radio_event_ready = 1U;
    last_event_timestamp_us = timestamp_us;
    if (primask == 0U) {
        __enable_irq();
    }
}

static void Radio_ResetInternalTimeAndEvents(void)
{
    uint32_t primask = __get_PRIMASK();
    __disable_irq();
    recognized_event_id = 0U;
    radio_event_ready = 0U;
    radio_event_packet.timestamp_sample = 0U;
    radio_event_packet.event_id = 0U;
    last_event_timestamp_us = 0U;
    last_event_sample_index = 0U;
    last_event.valid = 0U;
}

```

```

sample_counter = 0U;
process_pending_mask = 0U;
process_block_start_sample[0] = 0U;
process_block_start_sample[1] = 0U;
recording_start_us = 0U;
dc_estimate = 0;
env_state = 0U;
noise_mean = 512U;
noise_deviation = 64U;
sta_sum = 0U;
lta_sum = 1U;
sta_pos = 0U;
lta_pos = 0U;
cusum_pos = 0U;
det_state = DET_STATE_QUIET;
candidate_age = 0U;
candidate_peak = 0U;
candidate_peak_sta_lta_q8 = 0U;
candidate_dwell = 0U;
pretrigger_wr = 0U;
memset((void*)pretrigger_pcm, 0, sizeof(pretrigger_pcm));
memset((void*)pretrigger_env, 0, sizeof(pretrigger_env));
memset((void*)sta_ring, 0, sizeof(sta_ring));
memset((void*)lta_ring, 0, sizeof(lta_ring));
radio_time_correction_us = 0;
radio_last_sync_error_us = 0;
radio_sync_seen = 0U;
if (primask == 0U) {
    __enable_irq();
}

set_time_us(0U);
recording_start_us = get_time_us();
nrf_command(&nrf2_rx, NRF_CMD_FLUSH_RX);
nrf_command(&nrf3_tx, NRF_CMD_FLUSH_TX);

```

```

    }

    static void Radio_HandleSyncPacket(uint64_t master_time_us)
    {
        uint64_t actual = get_time_us();
        radio_nrf2_sync_count++;
        radio_sync_seen = 1U;
        radio_nrf2_last_sync_master_us = master_time_us;
        radio_nrf2_last_sync_master_us_lo = (uint32_t)(master_time_us & 0xFFFFFFFFFULL);
        radio_nrf2_last_sync_master_us_hi = (uint32_t)(master_time_us >> 32);

        int64_t diff64 = (int64_t)actual - (int64_t)master_time_us;
        if (diff64 > INT32_MAX) {
            diff64 = INT32_MAX;
        } else if (diff64 < INT32_MIN) {
            diff64 = INT32_MIN;
        }
        radio_last_sync_error_us = (int32_t)diff64;

        if (SynEnable != 0U) {
            set_time_us(master_time_us);
            radio_time_correction_us -= (int32_t)diff64;
        }
    }

    static void Radio_HandleNrf2Rx(void)
    {
        uint8_t status = nrf_read_reg(&nrf2_rx, NRF_REG_STATUS);
        radio_nrf2_last_status = status;

        if ((status & NRF_STATUS_RX_DR) != 0U) {
            uint8_t pipe = (uint8_t)((status & NRF_STATUS_RX_P_NO)
>> 1);

            uint8_t rx_payload[RADIO_CTRL_PAYLOAD_LEN] = {0U};

```

```

        nrf_read_payload_buf(&nrf2_rx,      rx_payload,      RA-
DIO_CTRL_PAYLOAD_LEN);

        radio_nrf2_last_pipe = pipe;
        radio_nrf2_last_payload = rx_payload[0];
        radio_nrf2_last_payload_len = RADIO_CTRL_PAYLOAD_LEN;

        if ((pipe == 0U) && (rx_payload[0] == RA-
DIO_PKT_TIME_RESET)) {
            radio_nrf2_reset_count++;
            nrf_clear_irq(&nrf2_rx, NRF_STATUS_RX_DR);
            Radio_ResetInternalTimeAndEvents();
            HAL_GPIO_WritePin(LED_GPIO_Port,
LED_Pin,GPIO_PIN_SET);
            return;
        }
        if ((pipe == 0U) && (rx_payload[0] == RADIO_PKT_SYNC))
        {
            uint64_t  master_time_us  =  get_u64_le(&rx_pay-
load[1]);

            Radio_HandleSyncPacket(master_time_us);
        } else if ((pipe == 1U) && (rx_payload[0] == RA-
DIO_PKT_HEALTH)) {
            /* ACK is generated by nRF24L01+ hardware because
EN_AA pipel is enabled. */
            radio_nrf2_health_count++;
        }
    }

    nrf_clear_irq(&nrf2_rx,  status  &  (NRF_STATUS_RX_DR  |
NRF_STATUS_TX_DS  |  NRF_STATUS_MAX_RT));
}

static void Radio_HandleNrf3TxIrq(void)
{

```

```

uint8_t status = nrf_read_reg(&nrf3_tx, NRF_REG_STATUS);
radio_nrf3_last_status = status;

if ((status & NRF_STATUS_MAX_RT) != 0U) {
    radio_nrf3_max_rt_count++;
    nrf_clear_irq(&nrf3_tx, NRF_STATUS_MAX_RT);
    nrf_command(&nrf3_tx, NRF_CMD_FLUSH_TX);
    radio_nrf3_tx_busy = 0U;
}

if ((status & NRF_STATUS_TX_DS) != 0U) {
    radio_nrf3_tx_ok_count++;
    nrf_clear_irq(&nrf3_tx, NRF_STATUS_TX_DS);
    radio_nrf3_tx_busy = 0U;
}
}

void Radio_IRQ_Handler(uint16_t GPIO_Pin)
{
    /* Never use HAL_SPI_* inside EXTI. NRF IRQ can stay low
while STATUS is
    * uncleared; servicing it here may block DMA/I2S callbacks
for milliseconds. */
    if (GPIO_Pin == NRF2_IRQ_Pin) {
        radio_nrf2_irq_pending = 1U;
    } else if (GPIO_Pin == NRF3_IRQ_Pin) {
        radio_nrf3_irq_pending = 1U;
    }
}

/* USER CODE END 0 */

/**
 * @brief The application entry point.
 * @retval int
 */

```

```

int main(void)
{
    /* USER CODE BEGIN 1 */

    /* USER CODE END 1 */

    /* MCU Configuration-----
-----*/

    /* Reset of all peripherals, Initializes the Flash interface
and the SysTick. */
    HAL_Init();

    /* USER CODE BEGIN Init */

    /* USER CODE END Init */

    /* Configure the system clock */
    SystemClock_Config();

    /* USER CODE BEGIN SysInit */

    /* USER CODE END SysInit */

    /* Initialize all configured peripherals */
    MX_GPIO_Init();
    MX_DMA_Init();
    MX_I2S1_Init();
    MX_SPI2_Init();
    MX_SPI3_Init();
    MX_TIM2_Init();
    /* USER CODE BEGIN 2 */

    HAL_TIM_Base_Start_IT(&htim2);
    recording_start_us = get_time_us();

```

```

Radio_Init();

/*
 * Запуск DMA приёма. INMP441 включён как один микрофон, но
 I2S-приёмник
 * принимает стереокадр: левый слот + правый слот. Для L/R=GND
 полезный
 * сигнал находится в левом слоте. Один моно-сэмпл = 4 полу-
 слова DMA.
 */
HAL_I2S_Receive_DMA(&hi2s1, (uint16_t*)i2s_buffer,
I2S_HAL_RX_SIZE_PARAM);
/* USER CODE END 2 */

/* Infinite loop */
/* USER CODE BEGIN WHILE */
while (1)
{
    /* USER CODE END WHILE */

    /* USER CODE BEGIN 3 */
    uint16_t *block_ptr = NULL;
    uint64_t block_start_sample = 0U;

    while (pop_dma_half(&block_ptr, &block_start_sample) !=
0xFFU) {
        process_audio_block(block_ptr, BLOCK_SAMPLES,
block_start_sample);
        block_ptr = NULL;
    }

    /* Радио обслуживается только когда аудио-очередь пуста.
Это защищает

```

```

        * непрерывный I2S/DMA тракт от блокировок SPI/nRF и
STM32CubeMonitor. */
        if (!audio_has_pending_block()) {
            Radio_ProcessMainLoop();
        }
        /* USER CODE END 3 */
    }
}

/**
 * @brief System Clock Configuration
 * @retval None
 */
void SystemClock_Config(void)
{
    RCC_OscInitTypeDef RCC_OscInitStruct = {0};
    RCC_ClkInitTypeDef RCC_ClkInitStruct = {0};

    /** Configure the main internal regulator output voltage
    */
    __HAL_RCC_PWR_CLK_ENABLE();
    __HAL_PWR_VOLTAGESCALING_CONFIG(PWR_REGULATOR_VOLT-
AGE_SCALE1);

    /** Initializes the RCC Oscillators according to the speci-
fied parameters
    * in the RCC_OscInitTypeDef structure.
    */
    RCC_OscInitStruct.OscillatorType = RCC_OSCILLATORTYPE_HSE;
    RCC_OscInitStruct.HSEState = RCC_HSE_ON;
    RCC_OscInitStruct.PLL.PLLState = RCC_PLL_ON;
    RCC_OscInitStruct.PLL.PLLSource = RCC_PLLSOURCE_HSE;
    RCC_OscInitStruct.PLL.PLLM = 25;
    RCC_OscInitStruct.PLL.PLLN = 200;
    RCC_OscInitStruct.PLL.PLLP = RCC_PLLP_DIV2;

```

```

RCC_OscInitStruct.PLL.PLLQ = 4;
if (HAL_RCC_OscConfig(&RCC_OscInitStruct) != HAL_OK)
{
    Error_Handler();
}

/** Initializes the CPU, AHB and APB buses clocks
 */
RCC_ClkInitStruct.ClockType = RCC_CLOCKTYPE_HCLK|RCC_CLOCK-
TYPE_SYCLK
                                |RCC_CLOCKTYPE_PCLK1|RCC_CLOCK-
TYPE_PCLK2;
RCC_ClkInitStruct.SYSCLKSource = RCC_SYSCLKSOURCE_PLLCLK;
RCC_ClkInitStruct.AHBCLKDivider = RCC_SYSCLK_DIV1;
RCC_ClkInitStruct.APB1CLKDivider = RCC_HCLK_DIV2;
RCC_ClkInitStruct.APB2CLKDivider = RCC_HCLK_DIV1;

if (HAL_RCC_ClockConfig(&RCC_ClkInitStruct, FLASH_LATENCY_3)
!= HAL_OK)
{
    Error_Handler();
}

/**
 * @brief I2S1 Initialization Function
 * @param None
 * @retval None
 */
static void MX_I2S1_Init(void)
{
    hi2s1.Instance = SPI1;
    hi2s1.Init.Mode = I2S_MODE_MASTER_RX;
    hi2s1.Init.Standard = I2S_STANDARD_PHILIPS;

```

```

        hi2s1.Init.DataFormat = I2S_DATAFORMAT_24B;          /* Оставляем
24 бита */
        hi2s1.Init.MCLKOutput = I2S_MCLKOUTPUT_DISABLE;
        hi2s1.Init.AudioFreq = I2S_AUDIOFREQ_16K;
        hi2s1.Init.CPOL = I2S_CPOL_LOW;
        hi2s1.Init.ClockSource = I2S_CLOCK_PLL;
        hi2s1.Init.FullDuplexMode = I2S_FULLDUPLEXMODE_DISABLE;
        if (HAL_I2S_Init(&hi2s1) != HAL_OK)
        {
            Error_Handler();
        }
    }

/**
 * @brief SPI2 Initialization Function
 * @param None
 * @retval None
 */
static void MX_SPI2_Init(void)
{
    hspi2.Instance = SPI2;
    hspi2.Init.Mode = SPI_MODE_MASTER;
    hspi2.Init.Direction = SPI_DIRECTION_2LINES;
    hspi2.Init.DataSize = SPI_DATASIZE_8BIT;
    hspi2.Init.CLKPolarity = SPI_POLARITY_LOW;
    hspi2.Init.CLKPhase = SPI_PHASE_1EDGE;
    hspi2.Init.NSS = SPI_NSS_SOFT;
    hspi2.Init.BaudRatePrescaler = SPI_BAUDRATEPRESCALER_8;
    hspi2.Init.FirstBit = SPI_FIRSTBIT_MSB;
    hspi2.Init.TIMode = SPI_TIMODE_DISABLE;
    hspi2.Init.CRCCalculation = SPI_CRCCALCULATION_DISABLE;
    hspi2.Init.CRCPolynomial = 10;
    if (HAL_SPI_Init(&hspi2) != HAL_OK)
    {
        Error_Handler();
    }
}

```

```

    }
}

/**
 * @brief SPI3 Initialization Function
 * @param None
 * @retval None
 */
static void MX_SPI3_Init(void)
{
    hspi3.Instance = SPI3;
    hspi3.Init.Mode = SPI_MODE_MASTER;
    hspi3.Init.Direction = SPI_DIRECTION_2LINES;
    hspi3.Init.DataSize = SPI_DATASIZE_8BIT;
    hspi3.Init.CLKPolarity = SPI_POLARITY_LOW;
    hspi3.Init.CLKPhase = SPI_PHASE_1EDGE;
    hspi3.Init.NSS = SPI_NSS_SOFT;
    hspi3.Init.BaudRatePrescaler = SPI_BAUDRATEPRESCALER_8;
    hspi3.Init.FirstBit = SPI_FIRSTBIT_MSB;
    hspi3.Init.TIMode = SPI_TIMODE_DISABLE;
    hspi3.Init.CRCCalculation = SPI_CRCCALCULATION_DISABLE;
    hspi3.Init.CRCPolynomial = 10;
    if (HAL_SPI_Init(&hspi3) != HAL_OK)
    {
        Error_Handler();
    }
}

/**
 * @brief TIM2 Initialization Function
 * @param None
 * @retval None
 */
static void MX_TIM2_Init(void)
{

```

```

TIM_ClockConfigTypeDef sClockSourceConfig = {0};
TIM_MasterConfigTypeDef sMasterConfig = {0};

htim2.Instance = TIM2;
htim2.Init.Prescaler = 99;
htim2.Init.CounterMode = TIM_COUNTERMODE_UP;
htim2.Init.Period = 4294967295;
htim2.Init.ClockDivision = TIM_CLOCKDIVISION_DIV1;
htim2.Init.AutoReloadPreload = TIM_AUTORELOAD_PRELOAD_DISABLE;

if (HAL_TIM_Base_Init(&htim2) != HAL_OK)
{
    Error_Handler();
}
sClockSourceConfig.ClockSource = TIM_CLOCKSOURCE_INTERNAL;
if (HAL_TIM_ConfigClockSource(&htim2, &sClockSourceConfig)
!= HAL_OK)
{
    Error_Handler();
}
sMasterConfig.MasterOutputTrigger = TIM_TRGO_RESET;
sMasterConfig.MasterSlaveMode = TIM_MASTERSLAVEMODE_DISABLE;
if (HAL_TIMEx_MasterConfigSynchronization(&htim2, &sMaster-
Config) != HAL_OK)
{
    Error_Handler();
}
}

/**
 * Enable DMA controller clock
 */
static void MX_DMA_Init(void)
{
    __HAL_RCC_DMA2_CLK_ENABLE();
}

```

```

    HAL_NVIC_SetPriority(DMA2_Stream0_IRQn, 0, 0);
    HAL_NVIC_EnableIRQ(DMA2_Stream0_IRQn);
}

/**
 * @brief GPIO Initialization Function
 * @param None
 * @retval None
 */
static void MX_GPIO_Init(void)
{
    GPIO_InitTypeDef GPIO_InitStruct = {0};

    __HAL_RCC_GPIOC_CLK_ENABLE();
    __HAL_RCC_GPIOH_CLK_ENABLE();
    __HAL_RCC_GPIOA_CLK_ENABLE();
    __HAL_RCC_GPIOB_CLK_ENABLE();

    HAL_GPIO_WritePin(LED_GPIO_Port, LED_Pin, GPIO_PIN_SET);
    HAL_GPIO_WritePin(NRF2_CSN_GPIO_Port, NRF2_CSN_Pin,
GPIO_PIN_SET);
    HAL_GPIO_WritePin(GPIOA, NRF2_CE_Pin|NRF3_CE_Pin,
GPIO_PIN_RESET);
    HAL_GPIO_WritePin(NRF3_CSN_GPIO_Port, NRF3_CSN_Pin,
GPIO_PIN_SET);

    GPIO_InitStruct.Pin = LED_Pin;
    GPIO_InitStruct.Mode = GPIO_MODE_OUTPUT_PP;
    GPIO_InitStruct.Pull = GPIO_NOPULL;
    GPIO_InitStruct.Speed = GPIO_SPEED_FREQ_LOW;
    HAL_GPIO_Init(LED_GPIO_Port, &GPIO_InitStruct);

    GPIO_InitStruct.Pin = NRF2_CSN_Pin;
    GPIO_InitStruct.Mode = GPIO_MODE_OUTPUT_PP;
    GPIO_InitStruct.Pull = GPIO_NOPULL;

```

```

GPIO_InitStruct.Speed = GPIO_SPEED_FREQ_HIGH;
HAL_GPIO_Init(NRF2_CSN_GPIO_Port, &GPIO_InitStruct);

GPIO_InitStruct.Pin = NRF2_IRQ_Pin|NRF3_IRQ_Pin;
GPIO_InitStruct.Mode = GPIO_MODE_IT_FALLING;
GPIO_InitStruct.Pull = GPIO_PULLUP;
HAL_GPIO_Init(GPIOA, &GPIO_InitStruct);

HAL_NVIC_SetPriority(EXTI9_5_IRQn, 1, 0);
HAL_NVIC_EnableIRQ(EXTI9_5_IRQn);
HAL_NVIC_SetPriority(EXTI15_10_IRQn, 1, 1);
HAL_NVIC_EnableIRQ(EXTI15_10_IRQn);

GPIO_InitStruct.Pin = NRF2_CE_Pin|NRF3_CE_Pin;
GPIO_InitStruct.Mode = GPIO_MODE_OUTPUT_PP;
GPIO_InitStruct.Pull = GPIO_NOPULL;
GPIO_InitStruct.Speed = GPIO_SPEED_FREQ_HIGH;
HAL_GPIO_Init(GPIOA, &GPIO_InitStruct);

GPIO_InitStruct.Pin = NRF3_CSN_Pin;
GPIO_InitStruct.Mode = GPIO_MODE_OUTPUT_PP;
GPIO_InitStruct.Pull = GPIO_NOPULL;
GPIO_InitStruct.Speed = GPIO_SPEED_FREQ_LOW;
HAL_GPIO_Init(NRF3_CSN_GPIO_Port, &GPIO_InitStruct);
}

/* USER CODE BEGIN 4 */

/* USER CODE END 4 */

/**
 * @brief This function is executed in case of error occurrence.
 *
 * @retval None
 */

```

```

void Error_Handler(void)
{
    __disable_irq();
    while (1)
    {
    }
}

#ifdef USE_FULL_ASSERT
/**
 * @brief Reports the name of the source file and the source
line number
 *
 * where the assert_param error has occurred.
 * @param file: pointer to the source file name
 * @param line: assert_param error line source number
 * @retval None
 */
void assert_failed(uint8_t *file, uint32_t line)
{
}
#endif /* USE_FULL_ASSERT */

Модуль main.c ведущего модуля.

/* Includes -----
-----*/

#include "main.h"
#include "spi.h"
#include "tim.h"
#include "gpio.h"

/* Private includes -----
-----*/

/* USER CODE BEGIN Includes */
#include "w5500_port.h"
#include "master_radio.h"
/* USER CODE END Includes */

```

```

/* Private typedef -----
-----*/
/* USER CODE BEGIN PTD */

/* USER CODE END PTD */

/* Private define -----
-----*/
/* USER CODE BEGIN PD */

/* USER CODE END PD */

/* Private macro -----
-----*/
/* USER CODE BEGIN PM */

/* USER CODE END PM */

/* Private variables -----
-----*/

/* USER CODE BEGIN PV */
static uint32_t g_last_test_send_ms = 0U;
static w5500_port_status_t g_w5500_status = W5500_PORT_OK;
/* USER CODE END PV */

/* Private function prototypes -----
-----*/
void SystemClock_Config(void);
/* USER CODE BEGIN PFP */

/* USER CODE END PFP */

```

```

/* Private user code -----
-----*/

/* USER CODE BEGIN 0 */

/* USER CODE END 0 */

/**
 * @brief The application entry point.
 * @retval int
 */
int main(void)
{

/* USER CODE BEGIN 1 */

/* USER CODE END 1 */

/* MCU Configuration-----
-----*/

/* Reset of all peripherals, Initializes the Flash interface
and the SysTick. */
HAL_Init();

/* USER CODE BEGIN Init */

/* USER CODE END Init */

/* Configure the system clock */
SystemClock_Config();

/* USER CODE BEGIN SysInit */

/* USER CODE END SysInit */

```

```

/* Initialize all configured peripherals */
MX_GPIO_Init();
MX_SPI2_Init();
MX_SPI3_Init();
MX_TIM2_Init();
MX_SPI1_Init();
/* USER CODE BEGIN 2 */
if (HAL_TIM_Base_Start_IT(&htim2) != HAL_OK)
{
    Error_Handler();
}

g_w5500_status = W5500_Init();
if (g_w5500_status != W5500_PORT_OK)
{
    //Error_Handler();
    HAL_GPIO_TogglePin(LED_GPIO_Port, LED_Pin);
}

MasterRadio_Init();
//g_last_test_send_ms = HAL_GetTick();
/* USER CODE END 2 */

/* Infinite loop */
/* USER CODE BEGIN WHILE */
while (1)
{
    /* USER CODE END WHILE */

    /* USER CODE BEGIN 3 */
    W5500_Service();
    MasterRadio_Service();

    /*if ((HAL_GetTick() - g_last_test_send_ms) >= 1000U)
    {

```

```

        g_last_test_send_ms = HAL_GetTick();
        HAL_GPIO_TogglePin(LED_GPIO_Port, LED_Pin);
    }*/
}
/* USER CODE END 3 */
}

/**
 * @brief System Clock Configuration
 * @retval None
 */
void SystemClock_Config(void)
{
    RCC_OscInitTypeDef RCC_OscInitStruct = {0};
    RCC_ClkInitTypeDef RCC_ClkInitStruct = {0};

    /** Configure the main internal regulator output voltage
     */
    __HAL_RCC_PWR_CLK_ENABLE();
    __HAL_PWR_VOLTAGESCALING_CONFIG(PWR_REGULATOR_VOLT-
AGE_SCALE1);

    /** Initializes the RCC Oscillators according to the speci-
    fied parameters
     * in the RCC_OscInitTypeDef structure.
     */
    RCC_OscInitStruct.OscillatorType = RCC_OSCILLATORTYPE_HSE;
    RCC_OscInitStruct.HSEState = RCC_HSE_ON;
    RCC_OscInitStruct.PLL.PLLState = RCC_PLL_ON;
    RCC_OscInitStruct.PLL.PLLSource = RCC_PLLSOURCE_HSE;
    RCC_OscInitStruct.PLL.PLLM = 25;
    RCC_OscInitStruct.PLL.PLLN = 200;
    RCC_OscInitStruct.PLL.PLLP = RCC_PLLP_DIV2;
    RCC_OscInitStruct.PLL.PLLQ = 4;
    if (HAL_RCC_OscConfig(&RCC_OscInitStruct) != HAL_OK)

```

```

    {
        Error_Handler();
    }

    /** Initializes the CPU, AHB and APB buses clocks
    */
    RCC_ClkInitStruct.ClockType = RCC_CLOCKTYPE_HCLK|RCC_CLOCK-
TYPE_SYSCLK
                                   |RCC_CLOCKTYPE_PCLK1|RCC_CLOCK-
TYPE_PCLK2;

    RCC_ClkInitStruct.SYSCLKSource = RCC_SYSCLKSOURCE_PLLCLK;
    RCC_ClkInitStruct.AHBCLKDivider = RCC_SYSCLK_DIV1;
    RCC_ClkInitStruct.APB1CLKDivider = RCC_HCLK_DIV2;
    RCC_ClkInitStruct.APB2CLKDivider = RCC_HCLK_DIV1;

    if (HAL_RCC_ClockConfig(&RCC_ClkInitStruct, FLASH_LATENCY_3)
!= HAL_OK)
    {
        Error_Handler();
    }
}

/* USER CODE BEGIN 4 */
void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim)
{
    MasterRadio_OnTimerPeriodElapsed(htim);
}

void HAL_GPIO_EXTI_Callback(uint16_t GPIO_Pin)
{
    if (GPIO_Pin == W5500_INT_Pin)
    {
        W5500_SetInterruptFlag();
    }
    else

```

```

    {
        MasterRadio_OnExti(GPIO_Pin);
    }
}
/* USER CODE END 4 */

/**
 * @brief This function is executed in case of error occurrence.
 * @retval None
 */
void Error_Handler(void)
{
    /* USER CODE BEGIN Error_Handler_Debug */
    /* User can add his own implementation to report the HAL error return state */
    __disable_irq();
    while (1)
    {
    }
    /* USER CODE END Error_Handler_Debug */
}

#ifdef USE_FULL_ASSERT
/**
 * @brief Reports the name of the source file and the source
line number
 *
 * where the assert_param error has occurred.
 * @param file: pointer to the source file name
 * @param line: assert_param error line source number
 * @retval None
 */
void assert_failed(uint8_t *file, uint32_t line)
{
    /* USER CODE BEGIN 6 */

```

```

        /* User can add his own implementation to report the file
name and line number,
        ex: printf("Wrong parameters value: file %s on line
%d\r\n", file, line) */
        /* USER CODE END 6 */
    }
#endif /* USE_FULL_ASSERT */

```

Модуль master_radio.c ведущего модуля.

```

#include "master_radio.h"
#include "spi.h"
#include "tim.h"
#include <string.h>

extern SPI_HandleTypeDef hspi2;
extern SPI_HandleTypeDef hspi3;

/* ===== User-tunable radio parameters
===== */
#ifndef NRF2_FREQ
#define NRF2_FREQ 40U
#endif
#ifndef NRF3_FREQ
#define NRF3_FREQ 76U
#endif

#define NRF_SPI_TIMEOUT_MS 1U
#define NRF_PAYLOAD_SIZE_MAX 32U
#define NRF_ADDR_LEN 5U

#define MASTER_SYNC_START_DELAY_MS 1000U
#define MASTER_SYNC_PERIOD_MS 100U
#define MASTER_HEALTH_PERIOD_MS 60000U
#define MASTER_EVENT_GROUP_TIMEOUT_MS 1000U

#define MASTER_CMD_RESET_TIME 0xA0U

```

```

#define MASTER_CMD_SYNC_TIME                0xA1U
#define MASTER_CMD_HEALTH_CHECK             0xA2U

#define NRF2_RESET_SYNC_ADDR                {0xD2U, 0xC2U, 0xB2U,
0xA2U, 0xE2U}
#define NRF2_HEALTH_ADDR_BASE               {0x10U, 0xC3U, 0xB3U,
0xA3U, 0xE3U}
#define NRF3_TIMESTAMP_ADDR_BASE            {0x00U, 0xCCU, 0xBBU,
0xAAU, 0xE1U}

/* ===== nRF24L01+ commands
===== */

#define NRF_CMD_R_REGISTER                   0x00U
#define NRF_CMD_W_REGISTER                   0x20U
#define NRF_CMD_R_RX_PAYLOAD                 0x61U
#define NRF_CMD_W_TX_PAYLOAD                 0xA0U
#define NRF_CMD_W_TX_PAYLOAD_NO_ACK          0xB0U
#define NRF_CMD_ACTIVATE                     0x50U
#define NRF_CMD_FLUSH_TX                     0xE1U
#define NRF_CMD_FLUSH_RX                     0xE2U
#define NRF_CMD_REUSE_TX_PL                  0xE3U
#define NRF_CMD_NOP                           0xFFU

/* ===== nRF24L01+ registers
===== */

#define NRF_REG_CONFIG                       0x00U
#define NRF_REG_EN_AA                       0x01U
#define NRF_REG_EN_RXADDR                   0x02U
#define NRF_REG_SETUP_AW                     0x03U
#define NRF_REG_SETUP_RETR                   0x04U
#define NRF_REG_RF_CH                        0x05U
#define NRF_REG_RF_SETUP                     0x06U
#define NRF_REG_STATUS                       0x07U
#define NRF_REG_OBSERVE_TX                   0x08U
#define NRF_REG_RX_ADDR_P0                   0x0AU

```

```

#define NRF_REG_RX_ADDR_P1          0x0BU
#define NRF_REG_RX_ADDR_P2          0x0CU
#define NRF_REG_RX_ADDR_P3          0x0DU
#define NRF_REG_RX_ADDR_P4          0x0EU
#define NRF_REG_RX_ADDR_P5          0x0FU
#define NRF_REG_TX_ADDR              0x10U
#define NRF_REG_RX_PW_P0             0x11U
#define NRF_REG_RX_PW_P1             0x12U
#define NRF_REG_RX_PW_P2             0x13U
#define NRF_REG_RX_PW_P3             0x14U
#define NRF_REG_RX_PW_P4             0x15U
#define NRF_REG_RX_PW_P5             0x16U
#define NRF_REG_FIFO_STATUS          0x17U
#define NRF_REG_DYNPD                 0x1CU
#define NRF_REG_FEATURE               0x1DU

#define NRF_STATUS_RX_DR              0x40U
#define NRF_STATUS_TX_DS              0x20U
#define NRF_STATUS_MAX_RT             0x10U
#define NRF_STATUS_RX_P_NO_MASK       0x0EU
#define NRF_STATUS_RX_P_NO_SHIFT      1U
#define NRF_STATUS_TX_FULL            0x01U

#define NRF_CONFIG_MASK_RX_DR         0x40U
#define NRF_CONFIG_MASK_TX_DS         0x20U
#define NRF_CONFIG_MASK_MAX_RT        0x10U
#define NRF_CONFIG_EN_CRC             0x08U
#define NRF_CONFIG_CRCO               0x04U
#define NRF_CONFIG_PWR_UP             0x02U
#define NRF_CONFIG_PRIM_RX            0x01U

#define NRF_FIFO_RX_EMPTY              0x01U
#define NRF_FIFO_TX_EMPTY              0x10U

#define NRF_FEATURE_EN_DYN_ACK         0x01U

```

```

/*          ===== Diagnostics
===== */

volatile uint32_t g_radio_timestamp_rx_count = 0U;
volatile uint32_t g_radio_grouped_event_count = 0U;
volatile uint32_t g_radio_nrf2_irq_count = 0U;
volatile uint32_t g_radio_nrf3_irq_count = 0U;
volatile uint32_t g_radio_nrf2_reset_tx_count = 0U;
volatile uint32_t g_radio_nrf2_sync_tx_count = 0U;
volatile uint32_t g_radio_nrf2_health_tx_count = 0U;
volatile uint32_t g_radio_nrf2_tx_ds_count = 0U;
volatile uint32_t g_radio_nrf2_max_rt_count = 0U;
volatile uint8_t  g_radio_nrf2_last_tx_cmd = 0U;
volatile uint8_t  g_radio_nrf2_last_tx_no_ack = 0U;
volatile uint8_t  g_radio_nrf2_last_payload_spi_cmd = 0U;
volatile uint8_t  g_radio_nrf2_status_before_tx = 0U;
volatile uint8_t  g_radio_nrf2_status_after_ce = 0U;
volatile uint8_t  g_radio_nrf2_tx_addr0 = 0U;
volatile uint8_t  g_radio_nrf2_addr_mode = 0U; /* 0=unknown,
1=common sync/reset, 2=health */
volatile uint8_t  g_radio_nrf2_rx_p0_addr0 = 0U;
volatile uint8_t  g_radio_health_current_index = 0xFFU;
volatile uint8_t  g_radio_health_waiting = 0U;
volatile uint8_t  g_radio_health_cycle_active = 0U;
volatile uint8_t  g_radio_health_status_0 = 0U;
volatile uint8_t  g_radio_health_status_1 = 0U;
volatile uint8_t  g_radio_health_status_2 = 0U;
volatile uint8_t  g_radio_health_status_3 = 0U;
volatile uint32_t g_radio_nrf2_common_addr_restore_count = 0U;
volatile uint32_t g_radio_nrf2_sync_deferred_count = 0U;
volatile uint8_t  g_radio_last_event_id = 0U;
volatile uint8_t  g_radio_last_pipe = 0xFFU;
volatile uint8_t  g_radio_health_mask = 0U;
volatile uint8_t  g_radio_nrf2_present = 0U;
volatile uint8_t  g_radio_nrf3_present = 0U;

```

```

volatile uint8_t g_radio_nrf2_last_status = 0xFFU;
volatile uint8_t g_radio_nrf2_last_fifo = 0xFFU;
volatile uint8_t g_radio_nrf2_config = 0xFFU;
volatile uint8_t g_radio_nrf2_observe_tx = 0xFFU;
volatile uint8_t g_radio_nrf2_rf_ch_read = 0xFFU;
volatile uint8_t g_radio_nrf2_setup_aw_read = 0xFFU;
volatile uint8_t g_radio_nrf2_status_nop = 0xFFU;
volatile uint8_t g_radio_nrf2_present_code = 0U;
volatile uint8_t g_radio_nrf2_irq_pin_level = 0xFFU;
volatile uint32_t g_radio_nrf2_spi_error_count = 0U;
volatile uint8_t g_radio_nrf2_status_after_clear = 0xFFU;
volatile uint8_t g_radio_nrf2_feature = 0xFFU;
volatile uint8_t g_radio_nrf2_dynpd = 0xFFU;
volatile uint8_t g_radio_nrf3_last_status = 0xFFU;
volatile uint8_t g_radio_nrf3_last_fifo = 0xFFU;
volatile uint8_t g_radio_nrf3_config = 0xFFU;
volatile uint8_t g_radio_nrf3_observe_tx = 0xFFU;
volatile uint8_t g_radio_nrf3_rf_ch_read = 0xFFU;
volatile uint8_t g_radio_nrf3_setup_aw_read = 0xFFU;
volatile uint8_t g_radio_nrf3_status_nop = 0xFFU;
volatile uint8_t g_radio_nrf3_present_code = 0U;
volatile uint8_t g_radio_nrf3_irq_pin_level = 0xFFU;
volatile uint32_t g_radio_nrf3_spi_error_count = 0U;
volatile uint8_t g_radio_nrf3_status_after_clear = 0xFFU;
volatile uint8_t g_radio_nrf3_feature = 0xFFU;
volatile uint8_t g_radio_nrf3_dynpd = 0xFFU;
volatile uint8_t g_radio_nrf3_rx_dr_seen = 0U;
volatile uint8_t g_radio_nrf3_poll_status = 0xFFU;
volatile uint32_t g_radio_nrf3_poll_rx_dr_count = 0U;
volatile uint32_t g_radio_nrf3_bad_pipe_count = 0U;
volatile uint32_t g_radio_nrf3_flush_count = 0U;
volatile uint32_t g_master_tim2_overflows = 0U;

```

```

/* ===== Internal state
===== */

```

```

typedef struct
{
    SPI_HandleTypeDef *hspi;
    GPIO_TypeDef *csn_port;
    uint16_t csn_pin;
    GPIO_TypeDef *ce_port;
    uint16_t ce_pin;
} nrf_bus_t;

typedef struct
{
    uint8_t active;
    uint8_t event_id;
    uint8_t mask;
    uint32_t deadline_ms;
    uint64_t timestamp_us[MASTER_RADIO_NODE_COUNT];
} master_event_group_t;

static const nrf_bus_t nrf2 = {
    .hspi = &hspi2,
    .csn_port = NRF2_CSN_GPIO_Port,
    .csn_pin = NRF2_CSN_Pin,
    .ce_port = NRF2_CE_GPIO_Port,
    .ce_pin = NRF2_CE_Pin
};

static const nrf_bus_t nrf3 = {
    .hspi = &hspi3,
    .csn_port = NRF3_CSN_GPIO_Port,
    .csn_pin = NRF3_CSN_Pin,
    .ce_port = NRF3_CE_GPIO_Port,
    .ce_pin = NRF3_CE_Pin
};

static volatile uint8_t nrf2_irq_pending = 0U;

```

```

static volatile uint8_t nrf3_irq_pending = 0U;

static      master_event_group_t      event_groups[MASTER_RADIO_NODE_COUNT];

static uint8_t health_status[MASTER_RADIO_NODE_COUNT] = {0U,
0U, 0U, 0U};

static uint8_t health_index = 0U;
static uint8_t health_waiting = 0U;
static uint8_t health_cycle_active = 0U;
static uint32_t health_deadline_ms = 0U;
static uint32_t next_sync_ms = 0U;
static uint32_t next_health_ms = 0U;
static uint32_t sync_start_ms = 0U;
static uint32_t next_debug_snapshot_ms = 0U;

/*      =====                               Byte      helpers
===== */

static void put_u64_le(uint8_t *p, uint64_t v)
{
    p[0] = (uint8_t)(v >> 0);
    p[1] = (uint8_t)(v >> 8);
    p[2] = (uint8_t)(v >> 16);
    p[3] = (uint8_t)(v >> 24);
    p[4] = (uint8_t)(v >> 32);
    p[5] = (uint8_t)(v >> 40);
    p[6] = (uint8_t)(v >> 48);
    p[7] = (uint8_t)(v >> 56);
}

static uint64_t get_u64_le(const uint8_t *p)
{
    return ((uint64_t)p[0] << 0) |
           ((uint64_t)p[1] << 8) |
           ((uint64_t)p[2] << 16) |
           ((uint64_t)p[3] << 24) |

```

```

        ((uint64_t)p[4] << 32) |
        ((uint64_t)p[5] << 40) |
        ((uint64_t)p[6] << 48) |
        ((uint64_t)p[7] << 56);
    }

static uint64_t master_time_us(void)
{
    uint32_t primask = __get_PRIMASK();
    uint32_t hi;
    uint32_t lo;

    __disable_irq();
    hi = g_master_tim2_overflows;
    lo = TIM2->CNT;
    if (((TIM2->SR & TIM_SR_UIF) != 0U) && (lo < 0x80000000UL))
    {
        hi++;
        lo = TIM2->CNT;
    }
    if (primask == 0U)
    {
        __enable_irq();
    }

    return (((uint64_t)hi) << 32) | (uint64_t)lo;
}

/*      ===== nRF      low-level
===== */
static void nrf_csn_low(const nrf_bus_t *dev)
{
    HAL_GPIO_WritePin(dev->csn_port, dev->csn_pin,
GPIO_PIN_RESET);
}

```

```

static void nrf_csn_high(const nrf_bus_t *dev)
{
    HAL_GPIO_WritePin(dev->csn_port, dev->csn_pin,
GPIO_PIN_SET);
}

static void nrf_ce_low(const nrf_bus_t *dev)
{
    HAL_GPIO_WritePin(dev->ce_port, dev->ce_pin, GPIO_PIN_RE-
SET);
}

static void nrf_ce_high(const nrf_bus_t *dev)
{
    HAL_GPIO_WritePin(dev->ce_port, dev->ce_pin,
GPIO_PIN_SET);
}

static uint8_t nrf_xfer(const nrf_bus_t *dev, uint8_t tx)
{
    uint8_t rx = 0xFFU;
    if (HAL_SPI_TransmitReceive(dev->hsapi, &tx, &rx, 1U,
NRF_SPI_TIMEOUT_MS) != HAL_OK)
    {
        if (dev == &nrf2)
        {
            g_radio_nrf2_spi_error_count++;
        }
        else if (dev == &nrf3)
        {
            g_radio_nrf3_spi_error_count++;
        }
        return 0xFFU;
    }
}

```

```

        return rx;
    }

static uint8_t nrf_cmd(const nrf_bus_t *dev, uint8_t cmd)
{
    uint8_t status;
    nrf_csn_low(dev);
    status = nrf_xfer(dev, cmd);
    nrf_csn_high(dev);
    return status;
}

static void nrf_activate_features(const nrf_bus_t *dev)
{
    /* nRF24L01+ FEATURE/DYNPD registers may be locked after
reset on many
    * modules. ACTIVATE 0x73 unlocks EN_DYN_ACK, which is re-
quired for
    * W_TX_PAYLOAD_NO_ACK reset/sync packets sent by NRF2.
Call only once
    * during init: ACTIVATE toggles the feature-bank state on
compatible chips. */
    nrf_csn_low(dev);
    (void)nrf_xfer(dev, NRF_CMD_ACTIVATE);
    (void)nrf_xfer(dev, 0x73U);
    nrf_csn_high(dev);
}

static uint8_t nrf_read_reg(const nrf_bus_t *dev, uint8_t reg);
static void nrf_write_reg(const nrf_bus_t *dev, uint8_t reg,
uint8_t value);
static void nrf_write_feature_checked(const nrf_bus_t *dev,
uint8_t value)
{
    /* ACTIVATE 0x73 toggles FEATURE access. If MCU resets while
nRF stays

```

* powered, an unconditional ACTIVATE can disable FEATURE again. Therefore

* first try writing FEATURE; ACTIVATE only if readback is not equal. */

```
nrf_write_reg(dev, NRF_REG_FEATURE, value);
if (nrf_read_reg(dev, NRF_REG_FEATURE) != value) {
    nrf_activate_features(dev);
    nrf_write_reg(dev, NRF_REG_FEATURE, value);
}
```

```
}
```

```
static uint8_t nrf_get_status_nop(const nrf_bus_t *dev)
```

```
{
```

```
    uint8_t status;
    nrf_csn_low(dev);
    status = nrf_xfer(dev, NRF_CMD_NOP);
    nrf_csn_high(dev);
    return status;
```

```
}
```

```
static uint8_t nrf_read_reg(const nrf_bus_t *dev, uint8_t reg)
```

```
{
```

```
    uint8_t v;
    nrf_csn_low(dev);
    (void)nrf_xfer(dev, (uint8_t)(NRF_CMD_R_REGISTER | (reg &
0x1FU)));
```

```
    v = nrf_xfer(dev, NRF_CMD_NOP);
    nrf_csn_high(dev);
    return v;
```

```
}
```

```
static void nrf_write_reg(const nrf_bus_t *dev, uint8_t reg,
uint8_t value)
```

```
{
```

```
    nrf_csn_low(dev);
```

```

        (void)nrf_xfer(dev, (uint8_t)(NRF_CMD_W_REGISTER | (reg &
0x1FU)));
        (void)nrf_xfer(dev, value);
        nrf_csn_high(dev);
    }

    static void nrf_read_buf(const nrf_bus_t *dev, uint8_t reg,
uint8_t *buf, uint8_t len)
    {
        nrf_csn_low(dev);
        (void)nrf_xfer(dev, (uint8_t)(NRF_CMD_R_REGISTER | (reg &
0x1FU)));
        for (uint8_t i = 0U; i < len; i++)
        {
            buf[i] = nrf_xfer(dev, NRF_CMD_NOP);
        }
        nrf_csn_high(dev);
    }

    static void nrf_write_buf(const nrf_bus_t *dev, uint8_t reg,
const uint8_t *buf, uint8_t len)
    {
        nrf_csn_low(dev);
        (void)nrf_xfer(dev, (uint8_t)(NRF_CMD_W_REGISTER | (reg &
0x1FU)));
        for (uint8_t i = 0U; i < len; i++)
        {
            (void)nrf_xfer(dev, buf[i]);
        }
        nrf_csn_high(dev);
    }

    static void nrf_write_payload(const nrf_bus_t *dev, const
uint8_t *payload, uint8_t len, uint8_t no_ack)
    {

```

```

        uint8_t cmd = no_ack ? NRF_CMD_W_TX_PAYLOAD_NO_ACK :
NRF_CMD_W_TX_PAYLOAD;
        if (dev == &nrf2) {
            g_radio_nrf2_last_payload_spi_cmd = cmd;
        }
        nrf_csn_low(dev);
        (void)nrf_xfer(dev, cmd);
        for (uint8_t i = 0U; i < len; i++)
        {
            (void)nrf_xfer(dev, payload[i]);
        }
        nrf_csn_high(dev);
    }

    static uint8_t nrf_read_payload(const nrf_bus_t *dev, uint8_t
*payload, uint8_t len)
    {
        uint8_t status;
        nrf_csn_low(dev);
        status = nrf_xfer(dev, NRF_CMD_R_RX_PAYLOAD);
        for (uint8_t i = 0U; i < len; i++)
        {
            payload[i] = nrf_xfer(dev, NRF_CMD_NOP);
        }
        nrf_csn_high(dev);
        return status;
    }

    static void nrf_clear_irqs(const nrf_bus_t *dev, uint8_t flags)
    {
        nrf_write_reg(dev, NRF_REG_STATUS, (uint8_t)(flags &
(NRF_STATUS_RX_DR | NRF_STATUS_TX_DS | NRF_STATUS_MAX_RT)));
    }

    static void nrf_clear_all_irqs_and_fifos(const nrf_bus_t *dev)

```

```

{
    nrf_ce_low(dev);
    nrf_write_reg(dev, NRF_REG_STATUS, NRF_STATUS_RX_DR |
NRF_STATUS_TX_DS | NRF_STATUS_MAX_RT);
    (void)nrf_cmd(dev, NRF_CMD_FLUSH_RX);
    (void)nrf_cmd(dev, NRF_CMD_FLUSH_TX);
    nrf_write_reg(dev, NRF_REG_STATUS, NRF_STATUS_RX_DR |
NRF_STATUS_TX_DS | NRF_STATUS_MAX_RT);

    if (dev == &nrf2)
    {
        g_radio_nrf2_status_after_clear = nrf_read_reg(dev,
NRF_REG_STATUS);
    }
    else if (dev == &nrf3)
    {
        g_radio_nrf3_status_after_clear = nrf_read_reg(dev,
NRF_REG_STATUS);
    }
}

static void MasterRadio_UpdateNrfDebugSnapshot(void)
{
    g_radio_nrf2_status_nop = nrf_get_status_nop(&nrf2);
    g_radio_nrf3_status_nop = nrf_get_status_nop(&nrf3);

    g_radio_nrf2_last_status = nrf_read_reg(&nrf2,
NRF_REG_STATUS);
    g_radio_nrf3_last_status = nrf_read_reg(&nrf3,
NRF_REG_STATUS);

    g_radio_nrf2_last_fifo = nrf_read_reg(&nrf2,
NRF_REG_FIFO_STATUS);
    g_radio_nrf3_last_fifo = nrf_read_reg(&nrf3,
NRF_REG_FIFO_STATUS);
}

```

```

g_radio_nrf2_config = nrf_read_reg(&nrf2, NRF_REG_CONFIG);
g_radio_nrf3_config = nrf_read_reg(&nrf3, NRF_REG_CONFIG);

g_radio_nrf2_observe_tx = nrf_read_reg(&nrf2, NRF_REG_OB-
SERVE_TX);
g_radio_nrf3_observe_tx = nrf_read_reg(&nrf3, NRF_REG_OB-
SERVE_TX);

g_radio_nrf2_rf_ch_read      =      nrf_read_reg(&nrf2,
NRF_REG_RF_CH);
g_radio_nrf3_rf_ch_read      =      nrf_read_reg(&nrf3,
NRF_REG_RF_CH);

g_radio_nrf2_setup_aw_read   =      nrf_read_reg(&nrf2,
NRF_REG_SETUP_AW);
g_radio_nrf3_setup_aw_read   =      nrf_read_reg(&nrf3,
NRF_REG_SETUP_AW);
g_radio_nrf2_feature = nrf_read_reg(&nrf2, NRF_REG_FEA-
TURE);
g_radio_nrf3_feature = nrf_read_reg(&nrf3, NRF_REG_FEA-
TURE);
g_radio_nrf2_dynpd = nrf_read_reg(&nrf2, NRF_REG_DYNPD);
g_radio_nrf3_dynpd = nrf_read_reg(&nrf3, NRF_REG_DYNPD);

g_radio_nrf2_irq_pin_level   =      (uint8_t)HAL_GPIO_Read-
Pin(NRF2_IRQ_GPIO_Port, NRF2_IRQ_Pin);
g_radio_nrf3_irq_pin_level   =      (uint8_t)HAL_GPIO_Read-
Pin(NRF3_IRQ_GPIO_Port, NRF3_IRQ_Pin);

g_radio_nrf2_present_code = 0U;
if (g_radio_nrf2_rf_ch_read == NRF2_FREQ) { g_ra-
dio_nrf2_present_code |= 0x01U; }
if (g_radio_nrf2_setup_aw_read == 0x03U) { g_ra-
dio_nrf2_present_code |= 0x02U; }

```

```

        if ((g_radio_nrf2_status_nop != 0xFFU) && (g_radio_nrf2_status_nop != 0x00U)) { g_radio_nrf2_present_code |= 0x04U;
    }

    g_radio_nrf2_present = ((g_radio_nrf2_present_code &
0x03U) == 0x03U) ? 1U : 0U;

    g_radio_nrf3_present_code = 0U;
    if (g_radio_nrf3_rf_ch_read == NRF3_FREQ) { g_radio_nrf3_present_code |= 0x01U; }
    if (g_radio_nrf3_setup_aw_read == 0x03U) { g_radio_nrf3_present_code |= 0x02U; }
    if ((g_radio_nrf3_status_nop != 0xFFU) && (g_radio_nrf3_status_nop != 0x00U)) { g_radio_nrf3_present_code |= 0x04U;
    }

    g_radio_nrf3_present = ((g_radio_nrf3_present_code &
0x03U) == 0x03U) ? 1U : 0U;
    }

    static void nrf2_reset_sync_addr(uint8_t out[NRF_ADDR_LEN]);
    static void nrf2_health_addr(uint8_t slave_index, uint8_t
out[NRF_ADDR_LEN]);

    static void nrf_set_tx_addr(const nrf_bus_t *dev, const uint8_t
addr[NRF_ADDR_LEN])
    {
        nrf_write_buf(dev, NRF_REG_TX_ADDR, addr, NRF_ADDR_LEN);
        /* Required for Enhanced ShockBurst auto-ack on PTX. */
        nrf_write_buf(dev, NRF_REG_RX_ADDR_P0, addr,
NRF_ADDR_LEN);
    }

    static void nrf2_apply_common_sync_addr(void)
    {
        uint8_t addr[NRF_ADDR_LEN];

```

```

    nrf2_reset_sync_addr(addr);
    nrf_set_tx_addr(&nrf2, addr);
    g_radio_nrf2_tx_addr0 = addr[0];
    g_radio_nrf2_rx_p0_addr0 = addr[0];
    g_radio_nrf2_addr_mode = 1U;
    g_radio_nrf2_common_addr_restore_count++;
}

static void nrf2_apply_health_addr(uint8_t slave_index)
{
    uint8_t addr[NRF_ADDR_LEN];

    nrf2_health_addr(slave_index, addr);
    nrf_set_tx_addr(&nrf2, addr);
    g_radio_nrf2_tx_addr0 = addr[0];
    g_radio_nrf2_rx_p0_addr0 = addr[0];
    g_radio_nrf2_addr_mode = 2U;
    g_radio_health_current_index = slave_index;
}

static uint8_t nrf_check_present(const nrf_bus_t *dev)
{
    uint8_t original = nrf_read_reg(dev, NRF_REG_RF_CH);
    uint8_t test = (uint8_t)((original == 0x2AU) ? 0x15U :
0x2AU);
    nrf_write_reg(dev, NRF_REG_RF_CH, test);
    if (nrf_read_reg(dev, NRF_REG_RF_CH) != test)
    {
        return 0U;
    }
    nrf_write_reg(dev, NRF_REG_RF_CH, original);
    return 1U;
}

```

```

/*      ===== Address helpers
===== */

static void nrf3_timestamp_addr(uint8_t slave_index, uint8_t
out[NRF_ADDR_LEN])
{
    const uint8_t base[NRF_ADDR_LEN] =
NRF3_TIMESTAMP_ADDR_BASE;
    memcpy(out, base, NRF_ADDR_LEN);
    out[0] = slave_index;
}

static void nrf2_reset_sync_addr(uint8_t out[NRF_ADDR_LEN])
{
    const uint8_t addr[NRF_ADDR_LEN] = NRF2_RESET_SYNC_ADDR;
    memcpy(out, addr, NRF_ADDR_LEN);
}

static void nrf2_health_addr(uint8_t slave_index, uint8_t
out[NRF_ADDR_LEN])
{
    const uint8_t base[NRF_ADDR_LEN] = NRF2_HEALTH_ADDR_BASE;
    memcpy(out, base, NRF_ADDR_LEN);
    out[0] = (uint8_t)(0x10U + slave_index);
}

/*      ===== nRF configuration
===== */

static void nrf3_init_timestamp_receiver(void)
{
    uint8_t addr[NRF_ADDR_LEN];

    nrf_ce_low(&nrf3);
    HAL_Delay(5U);

    nrf_clear_all_irqs_and_fifos(&nrf3);

```

```

        nrf_write_reg(&nrf3, NRF_REG_CONFIG, (uint8_t)(NRF_CONFIG_EN_CRC | NRF_CONFIG_CRCO | NRF_CONFIG_PWR_UP | NRF_CONFIG_PRIM_RX));

        nrf_write_reg(&nrf3, NRF_REG_EN_AA, 0x0FU); /* ACK for timestamp packets on pipes 0..3. */

        nrf_write_reg(&nrf3, NRF_REG_EN_RXADDR, 0x0FU); /* Pipes 0..3. */

        nrf_write_reg(&nrf3, NRF_REG_SETUP_AW, 0x03U); /* 5-byte addresses. */

        nrf_write_reg(&nrf3, NRF_REG_SETUP_RETR, 0x00U); /* PRX: not used. */

        nrf_write_reg(&nrf3, NRF_REG_RF_CH, NRF3_FREQ);
        nrf_write_reg(&nrf3, NRF_REG_RF_SETUP, 0x0EU); /* 2 Mbps, 0 dBm. */

        nrf_write_reg(&nrf3, NRF_REG_DYNPD, 0x00U);
        nrf_write_reg(&nrf3, NRF_REG_FEATURE, 0x00U);

        nrf3_timestamp_addr(0U, addr);
        nrf_write_buf(&nrf3, NRF_REG_RX_ADDR_P0, addr, NRF_ADDR_LEN);
        nrf3_timestamp_addr(1U, addr);
        nrf_write_buf(&nrf3, NRF_REG_RX_ADDR_P1, addr, NRF_ADDR_LEN);
        nrf3_timestamp_addr(2U, addr);
        nrf_write_reg(&nrf3, NRF_REG_RX_ADDR_P2, addr[0]);
        nrf3_timestamp_addr(3U, addr);
        nrf_write_reg(&nrf3, NRF_REG_RX_ADDR_P3, addr[0]);

        nrf_write_reg(&nrf3, NRF_REG_RX_PW_P0, MASTER_RADIO_EVENT_PAYLOAD_LEN);
        nrf_write_reg(&nrf3, NRF_REG_RX_PW_P1, MASTER_RADIO_EVENT_PAYLOAD_LEN);
        nrf_write_reg(&nrf3, NRF_REG_RX_PW_P2, MASTER_RADIO_EVENT_PAYLOAD_LEN);

```

```

        nrf_write_reg(&nrf3,      NRF_REG_RX_PW_P3,      MASTER_RA-
DIO_EVENT_PAYLOAD_LEN);

        nrf_clear_all_irqs_and_fifos(&nrf3);
        HAL_Delay(2U);
        nrf_ce_high(&nrf3);
    }

static void nrf2_init_sync_transmitter(void)
{
    uint8_t addr[NRF_ADDR_LEN];

    nrf_ce_low(&nrf2);
    HAL_Delay(5U);

    nrf_clear_all_irqs_and_fifos(&nrf2);

    nrf_write_reg(&nrf2,  NRF_REG_CONFIG,  (uint8_t)(NRF_CON-
FIG_EN_CRC | NRF_CONFIG_CRCO | NRF_CONFIG_PWR_UP));
    /* PTX receives ACK on pipe0. Health packets use ACK,
sync/reset use W_TX_PAYLOAD_NO_ACK. */
    nrf_write_reg(&nrf2, NRF_REG_EN_AA, 0x01U);
    nrf_write_reg(&nrf2, NRF_REG_EN_RXADDR, 0x01U); /* Pipe0
for ACK path when PTX. */
    nrf_write_reg(&nrf2, NRF_REG_SETUP_AW, 0x03U); /* 5-byte
addresses. */
    nrf_write_reg(&nrf2, NRF_REG_SETUP_RETR, 0xFFU); /* 4000
us, 15 retries for health check. */
    nrf_write_reg(&nrf2, NRF_REG_RF_CH, NRF2_FREQ);
    nrf_write_reg(&nrf2, NRF_REG_RF_SETUP, 0x0EU); /* 2
Mbps, 0 dBm. */
    nrf_write_feature_checked(&nrf2, NRF_FEATURE_EN_DYN_ACK);
    nrf_write_reg(&nrf2, NRF_REG_DYNPD, 0x00U);

    (void)addr;
}

```

```

    nrf2_apply_common_sync_addr();

    nrf_clear_all_irqs_and_fifos(&nrf2);
    HAL_Delay(2U);
}

/* ===== TX operations ===== */
static void nrf2_send_payload_start(const uint8_t *payload,
uint8_t len, uint8_t no_ack)
{
    nrf_ce_low(&nrf2);
    g_radio_nrf2_status_before_tx = nrf_read_reg(&nrf2,
NRF_REG_STATUS);
    (void)nrf_cmd(&nrf2, NRF_CMD_FLUSH_TX);
    nrf_clear_irqs(&nrf2, NRF_STATUS_TX_DS | NRF_STA-
TUS_MAX_RT);
    g_radio_nrf2_last_tx_no_ack = no_ack;
    if ((payload != NULL) && (len > 0U)) {
        g_radio_nrf2_last_tx_cmd = payload[0];
    }
    nrf_write_payload(&nrf2, payload, len, no_ack);
    nrf_ce_high(&nrf2);
    g_radio_nrf2_status_after_ce = nrf_read_reg(&nrf2,
NRF_REG_STATUS);
}

static void nrf2_finish_tx_pulse(void)
{
    /* nRF24L01+ PTX requires CE high for at least 10 us.
    HAL_Delay(1) is
    * intentionally used here: sync packets are sparse and the
    master has no
    * real-time audio path, while the delay keeps the pulse
    independent of DWT. */

```

```

        HAL_Delay(1U);
        nrf_ce_low(&nrf2);
    }

static void send_reset_time(void)
{
    uint8_t payload[MASTER_RADIO_RESET_PAYLOAD_LEN] = {0U};
    payload[0] = MASTER_CMD_RESET_TIME;

    nrf2_apply_common_sync_addr();
    nrf2_send_payload_start(payload, sizeof(payload), 1U);
    g_radio_nrf2_reset_tx_count++;
    nrf2_finish_tx_pulse();
}

static void send_sync_time(void)
{
    uint8_t payload[MASTER_RADIO_SYNC_PAYLOAD_LEN];

    payload[0] = MASTER_CMD_SYNC_TIME;
    put_u64_le(&payload[1], master_time_us());

    nrf2_apply_common_sync_addr();
    nrf2_send_payload_start(payload, sizeof(payload), 1U);
    g_radio_nrf2_sync_tx_count++;
    nrf2_finish_tx_pulse();
}

static void start_health_check_for_current_slave(void)
{
    uint8_t payload[MASTER_RADIO_HEALTH_PAYLOAD_LEN] = {0U};
    payload[0] = MASTER_CMD_HEALTH_CHECK;

    if (health_index >= MASTER_RADIO_NODE_COUNT)
    {

```

```

        health_waiting = 0U;
        g_radio_health_waiting = 0U;
        return;
    }

    nrf2_apply_health_addr(health_index);
    nrf2_send_payload_start(payload, sizeof(payload), 0U);
    g_radio_nrf2_health_tx_count++;
    nrf2_finish_tx_pulse();
    health_waiting = 1U;
    g_radio_health_waiting = 1U;
    health_deadline_ms = HAL_GetTick() + 90U;
}

/*      ===== Server      sending
===== */
static void send_health_status_to_server(void)
{
    uint8_t frame[TDOA_SERVER_FRAME_LEN];
    tdoa_node_timestamp_t nodes[MASTER_RADIO_NODE_COUNT];
    uint8_t mask = 0U;

    for (uint8_t i = 0U; i < MASTER_RADIO_NODE_COUNT; i++)
    {
        nodes[i].node_id = (uint8_t)(i + 1U);
        nodes[i].timestamp_us = health_status[i] ? UINT64_MAX
: 0ULL;

        if (health_status[i])
        {
            mask |= (uint8_t)(1U << i);
        }
    }

    g_radio_health_mask = mask;
    TDOA_BuildServerFrame(frame, nodes);

```

```

        (void)TDOA_SendFrameToServer(frame);
    }

    static void send_event_group_to_server(const master_event_group_t *group)
    {
        uint8_t frame[TDOA_SERVER_FRAME_LEN];
        tdoa_node_timestamp_t nodes[MASTER_RADIO_NODE_COUNT];

        for (uint8_t i = 0U; i < MASTER_RADIO_NODE_COUNT; i++)
        {
            nodes[i].node_id = (uint8_t)(i + 1U);
            nodes[i].timestamp_us = ((group->mask & (1U << i)) !=
0U) ? group->timestamp_us[i] : 0ULL;
        }
        HAL_GPIO_TogglePin(LED_GPIO_Port, LED_Pin);
        TDOA_BuildServerFrame(frame, nodes);
        if (TDOA_SendFrameToServer(frame) == W5500_PORT_OK)
        {
            g_radio_grouped_event_count++;
        }
    }

    /* ===== Event grouping ===== */
    static master_event_group_t *find_group(uint8_t event_id)
    {
        for (uint8_t i = 0U; i < MASTER_RADIO_NODE_COUNT; i++)
        {
            if (event_groups[i].active &&
(event_groups[i].event_id == event_id))
            {
                return &event_groups[i];
            }
        }
    }

```

```

        return NULL;
    }

static master_event_group_t *allocate_group(uint8_t event_id)
{
    uint32_t now = HAL_GetTick();
    master_event_group_t *oldest = &event_groups[0];

    for (uint8_t i = 0U; i < MASTER_RADIO_NODE_COUNT; i++)
    {
        if (!event_groups[i].active)
        {
            memset(&event_groups[i], 0,
sizeof(event_groups[i]));
            event_groups[i].active = 1U;
            event_groups[i].event_id = event_id;
            event_groups[i].deadline_ms = now + MAS-
TER_EVENT_GROUP_TIMEOUT_MS;
            return &event_groups[i];
        }
        if ((int32_t)(event_groups[i].deadline_ms - oldest-
>deadline_ms) < 0)
        {
            oldest = &event_groups[i];
        }
    }

    send_event_group_to_server(oldest);
    memset(oldest, 0, sizeof(*oldest));
    oldest->active = 1U;
    oldest->event_id = event_id;
    oldest->deadline_ms = now + MASTER_EVENT_GROUP_TIMEOUT_MS;
    return oldest;
}

```

```

static void accept_timestamp_packet(uint8_t pipe, const uint8_t
payload[MASTER_RADIO_EVENT_PAYLOAD_LEN])
{
    uint8_t node_index;
    uint8_t event_id;
    uint64_t timestamp_us;
    master_event_group_t *group;

    if (pipe >= MASTER_RADIO_NODE_COUNT)
    {
        return;
    }

    node_index = pipe;
    timestamp_us = get_u64_le(&payload[0]);
    event_id = payload[8];

    group = find_group(event_id);
    if (group == NULL)
    {
        group = allocate_group(event_id);
    }

    group->timestamp_us[node_index] = timestamp_us;
    group->mask |= (uint8_t)(1U << node_index);
    group->deadline_ms = HAL_GetTick() + MAS-
TER_EVENT_GROUP_TIMEOUT_MS;

    g_radio_timestamp_rx_count++;
    g_radio_last_event_id = event_id;
    g_radio_last_pipe = pipe;

    if (group->mask == 0x0FU)
    {
        send_event_group_to_server(group);
    }
}

```

```

        memset(group, 0, sizeof(*group));
    }
}

static void service_event_timeouts(void)
{
    uint32_t now = HAL_GetTick();
    for (uint8_t i = 0U; i < MASTER_RADIO_NODE_COUNT; i++)
    {
        if (event_groups[i].active && ((int32_t)(now -
event_groups[i].deadline_ms) >= 0))
        {
            send_event_group_to_server(&event_groups[i]);
            memset(&event_groups[i], 0,
sizeof(event_groups[i]));
        }
    }
}

/* ===== IRQ service
===== */
static void service_nrf3_timestamp_irq(void)
{
    uint8_t status = nrf_read_reg(&nrf3, NRF_REG_STATUS);
    g_radio_nrf3_last_status = status;
    g_radio_nrf3_last_fifo = nrf_read_reg(&nrf3,
NRF_REG_FIFO_STATUS);

    while ((status & NRF_STATUS_RX_DR) != 0U)
    {
        uint8_t pipe = (uint8_t)((status & NRF_STA-
TUS_RX_P_NO_MASK) >> NRF_STATUS_RX_P_NO_SHIFT);
        uint8_t payload[MASTER_RADIO_EVENT_PAYLOAD_LEN];

        if (pipe <= 3U)

```

```

        {
            g_radio_nrf3_rx_dr_seen = 1U;
            (void)nrf_read_payload(&nrf3, payload, sizeof(payload));

            accept_timestamp_packet(pipe, payload);
        }
        else
        {
            g_radio_nrf3_bad_pipe_count++;
            g_radio_nrf3_flush_count++;
            (void)nrf_cmd(&nrf3, NRF_CMD_FLUSH_RX);
            break;
        }

        nrf_clear_irqs(&nrf3, NRF_STATUS_RX_DR);
        status = nrf_read_reg(&nrf3, NRF_REG_STATUS);
        g_radio_nrf3_last_status = status;
        g_radio_nrf3_last_fifo = nrf_read_reg(&nrf3,
NRF_REG_FIFO_STATUS);
        if ((g_radio_nrf3_last_fifo & NRF_FIFO_RX_EMPTY) != 0U)
        {
            break;
        }
    }

    nrf_clear_irqs(&nrf3, NRF_STATUS_RX_DR | NRF_STATUS_TX_DS
| NRF_STATUS_MAX_RT);
}

static void service_nrf2_tx_irq(void)
{
    uint8_t status = nrf_read_reg(&nrf2, NRF_REG_STATUS);

    if (health_waiting != 0U)
    {

```

```

if ((status & NRF_STATUS_TX_DS) != 0U)
{
    g_radio_nrf2_tx_ds_count++;
    health_status[health_index] = 1U;
    if (health_index == 0U) { g_radio_health_status_0
= 1U; }

    else if (health_index == 1U) { g_radio_health_sta-
tus_1 = 1U; }

    else if (health_index == 2U) { g_radio_health_sta-
tus_2 = 1U; }

    else if (health_index == 3U) { g_radio_health_sta-
tus_3 = 1U; }

    nrf_clear_irqs(&nrf2, NRF_STATUS_TX_DS);
    health_index++;
    health_waiting = 0U;
    g_radio_health_waiting = 0U;
}
else if ((status & NRF_STATUS_MAX_RT) != 0U)
{
    g_radio_nrf2_max_rt_count++;
    health_status[health_index] = 0U;
    if (health_index == 0U) { g_radio_health_status_0
= 0U; }

    else if (health_index == 1U) { g_radio_health_sta-
tus_1 = 0U; }

    else if (health_index == 2U) { g_radio_health_sta-
tus_2 = 0U; }

    else if (health_index == 3U) { g_radio_health_sta-
tus_3 = 0U; }

    nrf_clear_irqs(&nrf2, NRF_STATUS_MAX_RT);
    (void)nrf_cmd(&nrf2, NRF_CMD_FLUSH_TX);
    health_index++;
    health_waiting = 0U;
    g_radio_health_waiting = 0U;
}

```

```

    }
    else
    {
        nrf_clear_irqs(&nrf2, (uint8_t)(status & (NRF_STA-
TUS_TX_DS | NRF_STATUS_MAX_RT | NRF_STATUS_RX_DR)));
        if ((status & NRF_STATUS_TX_DS) != 0U)
        {
            g_radio_nrf2_tx_ds_count++;
        }
        if ((status & NRF_STATUS_MAX_RT) != 0U)
        {
            g_radio_nrf2_max_rt_count++;
            (void)nrf_cmd(&nrf2, NRF_CMD_FLUSH_TX);
        }
    }
}

static void health_mark_current(uint8_t ok)
{
    if (health_index < MASTER_RADIO_NODE_COUNT)
    {
        health_status[health_index] = ok;
        if (health_index == 0U) { g_radio_health_status_0 = ok;
    }

        else if (health_index == 1U) { g_radio_health_status_1
= ok; }

        else if (health_index == 2U) { g_radio_health_status_2
= ok; }

        else if (health_index == 3U) { g_radio_health_status_3
= ok; }

    }
}

static void service_health_fsm(void)
{

```

```

uint32_t now = HAL_GetTick();

if (health_cycle_active == 0U)
{
    return;
}

if (health_waiting != 0U)
{
    uint8_t status = nrf_read_reg(&nrf2, NRF_REG_STATUS);
    g_radio_nrf2_last_status = status;

    if ((status & NRF_STATUS_TX_DS) != 0U)
    {
        g_radio_nrf2_tx_ds_count++;
        health_mark_current(1U);
        nrf_clear_irqs(&nrf2, NRF_STATUS_TX_DS | NRF_STA-
TUS_MAX_RT);

        (void)nrf_cmd(&nrf2, NRF_CMD_FLUSH_TX);
        health_index++;
        health_waiting = 0U;
        g_radio_health_waiting = 0U;
        return;
    }

    if ((status & NRF_STATUS_MAX_RT) != 0U)
    {
        g_radio_nrf2_max_rt_count++;
        health_mark_current(0U);
        nrf_clear_irqs(&nrf2, NRF_STATUS_MAX_RT | NRF_STA-
TUS_TX_DS);

        (void)nrf_cmd(&nrf2, NRF_CMD_FLUSH_TX);
        health_index++;
        health_waiting = 0U;
        g_radio_health_waiting = 0U;
    }
}

```

```

        return;
    }

    if ((int32_t)(now - health_deadline_ms) >= 0)
    {
        health_mark_current(0U);
        nrf_clear_irqs(&nrf2, NRF_STATUS_MAX_RT | NRF_STA-
TUS_TX_DS);

        (void)nrf_cmd(&nrf2, NRF_CMD_FLUSH_TX);
        health_index++;
        health_waiting = 0U;
        g_radio_health_waiting = 0U;
        return;
    }

    return;
}

if (health_index < MASTER_RADIO_NODE_COUNT)
{
    start_health_check_for_current_slave();
    return;
}

send_health_status_to_server();
nrf2_apply_common_sync_addr();
health_index = MASTER_RADIO_NODE_COUNT;
health_waiting = 0U;
health_cycle_active = 0U;
health_cycle_active = 0U;
g_radio_health_current_index = 0xFFU;
g_radio_health_waiting = 0U;
g_radio_health_cycle_active = 0U;
next_health_ms = HAL_GetTick() + MASTER_HEALTH_PERIOD_MS;
}

```

```

/*      ===== Public API
===== */

void MasterRadio_Init(void)
{
    HAL_GPIO_WritePin(NRF2_CSN_GPIO_Port,    NRF2_CSN_Pin,
GPIO_PIN_SET);
    HAL_GPIO_WritePin(NRF3_CSN_GPIO_Port,    NRF3_CSN_Pin,
GPIO_PIN_SET);
    HAL_GPIO_WritePin(NRF2_CE_GPIO_Port,      NRF2_CE_Pin,
GPIO_PIN_RESET);
    HAL_GPIO_WritePin(NRF3_CE_GPIO_Port,      NRF3_CE_Pin,
GPIO_PIN_RESET);
    HAL_Delay(10U);

    /* Basic SPI smoke-test before full configuration. The fi-
nal presence flags
    * are updated below from the configured RF_CH and SETUP_AW
registers. */
    (void)nrf_check_present(&nrf2);
    (void)nrf_check_present(&nrf3);

    nrf2_init_sync_transmitter();
    nrf3_init_timestamp_receiver();
    MasterRadio_UpdateNrfDebugSnapshot();

    memset(event_groups, 0, sizeof(event_groups));
    memset(health_status, 0, sizeof(health_status));
    g_radio_health_mask = 0U;
    g_radio_health_status_0 = 0U;
    g_radio_health_status_1 = 0U;
    g_radio_health_status_2 = 0U;
    g_radio_health_status_3 = 0U;
    g_radio_health_current_index = 0xFFU;
    g_radio_health_waiting = 0U;

```

```

g_radio_health_cycle_active = 0U;

send_reset_time();

sync_start_ms = HAL_GetTick() + MASTER_SYNC_START_DE-
LAY_MS;
next_sync_ms = sync_start_ms;
next_health_ms = HAL_GetTick() + MASTER_HEALTH_PERIOD_MS;
next_debug_snapshot_ms = HAL_GetTick() + 250U;
health_index = MASTER_RADIO_NODE_COUNT;
health_waiting = 0U;
health_cycle_active = 0U;
}

void MasterRadio_OnTimerPeriodElapsed(TIM_HandleTypeDef *htim)
{
    if (htim->Instance == TIM2)
    {
        g_master_tim2_overflows++;
    }
}

void MasterRadio_OnExti(uint16_t GPIO_Pin)
{
    if (GPIO_Pin == NRF2_IRQ_Pin)
    {
        nrf2_irq_pending = 1U;
        g_radio_nrf2_irq_count++;
    }
    else if (GPIO_Pin == NRF3_IRQ_Pin)
    {
        nrf3_irq_pending = 1U;
        g_radio_nrf3_irq_count++;
    }
}

```

```

void MasterRadio_Service(void)
{
    uint32_t now = HAL_GetTick();
    uint8_t need_nrf3_service = 0U;

    if (nrf3_irq_pending != 0U)
    {
        nrf3_irq_pending = 0U;
        need_nrf3_service = 1U;
    }

    /* Fallback: if EXTI edge was missed or IRQ line was already
low before
        * enabling NVIC, still service NRF3. This is safe on master
because there
        * is no real-time audio DMA path here. */
    if (HAL_GPIO_ReadPin(NRF3_IRQ_GPIO_Port, NRF3_IRQ_Pin) ==
GPIO_PIN_RESET)
    {
        need_nrf3_service = 1U;
    }

    if (need_nrf3_service == 0U)
    {
        uint8_t st = nrf_read_reg(&nrf3, NRF_REG_STATUS);
        g_radio_nrf3_poll_status = st;
        if ((st & NRF_STATUS_RX_DR) != 0U)
        {
            g_radio_nrf3_poll_rx_dr_count++;
            need_nrf3_service = 1U;
        }
    }

    if (need_nrf3_service != 0U)

```

```

    {
        service_nrf3_timestamp_irq();
    }

    if (nrf2_irq_pending != 0U)
    {
        nrf2_irq_pending = 0U;
        if (health_cycle_active == 0U)
        {
            service_nrf2_tx_irq();
        }
    }

    service_event_timeouts();

    if ((int32_t)(now - next_debug_snapshot_ms) >= 0)
    {
        MasterRadio_UpdateNrfDebugSnapshot();
        next_debug_snapshot_ms = now + 250U;
    }

    if ((int32_t)(now - sync_start_ms) >= 0)
    {
        if ((int32_t)(now - next_sync_ms) >= 0)
        {
            if ((health_waiting == 0U) && (health_index >= MASTER_RADIO_NODE_COUNT))
            {
                send_sync_time();
                next_sync_ms = now + MASTER_SYNC_PERIOD_MS;
            }
            else
            {
                /* Do not rewrite TX_ADDR/RX_ADDR_P0 while a
health-check ACK

```

```

        * transaction is active. A deferred sync will
be sent as soon

        * as the health cycle finishes and common ad-
dress is restored. */
        g_radio_nrf2_sync_deferred_count++;
        next_sync_ms = now + MASTER_SYNC_PERIOD_MS;
    }
}

if (health_cycle_active != 0U)
{
    service_health_fsm();
}
else if ((int32_t)(now - next_health_ms) >= 0)
{
    health_cycle_active = 1U;
    g_radio_health_cycle_active = 1U;
    health_index = 0U;
    health_waiting = 0U;
    g_radio_health_waiting = 0U;
    memset(health_status, 0, sizeof(health_status));
    g_radio_health_status_0 = 0U;
    g_radio_health_status_1 = 0U;
    g_radio_health_status_2 = 0U;
    g_radio_health_status_3 = 0U;
    service_health_fsm();
}
}/*

```

Модуль w5500_port.c ведущего модуля.

```

#include "w5500_port.h"

#include "spi.h"
#include "wizchip_conf.h"
#include "socket.h"

```

```

#include "w5500.h"

#include <stddef.h>
#include <string.h>

extern SPI_HandleTypeDef hspi1;

static volatile uint8_t g_w5500_int_flag = 0U;

#define W5500_SPI_TIMEOUT_MS      100U
#define W5500_SPI_BURST_CHUNK    64U
#define W5500_RESET_READY_DELAY_MS 150U
#define W5500_VERSION_RETRY_COUNT 5U
#define W5500_VERSION_RETRY_DELAY_MS 5U

static void W5500_Select(void)
{
    HAL_GPIO_WritePin(W5500_CS_GPIO_Port, W5500_CS_Pin,
GPIO_PIN_RESET);
}

static void W5500_Unselect(void)
{
    HAL_GPIO_WritePin(W5500_CS_GPIO_Port, W5500_CS_Pin,
GPIO_PIN_SET);
}

static uint8_t W5500_ReadByte(void)
{
    const uint8_t tx = 0xFFU;
    uint8_t rx = 0x00U;

    if (HAL_SPI_TransmitReceive(&hspi1, &tx, &rx, 1U,
W5500_SPI_TIMEOUT_MS) != HAL_OK)
    {

```

```

        return 0x00U;
    }

    return rx;
}

static void W5500_WriteByte(uint8_t byte)
{
    uint8_t rx_dummy = 0U;
    (void)HAL_SPI_TransmitReceive(&hspi1, &byte, &rx_dummy,
1U, W5500_SPI_TIMEOUT_MS);
}

static void W5500_ReadBurst(uint8_t *buffer, uint16_t len)
{
    uint8_t tx_dummy[W5500_SPI_BURST_CHUNK];

    if ((buffer == NULL) || (len == 0U))
    {
        return;
    }

    memset(tx_dummy, 0xFF, sizeof(tx_dummy));

    while (len > 0U)
    {
        uint16_t chunk = (len > W5500_SPI_BURST_CHUNK) ?
W5500_SPI_BURST_CHUNK : len;

        if (HAL_SPI_TransmitReceive(&hspi1, tx_dummy, buffer,
chunk, W5500_SPI_TIMEOUT_MS) != HAL_OK)
        {
            memset(buffer, 0, chunk);
            return;
        }
    }
}

```

```

        buffer += chunk;
        len -= chunk;
    }
}

static void W5500_WriteBurst(uint8_t *buffer, uint16_t len)
{
    uint8_t rx_dummy[W5500_SPI_BURST_CHUNK];

    if ((buffer == NULL) || (len == 0U))
    {
        return;
    }

    while (len > 0U)
    {
        uint16_t chunk = (len > W5500_SPI_BURST_CHUNK) ?
W5500_SPI_BURST_CHUNK : len;

        if (HAL_SPI_TransmitReceive(&hspi1, buffer, rx_dummy,
chunk, W5500_SPI_TIMEOUT_MS) != HAL_OK)
        {
            return;
        }

        buffer += chunk;
        len -= chunk;
    }
}

static uint8_t W5500_ReadVersionRaw(void)
{
    uint8_t tx[4] = {0x00U, 0x39U, 0x00U, 0xFFU};
    uint8_t rx[4] = {0U, 0U, 0U, 0U};

```

```

        W5500_Select();
        if (HAL_SPI_TransmitReceive(&hspi1, tx, rx,
(uint16_t)sizeof(tx), W5500_SPI_TIMEOUT_MS) != HAL_OK)
        {
            W5500_Unselect();
            return 0U;
        }
        W5500_Unselect();

        return rx[3];
    }

static void put_u32_be(uint8_t *p, uint32_t v)
{
    p[0] = (uint8_t)(v >> 24);
    p[1] = (uint8_t)(v >> 16);
    p[2] = (uint8_t)(v >> 8);
    p[3] = (uint8_t)(v >> 0);
}

static void put_u64_le(uint8_t *p, uint64_t v)
{
    p[0] = (uint8_t)(v >> 0);
    p[1] = (uint8_t)(v >> 8);
    p[2] = (uint8_t)(v >> 16);
    p[3] = (uint8_t)(v >> 24);
    p[4] = (uint8_t)(v >> 32);
    p[5] = (uint8_t)(v >> 40);
    p[6] = (uint8_t)(v >> 48);
    p[7] = (uint8_t)(v >> 56);
}

static uint8_t W5500_WaitSocketCommandDone(uint8_t sn,
uint32_t timeout_ms)

```

```

{
    uint32_t start = HAL_GetTick();

    while (getSn_CR(sn) != 0U)
    {
        if ((HAL_GetTick() - start) >= timeout_ms)
        {
            return 0U;
        }
    }

    return 1U;
}

static uint8_t W5500_SpiReadWriteSelfTest(void)
{
    uint8_t simr_original = getSIMR();
    uint8_t simr_test = (uint8_t)(simr_original ^ 0x01U);

    setSIMR(simr_test);
    if (getSIMR() != simr_test)
    {
        setSIMR(simr_original);
        return 0U;
    }

    setSIMR(simr_original);
    if (getSIMR() != simr_original)
    {
        return 0U;
    }

    return 1U;
}

```

```

static w5500_port_status_t W5500_OpenTcpClientSocket(uint8_t
sn)
{
    close(sn);
    (void)W5500_WaitSocketCommandDone(sn, 100U);

    if (socket(sn, Sn_MR_TCP, W5500_LOCAL_TCP_PORT, 0U) != sn)
    {
        return W5500_PORT_SOCKET_ERROR;
    }

    uint32_t start = HAL_GetTick();
    while (getSn_SR(sn) != SOCK_INIT)
    {
        if ((HAL_GetTick() - start) >= 1000U)
        {
            close(sn);
            return W5500_PORT_SOCKET_ERROR;
        }
    }

    return W5500_PORT_OK;
}

void W5500_Reset(void)
{
    HAL_GPIO_WritePin(W5500_RST_GPIO_Port, W5500_RST_Pin,
GPIO_PIN_RESET);
    HAL_Delay(10U);
    HAL_GPIO_WritePin(W5500_RST_GPIO_Port, W5500_RST_Pin,
GPIO_PIN_SET);
    HAL_Delay(W5500_RESET_READY_DELAY_MS);
}

uint8_t W5500_ReadVersion(void)

```

```

{
    uint8_t version = W5500_ReadVersionRaw();

    if (version == W5500_VERSION_VALUE)
    {
        return version;
    }

    version = getVERSIONR();
    if (version == W5500_VERSION_VALUE)
    {
        return version;
    }

    for (uint8_t i = 0U; i < W5500_VERSION_RETRY_COUNT; i++)
    {
        HAL_Delay(W5500_VERSION_RETRY_DELAY_MS);
        version = W5500_ReadVersionRaw();

        if (version == W5500_VERSION_VALUE)
        {
            break;
        }
    }

    return version;
}

void W5500_Port_Init(void)
{
    HAL_GPIO_WritePin(W5500_CS_GPIO_Port, W5500_CS_Pin,
GPIO_PIN_SET);

    W5500_Reset();
}

```

```

        reg_wizchip_cs_cbfunc(W5500_Select, W5500_Unselect);
        reg_wizchip_spi_cbfunc(W5500_ReadByte, W5500_WriteByte);
        reg_wizchip_spiburst_cbfunc(W5500_ReadBurst,
W5500_WriteBurst);
    }

w5500_port_status_t W5500_NetworkInit_Static(void)
{
    uint8_t tx_size[8] = {2U, 2U, 2U, 2U, 2U, 2U, 2U, 2U};
    uint8_t rx_size[8] = {2U, 2U, 2U, 2U, 2U, 2U, 2U, 2U};

    wiz_NetInfo net_info = {
        .mac = {W5500_MAC0, W5500_MAC1, W5500_MAC2, W5500_MAC3,
W5500_MAC4, W5500_MAC5},
        .ip  = {W5500_IP0, W5500_IP1, W5500_IP2, W5500_IP3},
        .sn  = {W5500_NETMASK0, W5500_NETMASK1, W5500_NETMASK2,
W5500_NETMASK3},
        .gw  = {W5500_GATEWAY0, W5500_GATEWAY1, W5500_GATEWAY2,
W5500_GATEWAY3},
        .dns   =   {W5500_DNS0,      W5500_DNS1,      W5500_DNS2,
W5500_DNS3},
        .dhcp = NETINFO_STATIC
    };

    if (wizchip_init(tx_size, rx_size) != 0)
    {
        return W5500_PORT_SPI_ERROR;
    }

    wizchip_setnetinfo(&net_info);

    /* Enable socket 0 interrupts on W5500 INTn pin. */
    setSIMR(0x01U);
    setSn_IMR(TDOA_W5500_SOCKET,
              (uint8_t)(Sn_IR_CON |

```

```

        Sn_IR_DISCON |
        Sn_IR_RECV |
        Sn_IR_TIMEOUT |
        Sn_IR_SENDOK) );

    return W5500_PORT_OK;
}

w5500_port_status_t W5500_Init(void)
{
    W5500_Port_Init();

    if (W5500_ReadVersion() != W5500_VERSION_VALUE)
    {
        return W5500_PORT_VERSION_ERROR;
    }

    if (W5500_SpiReadWriteSelfTest() == 0U)
    {
        return W5500_PORT_SPI_ERROR;
    }

    return W5500_NetworkInit_Static();
}

void W5500_SetInterruptFlag(void)
{
    g_w5500_int_flag = 1U;
}

void W5500_Service(void)
{
    if (g_w5500_int_flag == 0U)
    {
        return;
    }
}

```

```

    }

    g_w5500_int_flag = 0U;

    uint8_t sir = getSIR();

    if ((sir & 0x01U) != 0U)
    {
        uint8_t sn_ir = getSn_IR(W5500_TEST_SOCKET);

        /* Clear W5500 socket interrupt bits by writing 1 to
the asserted bits. */
        if (sn_ir != 0U)
        {
            setSn_IR(W5500_TEST_SOCKET, sn_ir);
        }
    }
}

void TDOA_BuildServerFrame(uint8_t out[TDOA_SERVER_FRAME_LEN],
                           const          tdoa_node_timestamp_t
nodes[4])
{
    if ((out == NULL) || (nodes == NULL))
    {
        return;
    }

    memset(out, 0, TDOA_SERVER_FRAME_LEN);

    /* Java server reads first 4 bytes as ByteBuffer.getInt(),
therefore Big-Endian. */
    put_u32_be(&out[0], TDOA_SERVER_PAYLOAD_LEN);

```

```

        /* Payload format: 4 x (node_id:uint8 + timestamp_us:uint64
little-endian). */
        out[4] = nodes[0].node_id;
        put_u64_le(&out[5], nodes[0].timestamp_us);

        out[13] = nodes[1].node_id;
        put_u64_le(&out[14], nodes[1].timestamp_us);

        out[22] = nodes[2].node_id;
        put_u64_le(&out[23], nodes[2].timestamp_us);

        out[31] = nodes[3].node_id;
        put_u64_le(&out[32], nodes[3].timestamp_us);
    }

    void                                TDOA_BuildTestServerFrame(uint8_t
out[TDOA_SERVER_FRAME_LEN])
    {
        const tdoa_node_timestamp_t test_nodes[4] = {
            {.node_id = 1U, .timestamp_us = 206225ULL},
            {.node_id = 2U, .timestamp_us = 196802ULL},
            {.node_id = 3U, .timestamp_us = 204544ULL},
            {.node_id = 4U, .timestamp_us = 224370ULL}
        };

        TDOA_BuildServerFrame(out, test_nodes);
    }

    w5500_port_status_t    TDOA_SendFrameToServer(const    uint8_t
frame[TDOA_SERVER_FRAME_LEN])
    {
        uint8_t server_ip[4] = {TDOA_SERVER_IP0, TDOA_SERVER_IP1,
TDOA_SERVER_IP2, TDOA_SERVER_IP3};
        uint8_t sn = W5500_TEST_SOCKET;
        int32_t sent;

```

```

    if (frame == NULL)
    {
        return W5500_PORT_SEND_ERROR;
    }

    w5500_port_status_t    status    =    W5500_OpenTcpClien-
tSocket(sn);
    if (status != W5500_PORT_OK)
    {
        return status;
    }

    if (connect(sn, server_ip, TDOA_SERVER_PORT) != SOCK_OK)
    {
        close(sn);
        return W5500_PORT_SOCKET_ERROR;
    }

    uint32_t start = HAL_GetTick();
    while (getSn_SR(sn) != SOCK_ESTABLISHED)
    {
        uint8_t sn_ir = getSn_IR(sn);
        if ((sn_ir & Sn_IR_TIMEOUT) != 0U)
        {
            setSn_IR(sn, Sn_IR_TIMEOUT);
            close(sn);
            return W5500_PORT_CONNECT_TIMEOUT;
        }

        if ((HAL_GetTick() - start) >= 3000U)
        {
            close(sn);
            return W5500_PORT_CONNECT_TIMEOUT;
        }
    }

```

```

}

sent = send(sn, (uint8_t *)frame, TDOA_SERVER_FRAME_LEN);
if (sent != (int32_t)TDOA_SERVER_FRAME_LEN)
{
    disconnect(sn);
    close(sn);
    return W5500_PORT_SEND_ERROR;
}

start = HAL_GetTick();
while ((getSn_IR(sn) & Sn_IR_SENDOK) == 0U)
{
    uint8_t sn_ir = getSn_IR(sn);
    if ((sn_ir & Sn_IR_TIMEOUT) != 0U)
    {
        setSn_IR(sn, Sn_IR_TIMEOUT);
        disconnect(sn);
        close(sn);
        return W5500_PORT_SEND_ERROR;
    }

    if ((HAL_GetTick() - start) >= 1000U)
    {
        disconnect(sn);
        close(sn);
        return W5500_PORT_SEND_ERROR;
    }
}

setSn_IR(sn, Sn_IR_SENDOK);

disconnect(sn);
start = HAL_GetTick();
while (getSn_SR(sn) != SOCK_CLOSED)
{

```

```

        if ((HAL_GetTick() - start) >= 1000U)
        {
            close(sn);
            break;
        }
    }

    return W5500_PORT_OK;
}

w5500_port_status_t TDOA_SendTestPacketToServer(void)
{
    uint8_t frame[TDOA_SERVER_FRAME_LEN];

    TDOA_BuildTestServerFrame(frame);
    return TDOA_SendFrameToServer(frame);
}

```

ПРИЛОЖЕНИЕ Б.

Графическая часть выпускной квалификационной работы

В графическую часть выпускной квалификационной работы входят:

- сводка технических решений и их обоснования;
- блок-схемы алгоритмов ведущего модуля;
- блок-схемы алгоритмов ведомого модуля;
- структуры данных программно-аппаратного комплекса;
- чертеж схемы электрической принципиальной ведущего модуля;
- чертеж схемы электрической принципиальной ведомого модуля;
- схемы натурных экспериментов;
- результаты натурных экспериментов;